



**Universidad
de Valladolid**

Escuela de Ingeniería Informática

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Ingeniería de Software

**Aplicación móvil para la información y
geolocalización de procesiones y eventos
en semana santa.**

Autor:

Elena Martínez Vázquez

Tutores:

Juan Alberto Muñoz Cristóbal
Pablo Federico Sanchez Mayoral

Dedicatoria

Agradecimientos

Resumen

Abstract

Índice general

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	2
1.3. Estado de la cuestión	2
1.3.1. Aplicaciones actuales de Semana Santa	3
1.3.2. Aplicaciones de seguimiento en tiempo real	5
1.4. Objetivos	6
1.5. Estructura de la memoria	7
2. Planificación	9
2.1. Metodología	9
2.2. Planificación, Fases e iteraciones	9
2.2.1. Planificación inicial	9
2.2.2. Fase de análisis	10
2.2.3. Fase de iteraciones	11
2.2.4. Fase final: documentación y entrega	12
2.3. Análisis de riesgos	12
2.3.1. Planificación de riesgo	13
2.4. Seguimiento del proyecto	13
2.4.1. Monitorización de riesgos	15
2.5. Estimación de costes	16
2.5.1. Costes de personal	16
2.5.2. Costes de equipo y amortización	16
2.5.3. Costes de licencias de software	17
2.5.4. Otros costes indirectos	18
2.5.5. Coste total	18
2.5.6. Coste estimado de la infraestructura en producción	18
3. Análisis	21
3.1. Actores del sistema	21
3.1.1. Ciudadano	21
3.1.2. Cofrade	21
3.1.3. Junta de Cofradías	22
3.1.4. Administrador	22

3.2. Requisitos	23
3.2.1. Requisitos funcionales	23
3.2.2. Requisitos de información	26
3.2.3. Requisitos no funcionales	28
3.3. Casos de uso	29
3.4. Modelo de dominio	31
3.4.1. Modelo de interacción	37
4. Descripción de las iteraciones	43
4.1. 1ª Iteración	43
4.1.1. Análisis	43
4.1.2. Diseño	43
4.1.3. Implementacion	63
4.1.4. Pruebas	63
4.2. 2ª Iteración	64
4.2.1. Análisis	64
4.2.2. Diseño	64
4.2.3. Implementacion	64
4.2.4. Pruebas	64
4.3. 3ª Iteración	65
4.3.1. Análisis	65
4.3.2. Diseño	65
4.3.3. Implementacion	65
4.3.4. Pruebas	65
4.4. 4ª Iteración	66
4.4.1. Análisis	66
4.4.2. Diseño	66
4.4.3. Implementacion	66
4.4.4. Pruebas	66
4.5. 5ª Iteración	67
4.5.1. Análisis	67
4.5.2. Diseño	67
4.5.3. Implementacion	67
4.5.4. Pruebas	67
5. Estado final de la aplicación	69
5.0.1. Historias de usuario	69
6. Pruebas	71
7. Conclusiones	73
7.1. Trabajo futuro	73
7.2. Puntos débiles	73

Bibliografía	77
Anexos	83
A. Acrónimos	83
B. Casos de uso adicionales	85
C. Esquema de la base de datos	103
C.1. Ciudad (ciudades)	103
C.2. Junta de Cofradías (juntas_cofradias)	103
C.3. Cofradía / Hermandad (cofradias)	104
C.4. Paso (pasos)	104
C.4.1. Paso-Procesión (pasos_procesiones) — tabla intermedia N:M	105
C.5. Código de Acceso (codigos_acceso)	105
C.6. Ubicación (ubicaciones)	105
C.7. Evento (eventos)	105
C.7.1. Evento-Cofradía (eventos_cofradias) — tabla intermedia N:M	106
C.8. Procesión (procesiones)	106
C.8.1. Posiciones actuales (posiciones_actuales) — histórico de pings anónimos	107
C.9. Recorrido (recorridos)	107
C.9.1. Punto de Ruta (puntos_ruta)	108
C.9.2. Punto de Interés (puntos_de_interes)	108
C.9.3. Recorrido-Punto de Ruta (recorrido_puntos_ruta) — tabla intermedia N:M	108
C.10. Usuario (usuarios)	109
C.10.1. Miembro de Junta de Cofradías (miembros_junta_cofradia)	110
C.10.2. Administrador (administradores)	110
C.11. Notificación (notificaciones)	110
C.11.1. Notificación Entregada (notificaciones_entregadas) — tabla intermedia N:M	111
C.12. Relaciones entre tablas	111
D. Manual de despliegue	113
D.1. Requisitos previos	113
D.2. Despliegue del backend en Railway	113
D.3. Documentación de la API	114
D.4. Configuración del cliente React Native	114
D.5. Generación del ejecutable de la aplicación móvil	114
E. Manual de uso	115
F. Contenido del TFG	117

Índice de figuras

1.1. Interfaz de la aplicación El Penitente [4]	3
1.2. Interfaz de la aplicación Valladolid Cofrade [5]	4
1.3. Ejemplo de geolocalización en tiempo real en Google Maps	5
1.4. Seguimiento en tiempo real de un vehículo en Uber [9]	6
2.1. Diagrama de Gantt general completo del proyecto	10
3.1. Diagrama de caso de uso Ciudadano	29
3.2. Diagrama de caso de uso Cofrade	29
3.3. Diagrama de caso de uso Junta de Cofradías	30
3.4. Diagrama de caso de uso Administrador	30
3.5. Diagrama de dominio del proyecto	36
3.6. Diagrama de secuencia del CU “Visualizar procesión en tiempo real”	38
3.7. Diagrama de secuencia del CU “Compartir ubicación durante procesión”	39
3.8. Diagrama de secuencia del CU “Acceso como cofrade”	40
3.9. Diagrama de secuencia del CU “Dar de alta ciudad”	41
4.1. Diagrama de paquetes de la aplicación cliente	46
4.2. Diagrama de paquetes del backend propio	48
4.3. Diagrama de despliegue de la aplicación	49
4.4. Diagrama entidad-relación de la base de datos	51
4.5. Mockup de la pantalla de selección de ciudad	53
4.6. Mockup de la pantalla de inicio: aviso de retraso, procesión en curso, estadísticas y agenda del día	54
4.7. Mockup de la pantalla de calendario: vista mensual y agenda del día seleccionado	55
4.8. Mockup del menú de la pantalla de inicio y de los listados de procesiones, pasos, eventos y cofradías	56
4.9. Mockup del detalle de un paso y de una cofradía	57
4.10. Mockup del detalle de una procesión (programada y en curso) y de su información ampliada	58
4.11. Mockup de la pantalla de búsqueda con filtros por día y estado	59
4.12. Mockup del mapa en vivo	60
4.13. Mockup de la pantalla de perfil, en modo ciudadano y en modo cofrade	61

Índice de tablas

2.1. Riesgos identificados en el proyecto	13
2.2. Riesgo 1: Estimaciones de tiempo erróneas	13
2.3. Riesgo 2: Problemas con la integración de Google Maps	13
2.4. Riesgo 3: Errores durante el desarrollo	14
2.5. Riesgo 4: Retrasos o errores por falta de experiencia con tecnologías móviles	14
2.6. Riesgo 5: Problemas con la geolocalización en tiempo real	14
2.7. Riesgo 6: Cambios en los requisitos	14
2.8. Riesgo 7: Especificaciones genéricas	14
2.9. Riesgo 8: Problemas con el entorno de desarrollo	15
2.10. Riesgo 9: Indisposición del desarrollador	15
2.11. Riesgo 10: Falta de experiencia con las tecnologías del backend propio	15
2.12. Reparto de horas y coste de personal por perfil	16
2.13. Amortización de los recursos de equipo, prorrateada al uso real del proyecto	17
2.14. Coste de licencias de software	17
2.15. Costes totales del proyecto	18
3.1. Resumen de los actores del sistema	22
3.2. Caso de uso: Visualizar procesión en tiempo real	31
3.3. Caso de uso: Compartir ubicación durante procesión	32
3.4. Caso de uso: Crear alerta	33
3.5. Caso de uso: Acceso como cofrade	34
3.6. Caso de uso: Gestionar ciudades disponibles	35
5.1. Historia de usuario HU01	69
B.1. Caso de uso: Consultar información de una cofradía, procesión, evento o paso	85
B.2. Caso de uso: Acceso como cofrade	86
B.3. Caso de uso: Añadir favoritos	87
B.4. Caso de uso: Eliminar favoritos	87
B.5. Caso de uso: Configurar notificaciones	88
B.6. Caso de uso: Crear cofradía/Hermandad	89
B.7. Caso de uso: Crear procesión	90
B.8. Caso de uso: Actualizar estado de procesión	91
B.9. Caso de uso: Gestionar información de Semana Santa	92
B.10. Caso de uso: Definir recorrido de procesión	93

B.11.Caso de uso: Dar de alta Junta de Cofradías	94
B.12.Caso de uso: Editar Junta de Cofradías	95
B.13.Caso de uso: Dar de baja Junta de Cofradías	96
B.14.Caso de uso: Editar ciudad	97
B.15.Caso de uso: Dar de baja ciudad	98
B.16.Caso de uso: Consultar calendario de Semana Santa	99
B.17.Caso de uso: Consultar procesiones de un día	99
B.18.Caso de uso: Acceder a listados desde el menú principal	100
B.19.Caso de uso: Gestionar perfil de la Junta de Cofradías	100
B.20.Caso de uso: Gestionar miembros de las Juntas de Cofradías	101
C.1. Campos de ciudades	103
C.2. Campos de juntas_cofradias	104
C.3. Campos de cofradias	104
C.4. Campos de pasos	104
C.5. Campos de pasos_procesiones	105
C.6. Campos de codigos_acceso	105
C.7. Campos de ubicaciones	105
C.8. Campos de eventos	106
C.9. Campos de eventos_cofradias	106
C.10.Campos de procesiones	107
C.11.Campos de posiciones_actuales	107
C.12.Campos de recorridos	107
C.13.Campos de puntos_ruta	108
C.14.Campos de puntos_de_interes	108
C.15.Campos de recorrido_puntos_ruta	109
C.16.Campos de usuarios	109
C.17.Campos de miembros_junta_cofradia	110
C.18.Campos de administradores	110
C.19.Campos de notificaciones	110
C.20.Campos de notificaciones_entregadas	111
C.21.Relaciones entre tablas de PostgreSQL	111

Capítulo 1

Introducción

1.1. Contexto

La Semana Santa es una de las fiestas culturales y religiosas más relevantes del calendario español. Detrás de esta tradición hay siglos de historia, ya que forma parte del patrimonio histórico y artístico del país. En ciudades como Sevilla, Málaga o Valladolid, entre otras, está declarada Fiesta de Interés Turístico Internacional. Esta celebración tiene un gran impacto tanto cultural como económico, debido a los miles de visitantes nacionales e internacionales que atrae cada año.

La estructura organizativa de la Semana Santa está formada por las cofradías o hermandades, las cuales son asociaciones religiosas encargadas de organizar procesiones y eventos durante estos días. Cada procesión sigue su itinerario por las calles de la ciudad, llevando consigo pasos¹ de gran valor artístico que representan escenas de la Biblia. Esta organización de horarios, recorridos y actos requiere una gran coordinación por parte de las distintas cofradías y de la Junta de Cofradías de cada ciudad. Durante estos días, la ciudad experimenta imprevistos, como cortes de tráfico, cambios en la movilidad urbana y una gran cantidad de personas en puntos estratégicos del recorrido.

Desde el punto de vista social, la Semana Santa puede implicar dificultades de movilidad debido al desconocimiento de la ubicación de las distintas procesiones, además de los cambios de última hora en cuanto a los horarios de comienzo de cada procesión debido al mal tiempo o incluso a cancelaciones. Como esto se sale de la planificación inicial, los cofrades encuentran dificultades a la hora de comunicar estos cambios y decisiones de última hora a los espectadores y fieles. A día de hoy, son las distintas Juntas de Cofradías las que deben gestionar toda esta información de forma manual o con herramientas anticuadas.

En el contexto tecnológico actual, la mayoría de los turistas y ciudadanos hacen uso de sus teléfonos móviles como principal medio para consultar información. Aplicaciones como Google Maps [1] permiten conocer en tiempo real la ubicación (geolocalización [2]) de calles, monumentos, bares o incluso el estado del tráfico o del transporte público. Esta funcionalidad podría aplicarse a la Semana Santa, de forma

¹Paso: plataforma o andamio, habitualmente de grandes dimensiones y ricamente ornamentado, sobre el que se transportan a hombros o mediante ruedas las imágenes religiosas durante la procesión.

que facilitaría a los visitantes y ciudadanos información constante y rápida sobre dónde se encuentra una procesión o si ha habido algún cambio o imprevisto en la misma.

Finalmente, debido a la cantidad de dinero que genera el turismo durante la Semana Santa, podemos concluir que es de gran interés para las ciudades que la experiencia del turista y del ciudadano sea positiva y que existan herramientas para que esto se haga realidad, de forma que una aplicación que mejore dicha experiencia repercutirá positivamente.

1.2. Motivación

La principal motivación de este proyecto surge de la dificultad que tienen tanto los ciudadanos como los visitantes a la hora de buscar y obtener información clara y actualizada durante los días de la Semana Santa. Aunque a día de hoy existan programas oficiales, carteles informativos o cuentas oficiales en redes sociales, estos medios no siempre consiguen dar a conocer en tiempo real la situación de las procesiones, ya sea por retrasos o cambios de última hora debidos a factores meteorológicos o a problemas dentro de la propia cofradía.

Debido a esto, los asistentes, en muchas ocasiones, deben desplazarse a los distintos puntos del recorrido sin tener la certeza de que la procesión no haya pasado ya, cuánto tiempo de espera se estima hasta que pase por ese punto o incluso si llegará a pasar debido a cancelaciones. Esto genera desorganización y aglomeraciones innecesarias, además de una experiencia menos satisfactoria para los observadores.

Gracias a estas observaciones, se ve clara la necesidad de contar con un medio común capaz de organizar y difundir la información necesaria para que la experiencia sea lo más clara y accesible posible para cualquier persona.

Como respuesta a estas limitaciones, se plantea la idea de una aplicación móvil que permita la centralización de la información sobre el estado de las procesiones de forma actualizada, además de incluir otras funciones como la visualización de los recorridos y la geolocalización en tiempo real de las mismas. De esta forma, se mejora no solo la experiencia de los usuarios, sino que también se facilita la gestión de la información durante la Semana Santa.

Además, el desarrollo de esta aplicación permite no solo aplicar los conocimientos adquiridos durante el grado sobre análisis de requisitos, diseño de software, desarrollo de aplicaciones móviles y gestión de proyectos, sino también ampliarlos mediante su aplicación en un proyecto real, enfrentándose a problemas prácticos como la integración de distintas tecnologías, el manejo de la geolocalización en tiempo real o la gestión de múltiples tipos de usuarios dentro de un mismo sistema.

1.3. Estado de la cuestión

En la actualidad existen diversas soluciones tecnológicas relacionadas con la Semana Santa, aunque la mayoría de ellas se encuentran limitadas a ámbitos geográficos concretos o se centran únicamente en

la difusión de información estática. A continuación, se analizan las principales herramientas disponibles, así como sus características más relevantes.

1.3.1. Aplicaciones actuales de Semana Santa

A día de hoy, existen diversas aplicaciones móviles orientadas a la Semana Santa para distintas ciudades españolas. Entre ellas encontramos *El Penitente* [3], centrada únicamente en las ciudades de Sevilla y Málaga. Esta aplicación ofrece información sobre horarios, itinerarios y una representación del recorrido de las procesiones sobre el mapa, mostrando el trayecto seguido y, en algunos casos, su posición aproximada.

La aplicación presenta una interfaz (Figura 1.1) en la que el usuario puede consultar información sobre las procesiones y sus recorridos, avisos y notificaciones, planificaciones para organizar la Semana Santa y horarios de hermandades o cofradías.

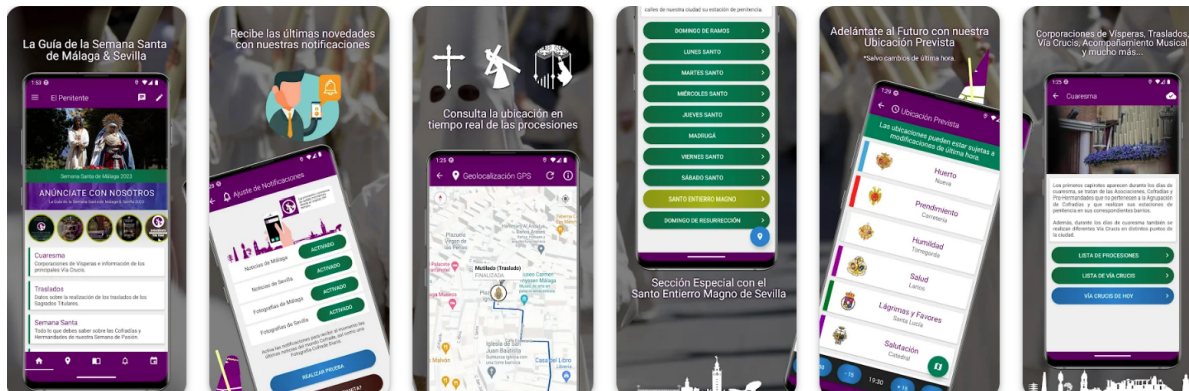


Figura 1.1: Interfaz de la aplicación El Penitente [4]

Su mayor limitación es que está diseñada y pensada para ciudades específicas y no constituye una solución unificada a nivel nacional.

Otra aplicación a analizar es *Valladolid Cofrade* [5], una herramienta reciente desarrollada como aplicación web progresiva (PWA) [6], es decir, una aplicación accesible desde el navegador que puede instalarse en el dispositivo y funcionar de forma similar a una aplicación nativa. Esta permite a los usuarios acceder a información sobre la Semana Santa sin realizar la instalación tradicional desde tiendas oficiales. La aplicación ofrece información sobre programas, recorridos y un sistema de avisos y notificaciones en tiempo real ante cambios o incidencias (Figura 1.2).

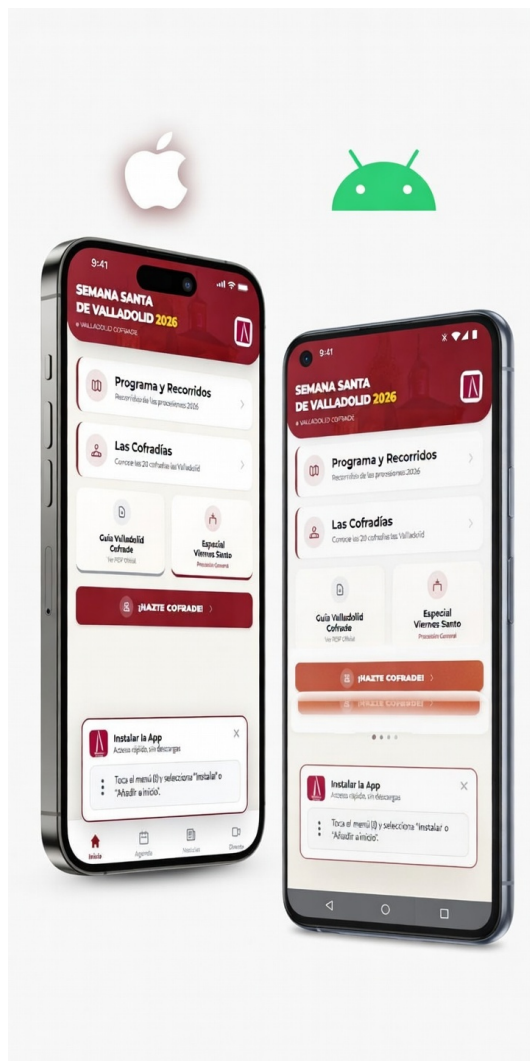


Figura 1.2: Interfaz de la aplicación Valladolid Cofrade [5]

En otras ciudades se cuenta con aplicaciones locales específicas o sitios web adaptados, como sSantaZgz en Zaragoza [7] o Junta Pro Semana Santa de Zamora [8]. Estas aplicaciones muestran el programa oficial, los distintos recorridos e incluso, en algunos casos, como en la de Zamora, incluyen herramientas para conocer la previsión meteorológica.

Más allá de estas limitaciones técnicas, también hay que destacar la diversidad organizativa de la Semana Santa en España, ya que esta depende de la ciudad, de la estructura que se siga en ella y de la terminología utilizada. Por ejemplo, en algunas ciudades, como Valladolid, es común emplear los términos *cofradía* o *cofrade*, mientras que en otras, como Sevilla, se utilizan *hermandad* o *nazareno* para referirse a conceptos equivalentes. Desde el punto de vista organizativo, existen diferentes formas de organizar los eventos, recorridos e itinerarios de las procesiones. Incluso hay diferencias en elementos como pasos, imágenes, homilías, rezos o cánticos que forman parte de estas procesiones o eventos.

Estas diferencias hacen que cada aplicación tenga un diseño distinto, pensado específicamente para una ciudad concreta, dificultando la creación de una solución reutilizable. Por tanto, en la actualidad no existe una plataforma que consiga unificar y generalizar estos conceptos y ofrecer una visión global de la

Semana Santa.

Además de las aplicaciones, los usuarios también hacen uso de las publicaciones oficiales de las cofradías en sus páginas web o de programas impresos. Sin embargo, los medios que realmente resultan útiles para la actualización inmediata de la información son las redes sociales oficiales de las cofradías o de las Juntas de Cofradías, que permiten a los espectadores recibir información sobre cambios o comunicados al instante. Aun así, esta forma de comunicación no siempre resulta fácil de encontrar o de organizar debido a la diversidad de cuentas que gestionan dicha información.

1.3.2. Aplicaciones de seguimiento en tiempo real

En cuanto a la geolocalización y el seguimiento en tiempo real, se trata de una tecnología ya existente que se utiliza en diversas aplicaciones como *Google Maps* [1], que permite conocer la ubicación, cómo llegar a distintos destinos o el estado del transporte público (Figura 1.3).

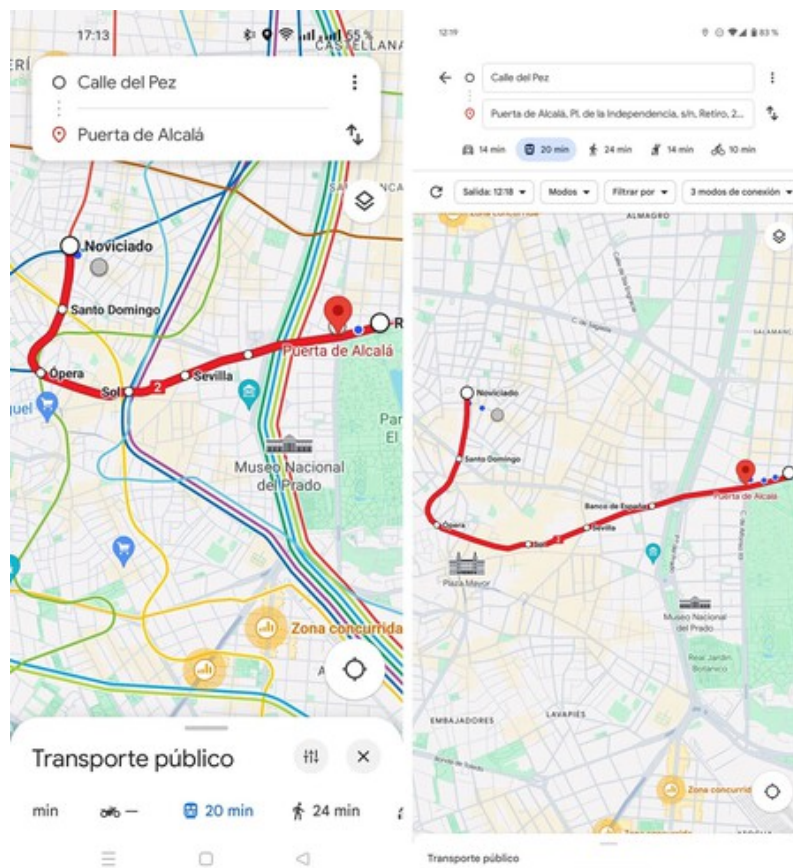


Figura 1.3: Ejemplo de geolocalización en tiempo real en Google Maps

Además, plataformas como *Uber* [9] permiten a los usuarios conocer en todo momento la ubicación exacta del vehículo solicitado (Figura 1.4).

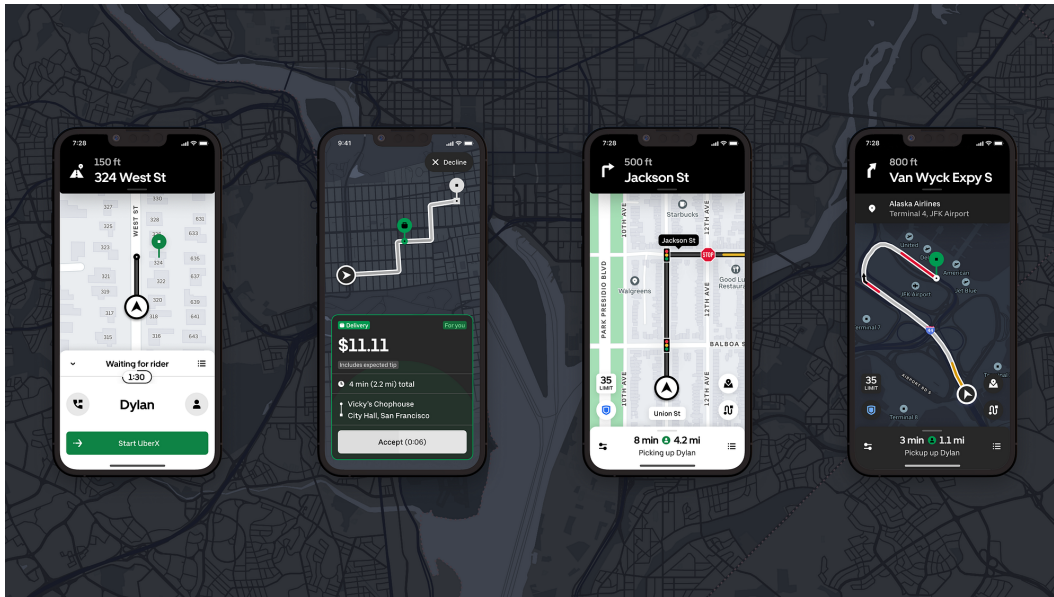


Figura 1.4: Seguimiento en tiempo real de un vehículo en Uber [9]

Además, existen diversos Trabajos de Fin de Grado centrados en el desarrollo de aplicaciones móviles y el uso de tecnologías de geolocalización. Un ejemplo es el proyecto realizado por Mario Llorente Pérez [10], enfocado en una aplicación móvil relacionada con actividades deportivas y el uso de mapas y geolocalización. En trabajos como este puede observarse el importante papel que juega la localización en tiempo real y la necesidad de diseñar interfaces que permitan al usuario interactuar con la aplicación de forma sencilla.

Todas estas aplicaciones y trabajos constituyen ejemplos claros de que el seguimiento en tiempo real es una funcionalidad ampliamente utilizada, a pesar de que en el contexto de la Semana Santa sigue estando limitada y, cuando se ofrece, suele restringirse a entornos locales o presentar un nivel de precisión reducido.

Por tanto, su incorporación en una solución a nivel nacional conseguiría no solo mejorar la experiencia de los usuarios, sino también superar las limitaciones de las soluciones actuales en cuanto a la oferta de información precisa sobre el estado y la ubicación de las procesiones o eventos.

1.4. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es el diseño y desarrollo de una aplicación móvil orientada a la Semana Santa, capaz de centralizar en una única plataforma la información de las distintas celebraciones de Semana Santa a nivel nacional, con independencia de la ciudad, cofradía o terminología empleada en cada una de ellas. En concreto, se busca ofrecer a los usuarios una herramienta que les permita consultar la información relevante de la Semana Santa de una ciudad y visualizar en tiempo real, mediante geolocalización, la ubicación, el recorrido y el estado de sus procesiones.

Para alcanzar este objetivo, se plantean los siguientes objetivos específicos:

- Diseñar dicha aplicación permitiendo a los usuarios elegir entre las distintas Semanas Santas dentro del territorio nacional.
- Proporcionar acceso a la información general relacionada con la Semana Santa, incluyendo historia, cofradías o hermandades y elementos representativos.
- Mostrar información detallada sobre los eventos, como nombre, descripción, fechas, horas e itinerarios.
- Añadir funcionalidades de personalización, como la selección de eventos o elementos de interés y la recepción de notificaciones ante cambios o incidencias.
- Implementar un sistema de visualización sobre mapa que permita representar las procesiones y su evolución en tiempo real.
- Facilitar la navegación del usuario mediante la posibilidad de calcular rutas hacia ubicaciones concretas relacionadas con los eventos.
- Desarrollar una solución accesible desde dispositivos móviles que garantice una experiencia de uso adecuada.
- Diseñar una arquitectura escalable que permita la incorporación de nuevas ciudades y eventos sin necesidad de rediseñar el sistema.

1.5. Estructura de la memoria

PENDIENTE: revisar y actualizar este apartado al terminar el TFG.

La memoria se ha organizado en distintos capítulos en los cuales se puede ver la estructura y el desarrollo de dicho proyecto, de forma que se pueda comprender el proyecto de forma clara y completa.

En primer lugar, en este capítulo de introducción se presenta el contexto del proyecto, la motivación que lo impulsa, el estado actual de las distintas soluciones existentes, así como los objetivos que se llevan a cabo.

Después, en el capítulo de planificación se describe la metodología seguida durante el desarrollo de la aplicación, la organización del proyecto, además de las distintas fases de desarrollo y gestión de tareas.

A continuación, en el capítulo de análisis se identifican los actores del sistema, se especifican los requisitos funcionales, de información y no funcionales, y se detallan los casos de uso y el modelo de dominio sobre los que se sustenta el desarrollo de la aplicación.

Seguido por el capítulo de las iteraciones, en el cual se muestran las distintas etapas en las que se ha dividido y repartido el desarrollo de la aplicación, mostrando la evolución que ha ido teniendo el sistema a lo largo del desarrollo del proyecto.

Además, cuenta con un capítulo de estado, en el que se muestra el estado final del sistema desarrollado.

Por último, incluye un capítulo de pruebas en el que se analizan los resultados obtenidos al conseguir construir la versión final y operativa de la aplicación.

Esta memoria concluye con un capítulo de conclusiones, donde se valoran los objetivos conseguidos y se plantean posibles mejoras futuras.

Capítulo 2

Planificación

2.1. Metodología

Para el desarrollo de este proyecto se ha decidido hacer uso de una metodología **iterativa incremental** [11]. Este modelo se ha seleccionado porque permite organizar el proyecto en ciclos o iteraciones sucesivas, donde cada una añade funcionalidad al sistema de forma progresiva. Cada iteración incluye sus propias fases de análisis, diseño, implementación y pruebas, lo que facilita la validación continua del producto y la detección temprana de errores.

La elección de esta metodología está justificada por la naturaleza del proyecto, ya que permite entregar incrementos funcionales del sistema de manera gradual, comenzando por las funcionalidades básicas del módulo ciudadano y avanzando progresivamente hacia características más complejas como la geolocalización en tiempo real, los paneles de gestión y el sistema de notificaciones. Al ser un proyecto desarrollado por un único desarrollador, este modelo resulta adecuado porque combina la flexibilidad de adaptarse a posibles ajustes durante el desarrollo con una planificación estructurada que facilita el seguimiento del avance.

El proyecto se ha organizado en una **fase inicial de análisis** general, seguida de **cinco iteraciones** que abordan de forma incremental los distintos módulos del sistema, y una **fase final** de documentación, integración y entrega.

2.2. Planificación, Fases e iteraciones

2.2.1. Planificación inicial

Este proyecto consta de tres grandes fases, cuya planificación general se puede observar en el siguiente Diagrama de Gantt [12] (Figura 2.1): una primera fase de análisis, una segunda fase de desarrollo iterativo e incremental, organizada en cinco iteraciones, y una tercera y última fase de documentación y entrega.

2.2. Planificación, Fases e iteraciones

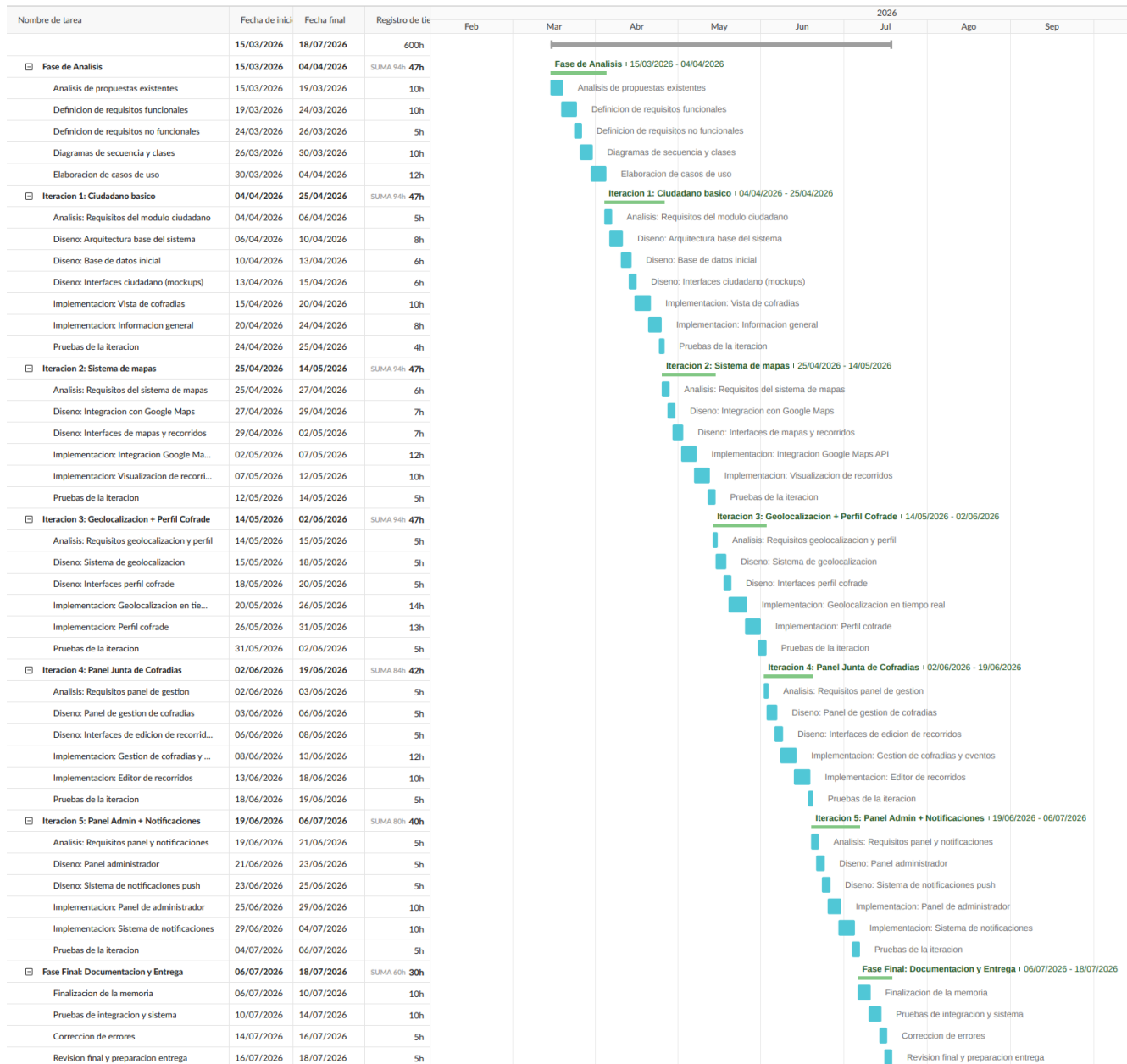


Figura 2.1: Diagrama de Gantt general completo del proyecto

2.2.2. Fase de análisis

Esta primera fase se centra en el análisis de las aplicaciones relacionadas con la Semana Santa existentes, la identificación de sus carencias y áreas de mejora, y en una versión inicial de los requisitos funcionales y no funcionales [13] del sistema, además de la elaboración de los casos de uso [14] y los distintos diagramas que servirán como base para la realización del desarrollo de la aplicación.

Esta fase comienza con el análisis de las propuestas existentes en el mercado, identificando las características principales de aplicaciones como *El Penitente* [3], *Valladolid Cofrade* [5] o *sSantaZgz* [7].

Seguido de este análisis se realiza la definición de los requisitos funcionales, los cuales describen las

funcionalidades concretas que el sistema debe ofrecer, y los requisitos no funcionales, que definen las condiciones de calidad, rendimiento y seguridad que debe cumplir [13], teniendo en cuenta los cuatro perfiles de usuario: ciudadano, cofrade, Junta de Cofradías y administrador. También se elaboran los diagramas de secuencia y el modelo de dominio, así como los casos de uso [14], representaciones estandarizadas de las interacciones entre los usuarios y el sistema, que servirán para el desarrollo posterior.

2.2.3. Fase de iteraciones

Esta segunda fase está centrada en el desarrollo de la aplicación siguiendo un enfoque iterativo e incremental [15], organizado en cinco iteraciones. En cada una de ellas se repite un mismo ciclo de trabajo formado por las etapas de análisis, diseño, implementación y pruebas, de modo que las funcionalidades correspondientes a esa iteración se analizan, diseñan, implementan y validan antes de darla por finalizada y avanzar a la siguiente. Este enfoque permite ir incorporando las funcionalidades de forma progresiva, obteniendo al final de cada iteración una versión incremental y funcional del sistema. El desglose detallado de tareas y horas de cada iteración puede consultarse en el Diagrama de Gantt (Figura 2.1).

Estas iteraciones abarcan desde las funcionalidades básicas del ciudadano hasta las más complejas, como la geolocalización en tiempo real, es decir, la capacidad de conocer y actualizar la posición geográfica de una procesión de forma continua haciendo uso de la tecnología GPS¹ del móvil, o el sistema de notificaciones (mensajes enviados desde el servidor a los dispositivos de los usuarios sin que estos tengan que abrir la aplicación activamente).

Iteración 1: Módulo ciudadano básico

Esta primera iteración tiene como objetivo establecer la arquitectura base del sistema y desarrollar las funcionalidades fundamentales del perfil de ciudadano, entre las que se incluyen la visualización de la información general de la Semana Santa, el listado de organizaciones y la consulta de eventos.

Iteración 2: Sistema de mapas

Esta segunda iteración se centra principalmente en la implementación del sistema de mapas, que permitirá visualizar los distintos recorridos de las procesiones.

Iteración 3: Geolocalización y perfil cofrade

Esta tercera iteración se centra en la implementación completa del sistema de geolocalización en tiempo real, de forma que se pueda conocer y actualizar de manera continua la posición de las procesiones

¹GPS (Global Positioning System): sistema de navegación por satélite que permite determinar la posición geográfica de un dispositivo en cualquier punto del planeta.

durante su recorrido. Además, se desarrolla el perfil de cofrade con sus correspondientes funcionalidades, incluyendo la posibilidad de compartir la ubicación durante las procesiones.

Iteración 4: Panel de la Junta de Cofradías

Esta cuarta iteración corresponde al desarrollo del panel de gestión para las Juntas de Cofradías, permitiendo administrar la información de la Semana Santa, incluyendo eventos, recorridos, imágenes y participantes. El acceso a este panel está restringido mediante un mecanismo de control de acceso [16], que permite diferenciar los permisos y limitaciones de los distintos roles del sistema.

Iteración 5: Panel de administrador y notificaciones

Esta quinta y última iteración se centra en la implementación del panel de administrador global del sistema, así como del sistema de notificaciones, que permitirá a la Junta de Cofradías comunicar a los usuarios cambios o incidencias en las procesiones de forma inmediata, sin necesidad de que estos tengan la aplicación abierta.

2.2.4. Fase final: documentación y entrega

En esta última fase se realizan las pruebas de integración del sistema completo, la corrección de errores detectados, la finalización de la memoria del proyecto y la preparación de la entrega final.

2.3. Análisis de riesgos

Otra parte importante en la planificación de un proyecto de desarrollo de software es el análisis de riesgos. Un riesgo es un evento de incertidumbre que puede afectar de forma positiva o negativa a los objetivos del proyecto [17]. Para garantizar el alcance de los objetivos del proyecto, es necesario realizar un análisis de estos riesgos.

Los riesgos identificados se organizan en función de su prioridad, la cual se determina mediante la técnica de análisis de exposición al riesgo (*Risk Exposure*, RE) [18], donde el riesgo se calcula mediante un valor numérico que representa la probabilidad e impacto estimados de cada evento. Se define de la siguiente manera:

$$RE = \text{probabilidad} \times \text{impacto} \quad (2.1)$$

La variable probabilidad representa la probabilidad de que el riesgo se materialice en el proyecto (escala de 1 a 10), mientras que el impacto representa la gravedad de sus consecuencias en caso de ocurrir (escala de 1 a 10). En la Tabla 2.1 se muestran los riesgos identificados en el proyecto.

Riesgo	Probabilidad	Impacto	Prioridad
1. Estimaciones de tiempo erróneas	8	7	56
2. Falta de experiencia con las tecnologías del backend propio (Spring Boot)	7	7	49
3. Problemas con la integración de Google Maps	6	8	48
4. Errores durante el desarrollo	7	6	42
5. Retrasos o errores por falta de experiencia con tecnologías móviles	7	6	42
6. Problemas con la geolocalización en tiempo real	5	8	40
7. Cambios en los requisitos	6	6	36
8. Especificaciones ambiguas o poco definidas	5	6	30
9. Problemas con el entorno de desarrollo	4	5	20
10. Indisponibilidad del desarrollador	3	6	18

Tabla 2.1: Riesgos identificados en el proyecto

2.3.1. Planificación de riesgo

Tras realizar la identificación de los riesgos, el siguiente paso es la decisión de qué medidas se deben tomar para reducir la probabilidad de que el riesgo ocurra, de forma que se pueda disminuir su impacto una vez se materialice. En las Tablas 2.2 a 2.11 se detallan, para cada uno de los riesgos identificados en la Tabla 2.1, las medidas de reducción propuestas para disminuir su probabilidad y las medidas de contingencia previstas en caso de que finalmente se materialice.

Riesgo	Estimaciones de tiempo erróneas
Reducción	Planificar las tareas con margen por si la estimación ha sido inferior al tiempo necesario. Dividir las tareas grandes en subtareas más pequeñas y manejables.
Contingencia	Cambiar la planificación, reducir el alcance del proyecto priorizando las funcionalidades esenciales.

Tabla 2.2: Riesgo 1: Estimaciones de tiempo erróneas

Riesgo	Problemas con la integración de Google Maps
Reducción	Realizar pruebas de concepto tempranas con la API de Google Maps. Documentarse sobre las limitaciones y cuotas del servicio.
Contingencia	Considerar alternativas como OpenStreetMap [19] o Mapbox en caso de problemas persistentes.

Tabla 2.3: Riesgo 2: Problemas con la integración de Google Maps

2.4. Seguimiento del proyecto

Nota: esta sección se completará al finalizar el desarrollo del proyecto, comparando la planificación inicial con la duración real del proyecto. Se comentará que ha habido distintas replanificaciones debido a diversos recortes de tiempo que han surgido a lo largo de los meses de desarrollo.

Riesgo	Errores durante el desarrollo
Reducción	Verificar que la funcionalidad está implementada correctamente antes de dar por finalizado el incremento. Realizar pruebas unitarias ^a
Contingencia	Volver al último punto correctamente probado y continuar desde ahí. Utilizar control de versiones [20] para recuperar estados anteriores del código.

Tabla 2.4: Riesgo 3: Errores durante el desarrollo

^aPruebas unitarias: pruebas que verifican el correcto funcionamiento de componentes individuales del código de forma aislada.

Riesgo	Retrasos o errores por falta de experiencia con tecnologías móviles
Reducción	Formarse en las tecnologías clave antes de proceder al desarrollo. Consultar documentación oficial y tutoriales.
Contingencia	Aprovechar la experiencia del tutor para solventar los posibles problemas. Buscar soluciones en comunidades de desarrolladores.

Tabla 2.5: Riesgo 4: Retrasos o errores por falta de experiencia con tecnologías móviles

Riesgo	Problemas con la geolocalización en tiempo real
Reducción	Investigar las mejores prácticas para implementar geolocalización en aplicaciones móviles. Realizar prototipos tempranos para validar la viabilidad técnica.
Contingencia	Implementar un modo de actualización manual de la ubicación como alternativa temporal.

Tabla 2.6: Riesgo 5: Problemas con la geolocalización en tiempo real

Riesgo	Cambios en los requisitos
Reducción	El desarrollo iterativo permite adaptar el sistema a cambios de forma gradual.
Contingencia	Reevaluar las prioridades y ajustar el alcance del proyecto según las nuevas necesidades.

Tabla 2.7: Riesgo 6: Cambios en los requisitos

Riesgo	Especificaciones genéricas
Reducción	Reunir toda la información posible antes de abordar una tarea. Clarificar requisitos con el tutor.
Contingencia	Prototipar en caso de ser necesario para validar la interpretación de los requisitos.

Tabla 2.8: Riesgo 7: Especificaciones genéricas

Riesgo	Problemas con el entorno de desarrollo
Reducción	Utilizar un repositorio remoto (GitHub [20]) para almacenar copias de seguridad del código.
Contingencia	Utilizar una máquina de respaldo o entorno de desarrollo alternativo.

Tabla 2.9: Riesgo 8: Problemas con el entorno de desarrollo

Riesgo	Indisposición del desarrollador
Reducción	Planificar las tareas con margen por si es necesario extender el tiempo para completar el incremento.
Contingencia	Realizar horas extra en caso necesario y ajustar la planificación.

Tabla 2.10: Riesgo 9: Indisposición del desarrollador

Riesgo	Falta de experiencia con las tecnologías del backend propio (Spring Boot, Spring Security, JPA/Hibernate)
Reducción	Realizar una prueba de concepto acotada (<i>spike</i>) antes de comenzar la implementación completa, construyendo un único recurso extremo a extremo (entidad, autenticación JWT y <i>endpoint</i> protegido por rol) para validar el ritmo de desarrollo real con la tecnología. Consultar la documentación oficial y limitar el alcance del backend a lo estrictamente necesario para los requisitos, evitando patrones o herramientas adicionales que no aporten valor al proyecto (por ejemplo, no se contemplan microservicios ni colas de mensajería).
Contingencia	Si el retraso es significativo, reducir el alcance del backend priorizando los endpoints imprescindibles para cada iteración y posponiendo funcionalidades secundarias a iteraciones posteriores, tal y como permite la metodología iterativa incremental (Sección 2.1).

Tabla 2.11: Riesgo 10: Falta de experiencia con las tecnologías del backend propio

2.4.1. Monitorización de riesgos

Durante el desarrollo del proyecto, se debe realizar un seguimiento continuo de los riesgos identificados para detectar de forma temprana su aparición y poder aplicar las medidas pertinentes.

Nota: esta sección se completará al finalizar el desarrollo del proyecto, documentando qué riesgos se manifestaron, el impacto que tuvieron y las medidas que se tomaron para subsanarlos.

2.5. Estimación de costes

A continuación se presenta el presupuesto estimado del proyecto, calculado como si este fuese desarrollado por un equipo profesional completo, siguiendo los perfiles habituales en el desarrollo de una aplicación móvil de estas características. Se trata de una estimación teórica con fines de planificación y no de los costes reales asumidos por el autor, quien ha desarrollado el TFG de forma individual.

2.5.1. Costes de personal

Un proyecto de estas características, correspondiente al desarrollo completo de una aplicación móvil sobre la Semana Santa, requeriría en un entorno profesional un equipo multidisciplinar. Se han considerado los siguientes seis perfiles, entre los que se reparten las 300 horas estimadas de duración del proyecto:

- **Jefe de Proyecto:** coordinación general, planificación y seguimiento.
- **Analista Funcional:** recogida y especificación de requisitos, análisis del dominio.
- **Diseñador UX/UI:** diseño de la experiencia e interfaz de usuario.
- **Desarrollador Backend:** implementación del servidor, base de datos y API.
- **Desarrollador Frontend/Mobile:** implementación de la aplicación Android.
- **Tester/QA:** pruebas funcionales, de regresión y control de calidad.

Para cada perfil se ha calculado el coste por hora a partir del sueldo bruto anual medio en España, incrementado en un 30 % en concepto de cotizaciones a la Seguridad Social a cargo de la empresa, y dividido entre 1.800 horas anuales de jornada completa. La Tabla 2.12 recoge el reparto de horas y el coste asociado a cada perfil.

Perfil	Horas	Coste/hora	Coste total
Jefe de Proyecto [21]	45	33€	1.485,00€
Analista Funcional [22]	45	27€	1.215,00€
Diseñador UX/UI [23]	30	27€	810,00€
Desarrollador Backend [24]	75	26€	1.950,00€
Desarrollador Frontend/Mobile [25]	75	19€	1.425,00€
Tester/QA [26]	30	17€	510,00€
Total	300		7.395,00€

Tabla 2.12: Reparto de horas y coste de personal por perfil

2.5.2. Costes de equipo y amortización

El equipo del proyecto necesitaría un ordenador portátil por persona, además de un dispositivo Android dedicado a las pruebas. Se considera una duración total del proyecto de 5 meses.

- **Ordenadores portátiles:** 6 unidades valoradas en 1.000€ cada una, con una vida útil estimada de 5 años (60 meses). La amortización mensual por unidad es de 16,66€, lo que supone 83,30€ por portátil durante los 5 meses del proyecto, y 500,00€ en total para los 6 puestos de trabajo.
- **Dispositivo Android de pruebas:** 1 unidad valorada en 300€, con una vida útil estimada de 3 años (36 meses). La amortización mensual es de 8,33€, lo que supone 41,65€ para los 5 meses del proyecto.

Ahora bien, estos importes corresponden a la ocupación de los equipos durante los 5 meses completos del proyecto. Dado que el trabajo real asciende a 300 horas, frente a una capacidad disponible de 750 horas en ese periodo (150 horas/mes de jornada completa, según el criterio de 1.800 horas anuales aplicado en el apartado anterior), el resto del tiempo los equipos podrían destinarse a otro uso. Por ello, la amortización se imputa de forma proporcional al uso real del proyecto ($300/750 = 40\%$).

El coste de amortización así calculado asciende a **216,66€**, tal y como recoge la Tabla 2.13.

Recurso	Valor	Vida útil	Amort. (5 meses)	Amort. proporcional (300h, 40 %)
Portátiles (x6)	6.000,00€	5 años	500,00€	200,00€
Dispositivo Android (x1)	300,00€	3 años	41,65€	16,66€
Total amortización			541,65€	216,66€

Tabla 2.13: Amortización de los recursos de equipo, prorrateada al uso real del proyecto

2.5.3. Costes de licencias de software

Buena parte de las herramientas empleadas disponen de planes gratuitos o licencias universitarias suficientes para un proyecto de este tamaño. Sin embargo, un equipo profesional de 6 personas necesitaría además herramientas colaborativas de pago para la gestión del proyecto y el diseño. La Tabla 2.14 recoge el coste y la fuente de cada licencia.

Herramienta	Coste	Fuente
Android Studio	0,00€	[27]
Visual Studio Code	0,00€	[28]
Overleaf (plan gratuito)	0,00€	—
Google Firebase (plan gratuito)	0,00€	[29]
Railway (plan gratuito)	0,00€	[30]
Google Maps API (crédito gratuito)	0,00€	[1]
GitHub (plan gratuito)	0,00€	[20]
Cuenta de desarrollador Google Play (pago único)	23,00€	[31]
Figma Professional (1 licencia, 5 meses)	69,00€	[32]
Jira Software Standard (6 usuarios, 5 meses)	218,40€	[33]
Total licencias	310,40€	

Tabla 2.14: Coste de licencias de software

2.5.4. Otros costes indirectos

Un equipo de 6 personas trabajando de forma presencial o híbrida necesitaría un espacio de oficina. Se ha tomado como referencia el alquiler de un despacho de 22 m² en un espacio de coworking en el centro de Valladolid, con todos los suministros incluidos (calefacción, refrigeración, electricidad, wifi y limpieza), por un importe de 475€ al mes [34]. Para los 5 meses de duración del proyecto, el coste estructural asciende a 2.375,00€.

Al igual que con la amortización de equipos, este importe corresponde a la disponibilidad del espacio durante los 5 meses completos, mientras que el uso real del proyecto es de 300 horas frente a una capacidad de 750 horas en ese periodo (40 %); el resto del tiempo el espacio podría destinarse a otro uso. Aplicando este mismo criterio de proporcionalidad, el coste estructural imputado al proyecto asciende a 950,00€ ($2.375,00€ \times 40\%$).

2.5.5. Coste total

La Tabla 2.15 resume el presupuesto completo del proyecto, considerando personal, amortización de equipo, licencias de software y costes estructurales —estos dos últimos prorrateados a las 300 horas reales de uso— para una duración de 300 horas repartidas en 5 meses.

Elemento	Coste
Personal (Tabla 2.12)	7.395,00€
Amortización de equipo, prorrateada (Tabla 2.13)	216,66€
Licencias de software (Tabla 2.14)	310,40€
Costes estructurales, prorrateados (oficina y suministros)	950,00€
Total	8.872,06€

Tabla 2.15: Costes totales del proyecto

2.5.6. Coste estimado de la infraestructura en producción

Los costes anteriores corresponden únicamente al desarrollo del TFG, durante el cual tanto Railway [35] como los servicios de Firebase utilizados (Storage y Cloud Messaging) se han mantenido dentro de sus respectivos planes gratuitos. Sin embargo, tal y como se señala en la Sección 7.2, el paso a producción con usuarios reales implicaría, por un lado, pasar del plan *Free* de Railway a un plan de pago (*Hobby* o *Pro*) facturado según el consumo real de CPU, memoria y red [30], y por otro, un coste de Firebase previsiblemente marginal, ya que Cloud Messaging no tiene coste y Storage solo se usa para las imágenes de los pasos, un volumen de datos pequeño y que apenas crece con el número de usuarios.

A modo orientativo, tomando las tarifas oficiales vigentes de Railway [30] –10 \$ por GB de RAM al mes, 20 \$ por vCPU al mes y 0,05 \$ por GB de red saliente— se puede estimar el coste de una semana de Semana Santa en una única ciudad piloto, bajo las mismas hipótesis de partida que en el planteamiento inicial con Firebase (Sección 7.2): 3.000 usuarios simultáneos consultando la posición de una procesión

en vivo, con una cadencia de 30 segundos (RNF-08), y 200 cofrades compartiendo su ubicación durante 2 horas al día.

- **Cómputo del backend:** una instancia con 1 GB de RAM y 1 vCPU es capaz de sostener sin dificultad el volumen de peticiones simples en JSON que generan estas hipótesis (del orden de 100-150 peticiones por segundo en el pico de la tarde). Manteniendo el servicio desplegado de forma continua durante los 7 días de Semana Santa (una sexta parte de un mes de 30 días), el coste de cómputo asciende a unos **7,00 \$** $((10 \$ + 20 \$) \times 7/30)$.
- **Base de datos:** una instancia de PostgreSQL con recursos equivalentes (0,5 GB de RAM, 1 GB de almacenamiento en volumen) añade aproximadamente **1,20 \$** para el mismo periodo.
- **Red saliente:** el volumen de datos transferido por las respuestas JSON de la API (posiciones, listados, fichas) se mantiene, para este número de usuarios y peticiones ligeras, muy por debajo de 10 GB en la semana, lo que supone menos de **0,50 \$**.

Bajo estas hipótesis, el coste de infraestructura para una semana de Semana Santa en una ciudad piloto rondaría los **9 \$** de consumo, a los que se sumaría la cuota fija del plan de Railway elegido (5 \$/mes en el plan *Hobby*) si no se opta por mantener el servicio en el plan *Free* el resto del año. Se trata de un orden de magnitud similar al estimado inicialmente para Firebase (unos 5€ semanales, Sección 7.2 de una versión anterior de este documento), con la diferencia de que aquí el coste depende de los recursos de cómputo reservados y no del número de operaciones de lectura o escritura, por lo que crece de forma más previsible al escalar a más ciudades y usuarios, aunque también exige dimensionar y, en su caso, escalar manualmente la instancia del backend a medida que crece la carga –una responsabilidad que antes asumía Firebase de forma automática–. Esta estimación –y las hipótesis de partida sobre el número de usuarios y ciudades– debería revisarse y ajustarse junto con el tutor antes de un despliegue real en producción.

Capítulo 3

Análisis

3.1. Actores del sistema

En el desarrollo de cualquier sistema software es fundamental identificar y definir los actores que interactuarán con la plataforma. Los actores representan a las entidades externas, ya sean personas u otros sistemas, que se comunican con la aplicación y hacen uso de sus funcionalidades.

En el caso de esta aplicación, orientada a la gestión y consulta de información sobre la Semana Santa, se han identificado cuatro actores principales: el ciudadano, el cofrade, la Junta de Cofradías y el administrador. Cada uno de ellos dispone de un nivel de acceso y unas funcionalidades diferenciadas, que se detallan a continuación y que sirven de base para la definición de los requisitos funcionales (Sección 3.2).

3.1.1. Ciudadano

El ciudadano es el actor principal de la aplicación y representa a cualquier persona interesada en seguir la Semana Santa, ya sea de forma presencial en la ciudad o de manera remota. No requiere registro previo para utilizar la aplicación.

Puede consultar la información detallada de cofradías, eventos y procesiones, realizar búsquedas y aplicar filtros, y visualizar en tiempo real la posición de las procesiones mediante geolocalización. Además, puede generar rutas hacia una procesión o entre dos puntos evitando aquellas que estén activas, marcar contenido como favorito, configurar sus preferencias de notificaciones y recibir alertas sobre los eventos y procesiones que sigue.

3.1.2. Cofrade

El cofrade es un ciudadano que, además, pertenece a una cofradía o hermandad. Accede al modo cofrade introduciendo el código de acceso de su cofradía, validado por la Junta de Cofradías correspondiente, y hereda todas las funcionalidades disponibles para el ciudadano.

Su funcionalidad diferencial es la posibilidad de compartir su ubicación en tiempo real durante las procesiones en las que participa, lo que permite alimentar la geolocalización que consultan el resto de usuarios.

3.1.3. Junta de Cofradías

La Junta de Cofradías es el actor responsable de la gestión del contenido asociado a su cofradía o hermandad dentro de la aplicación. Accede a un panel de gestión desde el que puede crear y editar cofradías, eventos y procesiones, así como definir y modificar sobre el mapa los recorridos de estas últimas.

Asimismo, se encarga de actualizar el estado de las procesiones en tiempo real, gestionar los pasos y los puntos de interés de la ciudad, publicar información sobre cortes de calles, y crear las alertas y notificaciones que reciben los ciudadanos y cofrades. También es responsable de generar, para cada cofradía, su propio código de acceso, y de validar y aprobar mediante dicho código el registro de nuevos cofrades en la cofradía correspondiente.

3.1.4. Administrador

El administrador es el actor con mayor nivel de privilegios dentro del sistema y se encarga de su configuración y mantenimiento a nivel global, por encima del ámbito de una cofradía concreta. Su principal responsabilidad es la gestión de las ciudades disponibles en la aplicación, dando de alta, editando o dando de baja aquellas en las que el sistema ofrece cobertura.

La Tabla 3.1 resume los cuatro actores identificados y su rol dentro del sistema.

Actor	Rol
Ciudadano	Consulta información y sigue la Semana Santa sin necesidad de registro.
Cofrade	Ciudadano perteneciente a una cofradía; comparte su ubicación durante las procesiones.
Junta de Cofradías	Gestiona el contenido de su cofradía: cofradías, eventos, procesiones, recorridos y alertas.
Administrador	Gestiona la configuración global del sistema, en particular las ciudades disponibles.

Tabla 3.1: Resumen de los actores del sistema

3.2. Requisitos

3.2.1. Requisitos funcionales

Los requisitos funcionales definen las diferentes funciones del sistema, además de cómo debe ser su comportamiento en ciertas situaciones. Cada requisito incluye, junto a su descripción, el nivel de prioridad asignado (Alta, Media o Baja), que indica su importancia relativa dentro del proyecto.

Para facilitar su lectura, los requisitos se han agrupado según el perfil de usuario (Sección 3.1) al que van dirigidos –ciudadano, cofrade, Junta de Cofradías o administrador–, y dentro de cada grupo se han ordenado de mayor a menor prioridad. Además el perfil de cofrade incluye, a mayores de los requisitos propios de dicho perfil, todos los requisitos del ciudadano, ya que un cofrade puede hacer uso de todas las funciones generales de la aplicación.

Ciudadano

- **RF-01** (*Prioridad: Alta*) El sistema debe permitir el uso de la aplicación como ciudadano sin la necesidad de realizar un registro previo; únicamente se requiere introducir un código de acceso para acceder al modo cofrade (RF-02).
- **RF-03** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de las cofradías.
- **RF-04** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de los eventos.
- **RF-05** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de las procesiones.
- **RF-06** (*Prioridad: Alta*) El sistema debe permitir a los usuarios realizar búsquedas sobre eventos, procesiones o cofradías específicas.
- **RF-07** (*Prioridad: Alta*) El sistema debe permitir visualizar la posición de las procesiones en tiempo real mediante geolocalización.
- **RF-08** (*Prioridad: Alta*) El sistema debe poder generar rutas hacia la ubicación actual de una procesión.
- **RF-11** (*Prioridad: Alta*) El sistema debe permitir al usuario seleccionar la ciudad que desea consultar.
- **RF-13** (*Prioridad: Alta*) El sistema debe enviar notificaciones al usuario sobre los eventos o procesiones, de acuerdo con sus preferencias de notificación.
- **RF-14** (*Prioridad: Alta*) El sistema debe enviar siempre las alertas sobre los eventos o procesiones.
- **RF-17** (*Prioridad: Alta*) El sistema debe mostrar la información detallada de las procesiones (recorrido, horarios, pasos, ...).
- **RF-18** (*Prioridad: Alta*) El sistema debe solicitar permiso a los usuarios para acceder a su ubicación en tiempo real.

- **RF-09** (*Prioridad: Baja*) El sistema debe poder generar rutas entre dos puntos evitando las procesiones activas.
- **RF-12** (*Prioridad: Media*) El sistema debe permitir a los usuarios marcar cofradías, eventos o procesiones como favoritas, para filtrar la información mostrada en la aplicación y recibir notificaciones sobre ellas.
- **RF-16** (*Prioridad: Media*) El sistema debe permitir al usuario interactuar con el mapa (zoom, desplazamiento, ...).
- **RF-19** (*Prioridad: Media*) El sistema debe seleccionar automáticamente la ciudad más cercana en función de la ubicación del usuario.
- **RF-22** (*Prioridad: Media*) El sistema debe permitir al usuario suscribirse o desuscribirse de las notificaciones sobre cofradías, eventos o procesiones concretas, para recibir únicamente las que le interesen.
- **RF-23** (*Prioridad: Media*) El sistema debe permitir filtrar cofradías, eventos y procesiones por distintos criterios (día, hora, tipo, etc.).
- **RF-10** (*Prioridad: Baja*) El sistema debe poder sugerir rutas hacia las ubicaciones más relevantes de una procesión (inicio, puntos de interés, fin, ...).
- **RF-25** (*Prioridad: Baja*) El sistema debe mostrar en el mapa puntos de interés cuando el usuario no haya seleccionado ninguna cofradía, evento o procesión.
- **RF-35** (*Prioridad: Alta*) El sistema debe mostrar un calendario con los días de la Semana Santa, indicando en cada día si hay eventos o procesiones programadas.
- **RF-36** (*Prioridad: Alta*) El sistema debe permitir a los usuarios seleccionar un día concreto del calendario para consultar el listado de procesiones programadas ese día.
- **RF-37** (*Prioridad: Media*) El sistema debe permitir a los usuarios acceder, desde un menú principal, a los listados completos de procesiones, pasos, eventos y cofradías.

Cofrade

- **RF-02** (*Prioridad: Alta*) El sistema debe permitir a los cofrades de la cofradía acceder al modo cofrade mediante el código de acceso.
- **RF-15** (*Prioridad: Alta*) El sistema debe permitir a los cofrades compartir, y dejar de compartir en cualquier momento, su ubicación en tiempo real durante las procesiones.
- **RF-38** (*Prioridad: Media*) El sistema debe detectar e ignorar las geolocalizaciones erróneas o inconsistentes recibidas de los cofrades.

Junta de Cofradías

- **RF-20** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías editar información de cofradías, eventos, procesiones y de la Semana Santa de su ciudad.
- **RF-21** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías crear y eliminar cofradías, eventos y procesiones.
- **RF-27** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías crear y eliminar alertas dirigidas a todos los usuarios sobre incidencias o cambios en eventos y procesiones; las notificaciones, en cambio, dependen de la suscripción de cada usuario (RF-22).
- **RF-28** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías actualizar el estado de las procesiones en tiempo real (programada, en curso, finalizada o cancelada).
- **RF-30** (*Prioridad: Alta*) El sistema debe permitir a la Junta de Cofradías definir y modificar los recorridos de las procesiones en el mapa, como parte de la configuración de la información estática de la aplicación antes del inicio de la Semana Santa.
- **RF-29** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías gestionar los puntos de interés de las procesiones.
- **RF-31** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías gestionar la información de los pasos de cada cofradía.
- **RF-33** (*Prioridad: Media*) El sistema debe generar automáticamente un código de acceso para cada cofradía al darla de alta (RF-21), para que sus cofrades puedan acceder al modo cofrade.
- **RF-39** (*Prioridad: Media*) El sistema debe permitir a la Junta de Cofradías gestionar la información de su propio perfil.

Administrador

- **RF-26** (*Prioridad: Alta*) El sistema debe permitir al administrador gestionar las ciudades disponibles y las juntas asociadas a ellas en la aplicación.
- **RF-34** (*Prioridad: Alta*) El sistema debe permitir al administrador dar de alta, editar y dar de baja las Juntas de Cofradías del sistema.
- **RF-40** (*Prioridad: Alta*) El sistema debe permitir al administrador gestionar (dar de alta, editar y dar de baja) a los miembros de las Juntas de Cofradías.
- **RF-41** (*Prioridad: Baja*) El sistema debe permitir al administrador gestionar la información general de la aplicación (por ejemplo, textos legales o de ayuda).

3.2.2. Requisitos de información

Los requisitos de información definen los datos que el sistema debe almacenar y gestionar para su correcto funcionamiento. Al igual que en los requisitos funcionales, se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. En este caso no se agrupan por perfil de usuario, ya que describen el modelo de datos del sistema en su conjunto y no están asociados a un único actor.

- **RI-01** (*Prioridad: Alta*) El sistema debe almacenar la información sobre los cofrades de las cofradías/hermandades:
 - Identificador de usuario.
 - Cofradía a la que pertenece.
 - Código de acceso validado que le da acceso al modo cofrade.
 - Fecha de ingreso.
 - Si está compartiendo su ubicación en ese momento.
 - Posición actual (latitud, longitud e instante de la lectura) mientras comparte su ubicación durante una procesión.
- **RI-02** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las cofradías/hermandades:
 - Identificador.
 - Nombre.
 - Historia.
 - Archivo de información.
 - Fecha de creación.
 - Junta de Cofradías a la que pertenece.
 - Pasos, eventos y procesiones que organiza.
- **RI-03** (*Prioridad: Alta*) El sistema debe almacenar la información sobre los eventos:
 - Identificador.
 - Nombre.
 - Descripción.
 - Fecha.
 - Estado (programado, en curso, finalizado o cancelado).
 - Cofradía que lo organiza.
- **RI-04** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las procesiones:
 - Fecha de inicio y fecha de fin.
 - Estado (programada, en curso, finalizada o cancelada).
 - Evento al que sigue.

- Cofradía a la que pertenece.
 - Pasos que desfilan en ella.
 - Recorrido asociado.
 - Posición actual en tiempo real mientras está en curso.
- **RI-05** (*Prioridad: Alta*) El sistema debe almacenar la información sobre los recorridos:
 - Identificador.
 - Nombre.
 - Distancia total.
 - Tiempo estimado.
 - Puntos de ruta que lo componen (ubicaciones y puntos de interés), cada uno con la hora prevista de paso.
 - **RI-08** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las alertas:
 - Identificador.
 - Título.
 - Fecha de creación.
 - Tipo de alerta (incidencia, cambio de horario, cancelación, corte de calle, meteorología o seguridad).
 - Prioridad (baja, media, alta o urgente).
 - Para cada usuario al que se entrega: si ha sido leída y la fecha de lectura.
 - **RI-09** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las Juntas de Cofradías/hermandades:
 - Identificador.
 - Nombre.
 - Email.
 - Teléfono.
 - Ciudad que gestiona.
 - Cofradías que agrupa.
 - **RI-10** (*Prioridad: Alta*) El sistema debe almacenar la información sobre las ciudades:
 - Identificador.
 - Nombre.
 - Comunidad autónoma.
 - Descripción.
 - Junta de Cofradías que la gestiona.
 - **RI-06** (*Prioridad: Media*) El sistema debe almacenar la información sobre los puntos de interés y demás ubicaciones de referencia utilizadas en los recorridos y en el mapa:

- Latitud.
 - Longitud.
 - Dirección.
- **RI-07** (*Prioridad: Media*) El sistema debe almacenar la información sobre los pasos:
 - Identificador.
 - Nombre.
 - Descripción.
 - Imagen.
 - Cofradía a la que pertenece.
 - Procesiones en las que desfila.

3.2.3. Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad del sistema, así como restricciones sobre su funcionamiento. También se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. Al igual que los requisitos de información, no se agrupan por perfil de usuario, ya que son de aplicación transversal a todo el sistema.

- **RNF-01** (*Prioridad: Alta*) El sistema debe garantizar la seguridad de los datos de los usuarios, especialmente en lo relativo a la información de ubicación.
- **RNF-02** (*Prioridad: Alta*) La aplicación debe ser accesible y fácil de usar para usuarios sin experiencia previa.
- **RNF-05** (*Prioridad: Alta*) El sistema debe implementar mecanismos de control de acceso que diferencien entre ciudadanos, cofrades, miembros de la junta de cofradías y administrador.
- **RNF-07** (*Prioridad: Alta*) El sistema debe ser compatible tanto con dispositivos Android como con dispositivos iOS.
- **RNF-08** (*Prioridad: Alta*) El sistema debe actualizar la posición de las procesiones con una frecuencia mínima de 30 segundos¹.
- **RNF-03** (*Prioridad: Media*) El sistema debe funcionar correctamente en dispositivos móviles con conexión a internet. Sin conexión, únicamente estará disponible la información estática de la aplicación (previamente almacenada en local), quedando inoperativas la geolocalización en tiempo real, las notificaciones y las alertas.
- **RNF-04** (*Prioridad: Media*) El sistema debe gestionar de forma eficiente la carga del mapa, garantizando tiempos de respuesta menores a 10 segundos².

¹Este intervalo se ha elegido como equilibrio entre ofrecer una posición suficientemente precisa para el usuario y evitar un consumo excesivo de batería y datos móviles en los dispositivos de los cofrades que comparten su ubicación.

²Valor tomado como referencia habitual de usabilidad para evitar que el usuario perciba la aplicación como lenta al cargar el mapa.

- **RNF-06 (Prioridad: Media)** El sistema debe ser capaz de enviar notificaciones de manera fiable y en tiempo adecuado (menos de 60 segundos de latencia).

3.3. Casos de uso

Los casos de uso son los escenarios detallados que describen cómo interactúan los diferentes usuarios con la aplicación, desde la consulta de información hasta la gestión administrativa de la Semana Santa. Debido a que los usuarios pueden comportarse con diferentes roles (ciudadano, cofrade, Junta de Cofradías o administrador), es importante diferenciar los casos de uso entre cada uno de ellos. Para ello se han dividido los mismos en cuatro diagramas (ver figuras 3.1, 3.2, 3.3, 3.4), uno para cada rol.

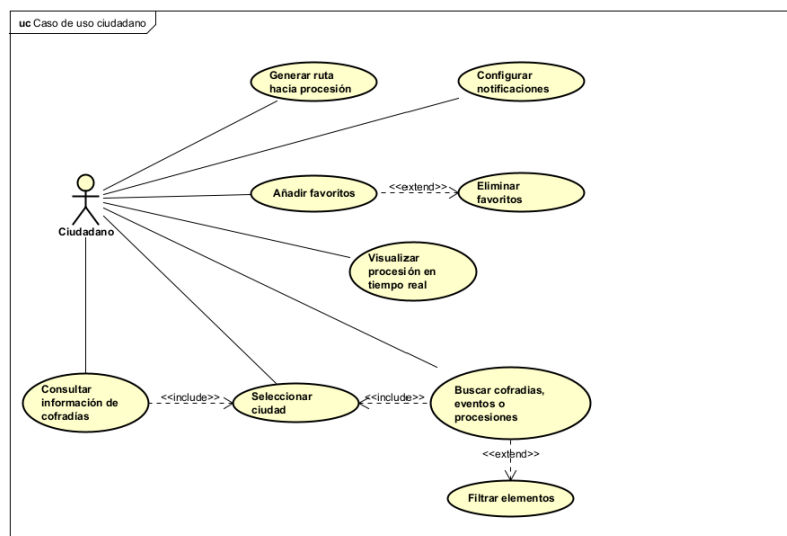


Figura 3.1: Diagrama de caso de uso Ciudadano

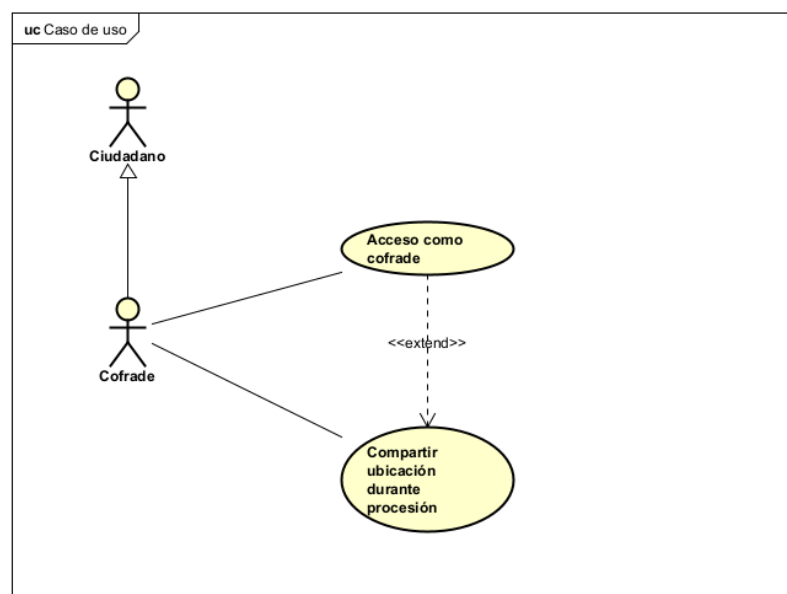


Figura 3.2: Diagrama de caso de uso Cofrade

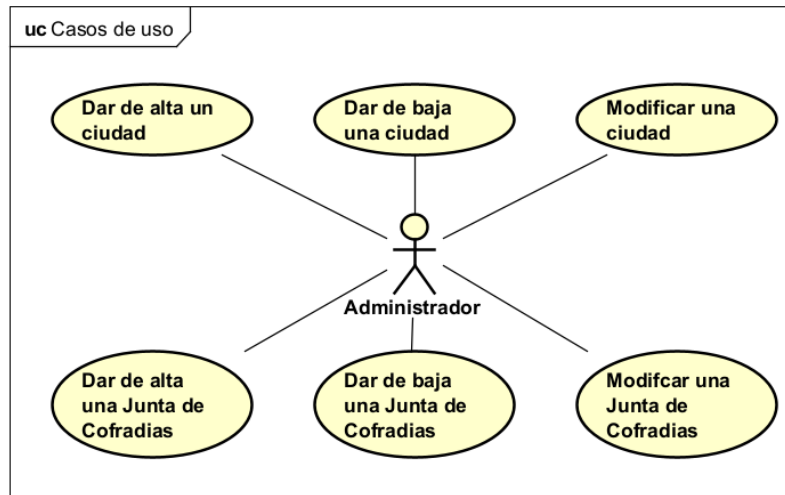


Figura 3.3: Diagrama de caso de uso Junta de Cofradías

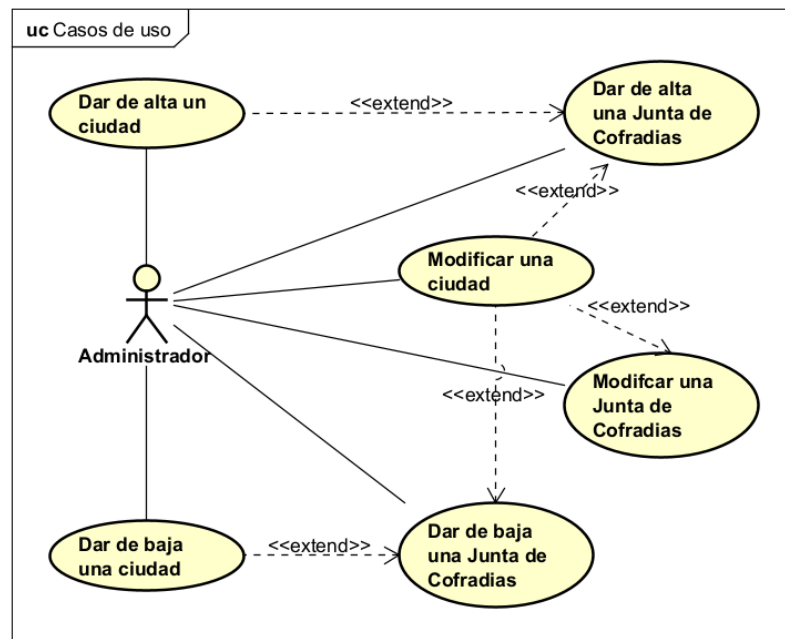


Figura 3.4: Diagrama de caso de uso Administrador

En las siguientes tablas se exponen los cinco casos de uso más representativos del sistema, uno por cada rol implicado (ciudadano, cofrade, Junta de Cofradías y administrador), donde se detallan: el caso de uso, los actores que lo realizan, las precondiciones necesarias para iniciar el caso de uso, las postcondiciones que deben cumplirse al completarse, una secuencia base de pasos, secuencias alternativas que pueden darse durante la ejecución y los requisitos relacionados con el caso de uso. El resto de los casos de uso identificados, de carácter más secundario, se recogen en el Apéndice B.

Caso de uso: Visualizar procesión en tiempo real	
Resumen	El usuario visualiza la posición actual de una procesión en el mapa.
Actor	Ciudadano
Precondición	La procesión está en curso y el usuario tiene conexión a internet.
Postcondición	Se muestra la posición actual de la procesión sobre el mapa.
Secuencia Base	
[1] El usuario selecciona una procesión activa.	
[2] El sistema muestra el mapa con el recorrido de la procesión.	
[3] El sistema actualiza la posición en tiempo real.	
[4] El usuario puede interactuar con el mapa (zoom, desplazamiento).	
Secuencia Alternativa	
[1.a.1] La procesión no está activa: El sistema muestra el recorrido planificado sin posición en tiempo real.	
[3.a.1] Pérdida de conexión: El sistema muestra la última posición conocida.	
Requisitos Relacionados	RF-05, RF-07 y RF-16.

Tabla 3.2: Caso de uso: Visualizar procesión en tiempo real

3.4. Modelo de dominio

En esta sección se detalla el diseño conceptual del sistema, incluyendo las entidades clave y sus relaciones. El modelo refleja la estructura lógica del dominio de la Semana Santa, garantizando una representación fiel de sus procesos, actores y reglas de negocio. Se explican brevemente las abstracciones incluidas en la Figura 3.5.

Caso de uso: Compartir ubicación durante procesión	
Resumen	Un cofrade comparte su ubicación en tiempo real durante una procesión.
Actor	Cofrade
Precondición	El cofrade está registrado, el cofrade tiene permisos de ubicación y hay una procesión de su cofradía en curso
Postcondición	La ubicación del cofrade se transmite al sistema.
Secuencia Base	
[1] El cofrade accede a la procesión activa de su cofradía.	
[2] El cofrade activa la opción de compartir ubicación.	
[3] El sistema solicita permisos de ubicación si no los tiene.	
[4] El cofrade concede los permisos.	
[5] El sistema comienza a transmitir la ubicación.	
[6] El sistema muestra indicador de ubicación compartida, desde el que el cofrade puede dejar de compartirla en cualquier momento.	
Secuencia Alternativa	
[4.a.1] El usuario deniega permisos: El sistema informa que no puede compartir ubicación sin permiso.	
[2.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Pérdida de conexión: El sistema muestra la última posición conocida.	
Requisitos Relacionados	RF-15 y RF-18.

Tabla 3.3: Caso de uso: Compartir ubicación durante procesión

Caso de uso: Crear alerta	
Resumen	La Junta de Cofradías crea una alerta para informar a los usuarios sobre incidencias o información relevante.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías.
Postcondición	La alerta queda creada y se envían notificaciones a los usuarios afectados.
Secuencia Base	
[1] El usuario accede al panel de gestión de alertas.	
[2] El usuario selecciona crear nueva alerta.	
[3] El sistema muestra el formulario de creación.	
[4] El usuario selecciona el tipo de alerta.	
[5] El usuario introduce título y mensaje de la alerta.	
[6] El usuario selecciona la prioridad de la alerta.	
[7] El usuario define fecha de expiración (opcional).	
[8] El usuario selecciona los destinatarios.	
[9] El usuario confirma la creación.	
[10] El sistema crea la alerta y envía notificaciones.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[6.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[7.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[8.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[9.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
[10.a.1] Error al enviar notificaciones: El sistema reintenta y registra el error.	
Requisitos Relacionados	RF-27.

Tabla 3.4: Caso de uso: Crear alerta

Caso de uso: Acceso como cofrade	
Resumen	Un usuario se registra en la aplicación como miembro de una cofradía.
Actor	Cofrade
Precondición	El usuario no está registrado.
Postcondición	El usuario queda registrado y puede acceder a funciones de cofrade.
Secuencia Base	
[1] El usuario selecciona la opción de registro.	
[2] El sistema muestra el formulario de registro (Introducir código de acceso para dicha cofradía).	
[3] El usuario introduce el código de acceso de la cofradía.	
[4] El sistema valida el código.	
[5] El sistema muestra mensaje informando del correcto acceso.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Código inválido: El sistema muestra los errores de validación.	
Requisitos Relacionados	RF-02.

Tabla 3.5: Caso de uso: Acceso como cofrade

Caso de uso: Gestionar ciudades disponibles	
Resumen	El administrador da de alta, edita o retira ciudades disponibles en la aplicación.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador.
Postcondición	La lista de ciudades disponibles queda actualizada en el sistema.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de ciudades existentes.	
[3] El usuario selecciona añadir, editar o eliminar una ciudad.	
[4] El usuario introduce o modifica los datos de la ciudad.	
[5] El sistema valida y guarda los cambios.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
[3.a.1] Ciudad con datos asociados: El sistema advierte antes de eliminar una ciudad que tenga cofradías, eventos o procesiones registrados.	
Requisitos Relacionados	RF-26.

Tabla 3.6: Caso de uso: Gestionar ciudades disponibles

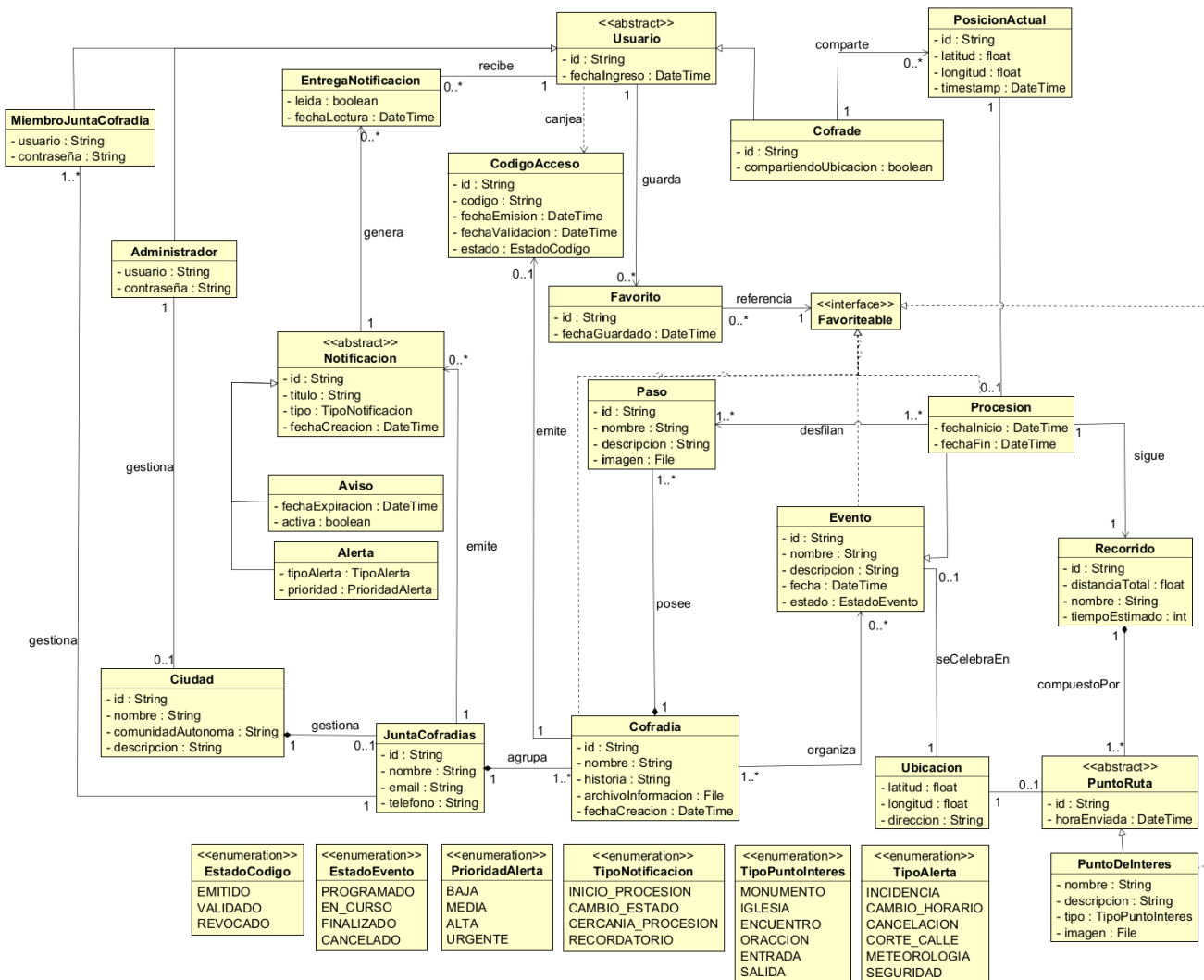


Figura 3.5: Diagrama de dominio del proyecto

Nota: la construcción del backend (Sección 4.1) obligó a revisar varias relaciones de este diagrama conforme se traducían a un modelo relacional; el texto de esta sección ya refleja esos cambios, pero la imagen de la Figura 3.5 está pendiente de redibujarse en consecuencia.

El modelo de dominio representa las principales entidades del sistema y las relaciones existentes entre ellas. La clase Ciudad agrupa las procesiones y actos propios de cada localidad, y agrupa también directamente las distintas Cofradia de esa ciudad; JuntaCofradias es quien gestiona esa ciudad (relación 1:1), pero no es ella quien agrupa a las cofradías, de forma que una renovación de la Junta no afecta a los datos de las cofradías que gestiona. Cada Cofradia puede participar en varios Evento y un mismo Evento puede tener más de una Cofradia participando –relación de muchos a muchos, pensada para actos conjuntos entre varias hermandades–, y posee uno o varios Paso, que pueden desfilan en varias procesiones distintas, de ahí también la relación de muchos a muchos entre Paso y Procesion. Procesion **extiende** a Evento –hereda sus atributos y sus cofradías participantes, y añade únicamente su fecha de inicio y fin propias– en lugar de referenciarlo mediante una simple asociación, y tiene asociado un Recorrido, compuesto a su vez por distintos PuntoRuta: un mismo punto puede formar parte de varios recorridos, por

ejemplo cuando dos procesiones comparten un tramo de calle. `PuntoDeInteres` extiende a `PuntoRuta` añadiendo nombre, descripción, imagen y una categoría, para los puntos que merece la pena mostrar con detalle; `Ubicacion` ya no es un subtipo de `PuntoRuta`, sino una clase independiente y reutilizable (latitud, longitud y dirección) referenciada tanto por `PuntoRuta` como por `Evento`.

Para la geolocalización en tiempo real, un `Cofrade` autenticado con el código de acceso de su cofradía puede compartir su ubicación mientras sigue una procesión; cada lectura se registra como una `PosicionActual`, sin ninguna referencia a la persona que la envió. La posición que consultan el resto de usuarios de la aplicación no es ninguna `PosicionActual` guardada en concreto, sino el promedio calculado a partir de las lecturas más recientes de esa procesión.

La comunicación con los usuarios se modela mediante la clase abstracta `Notificacion`, de la que heredan `Aviso` y `Alerta`: los avisos son mensajes informativos con una fecha de expiración, mientras que las alertas incorporan un tipo (incidencia, cambio de horario, cancelación, corte de calle, meteorología o seguridad) y una prioridad. Cada notificación entregada a un usuario se registra mediante una `EntregaNotificacion`, que almacena si ha sido leída y en qué momento, evitando así duplicar la notificación en sí para cada destinatario.

Por último, el acceso al modo `cofrade` se controla mediante la clase `CodigoAcceso`: la Junta de Cofradías de la ciudad emite, cuando lo necesita, un código en nombre de una de sus cofradías –`Cofradia` no tiene cuenta propia con la que emitirlo ella misma–, y quien lo valida obtiene los privilegios de `cofrade` sin llegar a registrarse como `Usuario`. De forma similar, los elementos marcados como favoritos por un usuario (cofradías, eventos o procesiones) se representan mediante la clase `Favorito`, que hace referencia a cualquier elemento que implemente la interfaz `Favoriteable`.

3.4.1. Modelo de interacción

En esta sección se describen los principales casos de uso de la aplicación mediante diagramas de secuencia obtenidos a partir del análisis. Se han excluido las secuencias alternativas de cancelar con el fin de simplificar los diagramas.

Para el caso de uso *Visualizar procesión en tiempo real* (Figura 3.6), el ciudadano selecciona una procesión activa y el sistema obtiene la instancia de `Procesion` correspondiente, junto con su recorrido planificado, y se suscribe a las actualizaciones de su posición actual. Mientras la procesión está en curso, el sistema recalcula sobre el recorrido el tramo ya realizado y el tramo restante en función de la posición recibida, para mantener el mapa del usuario actualizado.

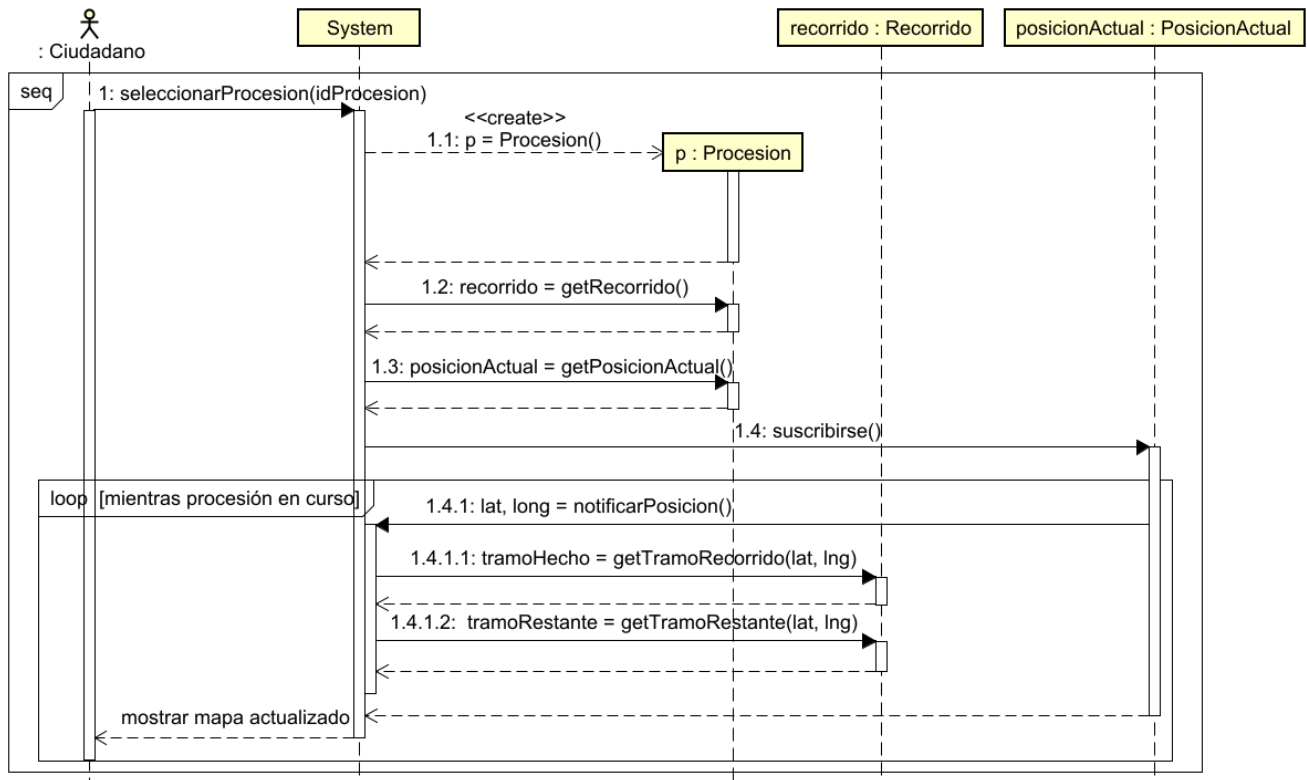


Figura 3.6: Diagrama de secuencia del CU “Visualizar procesión en tiempo real”

En el caso de uso *Compartir ubicación durante procesión* (Figura 3.7), el cofrade accede a la procesión activa de su cofradía y activa la opción de compartir su ubicación. El sistema solicita los permisos de localización si no los tiene concedidos: si el cofrade los deniega, se le informa de que no es posible compartir la ubicación sin permiso; si los concede, el sistema comienza a actualizar de forma periódica la posición del cofrade mientras la procesión sigue en curso.

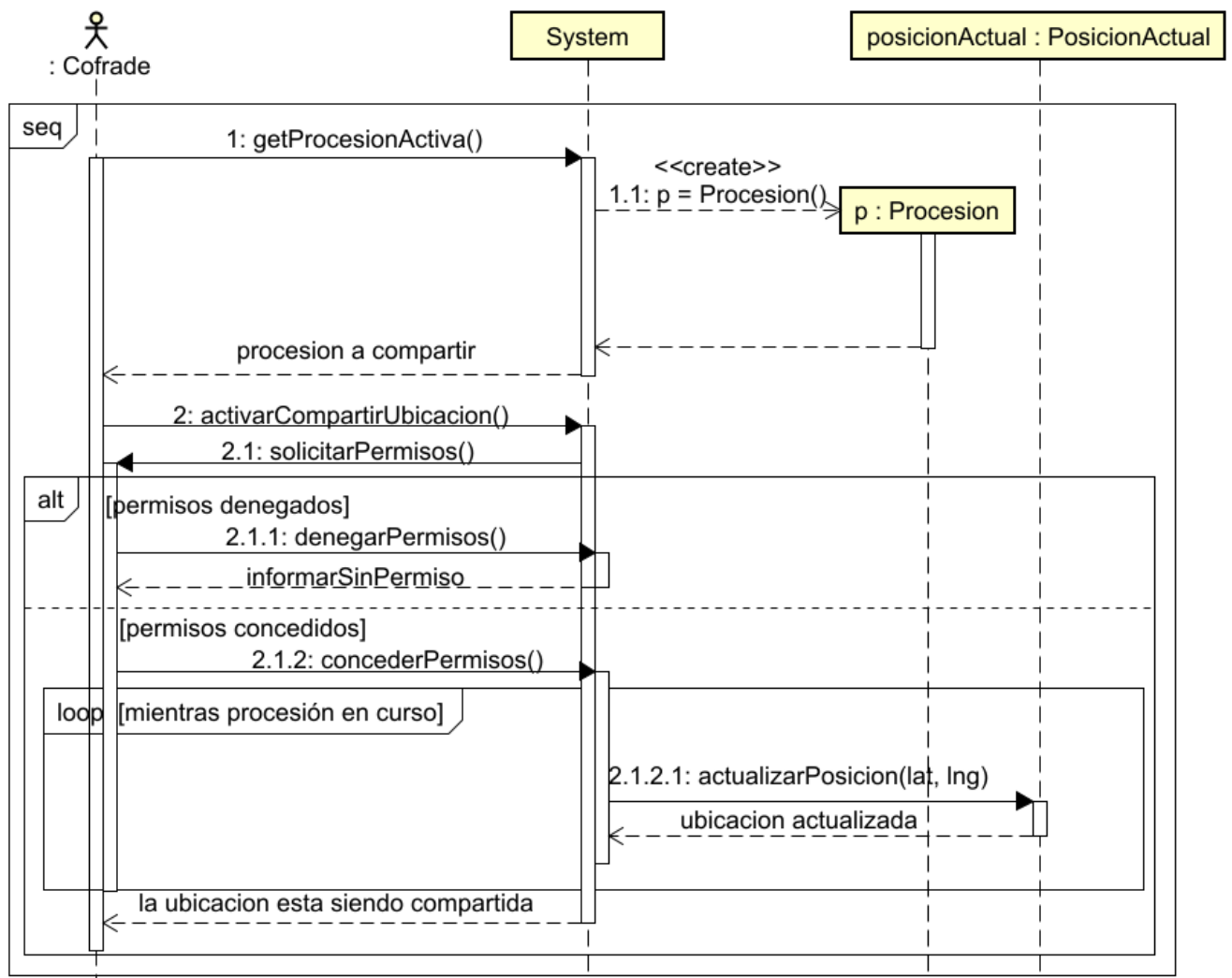


Figura 3.7: Diagrama de secuencia del CU “Compartir ubicación durante procesión”

En el caso de uso *Acceso como cofrade* (Figura 3.8), el usuario selecciona la opción de registro y el sistema le muestra el formulario correspondiente. Al introducir el código de acceso de su cofradía, el sistema obtiene el `CodigoAcceso` correspondiente y lo valida, informando al usuario del acceso correcto o de un error de validación según el resultado.

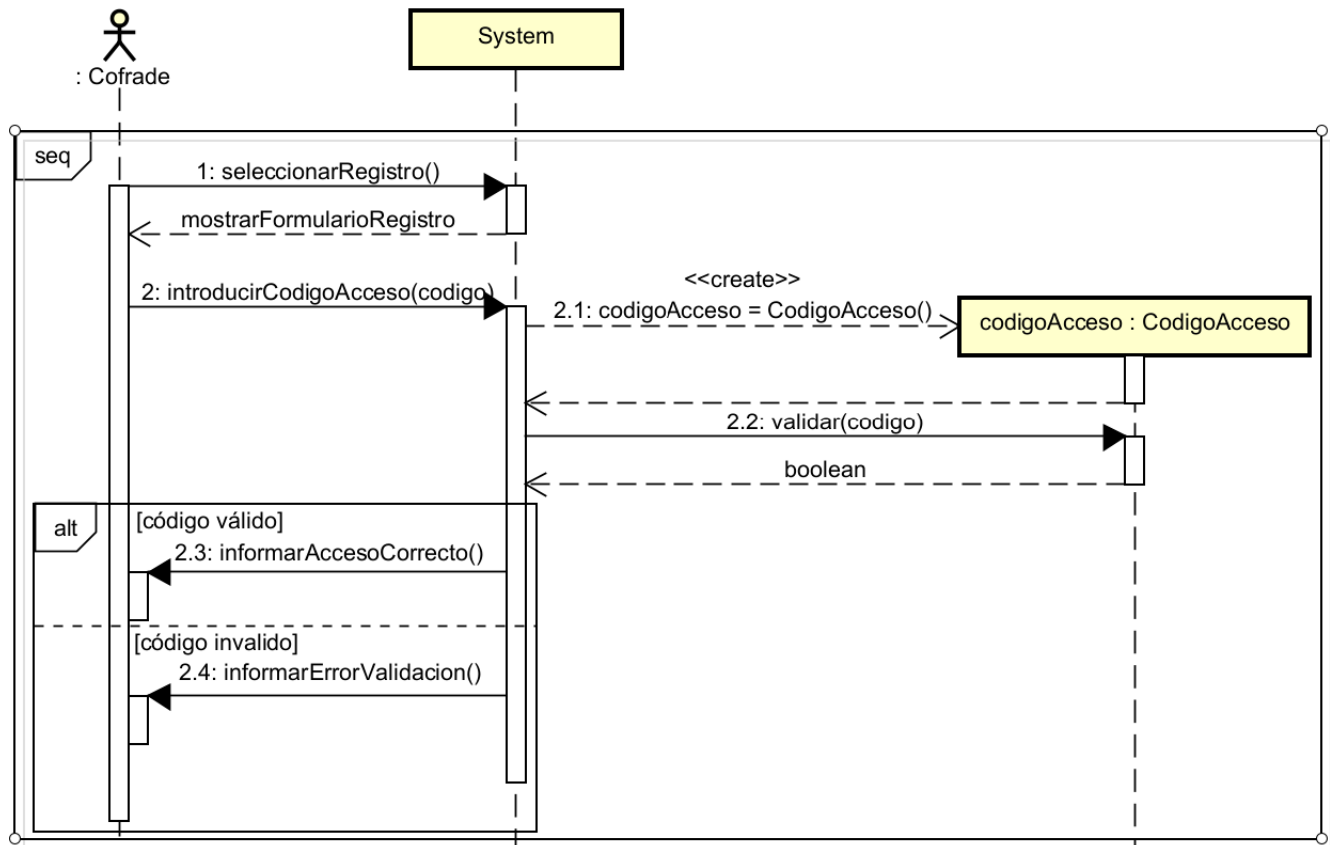


Figura 3.8: Diagrama de secuencia del CU “Acceso como cofrade”

En el caso de uso *Dar de alta ciudad* (Figura 3.9), el administrador accede al panel de administración, donde el sistema le muestra la lista de ciudades existentes, y selecciona la opción de añadir una nueva ciudad. Tras introducir los datos, el sistema construye la instancia de Ciudad correspondiente y valida los datos introducidos, repitiendo la solicitud hasta que estos son válidos, momento en el que la ciudad queda dada de alta.

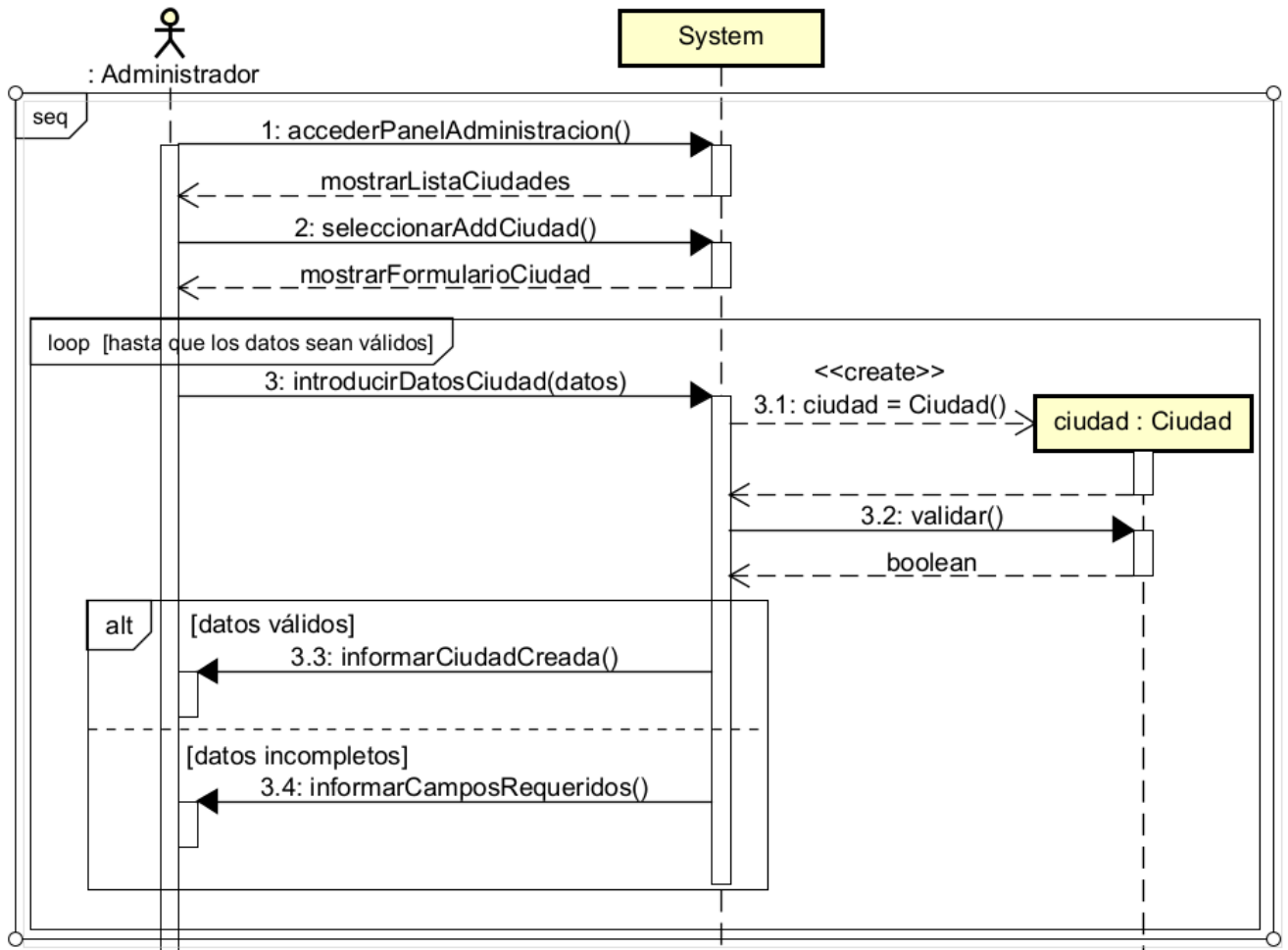


Figura 3.9: Diagrama de secuencia del CU “Dar de alta ciudad”

Capítulo 4

Descripción de las iteraciones

4.1. 1ª Iteración

4.1.1. Análisis

4.1.2. Diseño

Antes de comenzar la implementación de esta primera iteración ha sido necesario tomar una serie de decisiones de diseño que condicionan el resto del proyecto: qué tecnologías se van a usar, cómo se organiza el código, cómo se guardan los datos y qué soluciones ya conocidas (patrones de diseño) conviene aplicar. Estas decisiones se explican a continuación con el detalle suficiente para que puedan entenderse sin conocimientos previos de React Native o Firebase, ya que sientan la base para el resto de iteraciones del proyecto.

Decisiones tecnológicas

Para el cliente¹ móvil se ha optado por **React Native** [36], un *framework*² que permite escribir una única aplicación y ejecutarla tanto en Android como en iOS, en lugar de programar dos aplicaciones nativas independientes (una en Kotlin/Java para Android y otra en Swift para iOS). Se descarta el desarrollo nativo duplicado por dos motivos: el requisito RNF-07 exige que la aplicación funcione en ambas plataformas, y al tratarse de un proyecto desarrollado por una única persona, mantener y probar dos bases de código distintas multiplicaría el tiempo de desarrollo sin aportar ninguna ventaja funcional.

Dentro de React Native, el código se construye con **Expo** [37], siguiendo la recomendación de la propia documentación oficial de React Native, que aconseja apoyarse en un *framework* sobre React Native –en lugar de configurar manualmente el entorno nativo de cada plataforma– y señala a Ex-

¹Cliente: en una arquitectura cliente-servidor, es la parte del sistema con la que interactúa directamente el usuario; en este caso, la propia aplicación instalada en el teléfono móvil.

²Framework: conjunto de herramientas y bibliotecas de código que facilitan el desarrollo de un tipo concreto de aplicación, proporcionando una estructura base sobre la que trabajar.

po como la vía más sencilla para empezar, especialmente para quien no tiene experiencia previa en desarrollo móvil [36]. Expo proporciona, entre otras cosas, una biblioteca estándar de módulos nativos ya integrados (cámara, notificaciones, geolocalización), un proceso de compilación gestionado (*EAS Build*) que evita instalar y mantener Android Studio y Xcode de forma completa para generar el ejecutable, y la posibilidad de publicar actualizaciones del código JavaScript sin pasar de nuevo por las tiendas de aplicaciones. Por esta razón, para la geolocalización del dispositivo se usa concretamente *expo-location*, el módulo de la propia biblioteca estándar de Expo, en lugar de una alternativa externa como *react-native-geolocation-service*; para la integración con mapas se mantiene *react-native-maps*, compatible con proyectos Expo.

Como *backend*³ se ha optado por desarrollar una **API REST propia** con **Spring Boot** [38], en lugar de apoyarse en una plataforma *Backend-as-a-Service* (BaaS) como Firebase. Esta decisión se reconsideró durante la fase de diseño de la primera iteración: una plataforma BaaS agiliza notablemente el desarrollo porque resuelve de fábrica aspectos como el modelo de datos, la autenticación o el control de acceso, pero precisamente por eso deja poco margen para diseñar y justificar esas mismas decisiones, que son el núcleo de la formación de la mención de Ingeniería del Software cursada en este Grado: arquitectura en capas, modelado relacional de los datos, autenticación y autorización propias, y diseño de una API. Se ha optado por tanto por construir esa capa de backend desde cero, reservando los servicios de terceros (Firebase) únicamente para aquello que no aporta valor académico programarlo desde cero: el almacenamiento de imágenes (**Firestore Storage** [39]) y el envío de notificaciones *push* (**Firestore Cloud Messaging** [29]). Esta combinación –backend propio más un uso puntual y acotado de servicios externos– es habitual en proyectos reales y permite demostrar competencias de desarrollo de backend sin invertir tiempo en infraestructura ya resuelta de forma robusta por terceros.

La persistencia de los datos se apoya en **PostgreSQL** [40], un sistema gestor de bases de datos **relacional**, frente a alternativas NoSQL como MongoDB o Firestore. Se ha preferido un modelo relacional porque el dominio del proyecto (Sección 3) está compuesto por entidades fuertemente relacionadas entre sí –una ciudad agrupa cofradías, una cofradía organiza eventos y procesiones, una procesión sigue un recorrido compuesto por puntos de ruta, etc.– con relaciones que conviene que la propia base de datos garantice mediante claves foráneas y restricciones de integridad referencial, en lugar de resolverlas a mano en el código de la aplicación como sería necesario en un modelo documental. El acceso a la base de datos se realiza mediante **Spring Data JPA** [41] sobre **Hibernate** [42], un *ORM*⁴ que evita escribir a mano la mayor parte de las consultas SQL.

La autenticación y autorización de la API se resuelve con **Spring Security** [43] y tokens **JWT** (*JSON Web Token*) [44]: al validar sus credenciales (obtenidas, como se explica en el apartado Diseño de la base de datos, al introducir un código de acceso válido), el usuario recibe un token firmado por el servidor que incluye su identificador y su rol; ese token se envía en cada petición posterior a un recurso protegido, y es el propio servidor quien lo valida y comprueba el rol antes de autorizar la operación. Se trata de un mecanismo sin estado (*stateless*): el servidor no necesita guardar la sesión de cada usuario, lo que

³Backend: la parte del sistema que no ve el usuario directamente, encargada de almacenar los datos, gestionar la lógica de negocio y responder a las peticiones que le hace el cliente.

⁴ORM (*Object-Relational Mapping*): técnica que traduce automáticamente entre los objetos de un lenguaje de programación orientado a objetos (Java, en este caso) y las tablas de una base de datos relacional, evitando escribir manualmente la mayoría de las sentencias SQL.

simplifica el despliegue y la futura escalabilidad de la API. Este mecanismo cubre el requisito RNF-05 (control de acceso diferenciado por rol), del mismo modo que antes lo hacían las reglas de seguridad de Firestore, pero ahora la lógica de autorización reside y se puede inspeccionar en el propio código del backend.

Para los mapas se mantiene **Google Maps Platform** [1], ya justificado en el estado de la cuestión (Sección 1.3.2), por ser la solución de mapas más extendida y con mejor documentación. La contingencia prevista ante el riesgo “Problemas con la integración de Google Maps” (Sección 2.3) sigue siendo migrar a OpenStreetMap [19] o Mapbox si en la práctica surgen problemas de cuota o de coste.

Por último, la posición en vivo de una procesión (RNF-08: actualización cada 30 segundos como mínimo) no exige por sí misma un mecanismo de *push* en tiempo real: se resuelve mediante una API REST convencional, en la que la aplicación del cofrade envía su posición con una petición `POST /procesiones/{id}/ubicacion` cada 30 segundos, y la aplicación del ciudadano consulta la posición agregada con una petición `GET /procesiones/{id}/ubicacion` con la misma cadencia. Esta solución es más sencilla de construir y de defender que montar un servidor de *sockets* propio (WebSockets o *Server-Sent Events*), y es suficiente para el requisito tal y como está definido; si en el futuro se quisiera ofrecer una actualización percibida como instantánea o escalar a un número de usuarios muy superior, esa migración se recoge como línea de trabajo futuro (Sección 7.1).

Arquitectura del sistema

La arquitectura de un sistema describe, a grandes rasgos, en qué piezas se divide y cómo se comunican entre sí. En este proyecto se ha definido una arquitectura **cliente-servidor con servidor propio**: la aplicación React Native (el cliente) se comunica mediante peticiones `HTTPS`⁵ a la API REST propia, desarrollada con Spring Boot, que expone un conjunto de *endpoints*⁶ organizados por recurso (`/ciudades`, `/cofrades`, `/procesiones`, `/usuarios`, `/notificaciones`, etc.). Es la propia API quien centraliza toda la lógica de negocio y quien, y solo quien, se comunica con la base de datos PostgreSQL: en ningún caso el cliente accede directamente a la base de datos. De forma independiente, el cliente también se comunica directamente con los SDK⁷ de Firebase Storage (para leer y subir imágenes) y de Firebase Cloud Messaging (para registrarse y recibir notificaciones *push*), y con el SDK de Google Maps Platform (visualización de mapas y cálculo de rutas); estos tres servicios se mantienen como se justifica en el apartado anterior, y no pasan por la API propia salvo para persistir la referencia resultante (por ejemplo, la URL de una imagen ya subida).

Estructura del front-end Dentro del propio cliente, el código se organiza siguiendo una **arquitectura en capas** [11]: el proyecto se divide en varios grupos de carpetas (o *paquetes*), cada uno con una responsabilidad concreta, de forma que cada capa solo puede depender de la capa inmediatamente inferior

⁵HTTPS: versión segura (cifrada) del protocolo HTTP, usado habitualmente para la comunicación entre aplicaciones y servicios en internet.

⁶Endpoint: dirección concreta de una API a la que el cliente envía una petición para realizar una operación determinada (por ejemplo, obtener el listado de procesiones de una ciudad).

⁷SDK (*Software Development Kit*): conjunto de herramientas que proporciona un servicio para que otras aplicaciones puedan integrarse con él fácilmente.

y nunca al revés. Esta regla se sigue para que un cambio en una capa concreta afecte al menor número posible de partes del código, y para poder probar cada capa de forma aislada. Las cuatro capas definidas son:

- **UI** (`screens`, `components`, `navigation`): las pantallas que ve el usuario, agrupadas por su rol (ciudadano, cofrade, junta de cofradías, administrador), siguiendo los cuatro diagramas de casos de uso del Capítulo 3; los componentes visuales reutilizables entre pantallas; y la navegación entre pantallas.
- **Application** (`context`, `hooks`): el estado que se comparte entre varias pantallas a la vez, como la sesión del usuario (token JWT y rol activo), la ciudad seleccionada o si está compartiendo su ubicación en ese momento.
- **Data** (`services`, `models`): un servicio por cada tipo de información del dominio (`cofradiaService`, `procesionService`, `geoService`, `notificationService`...) que es el único responsable de hablar con la API propia –mediante peticiones HTTP gestionadas y cacheadas con TanStack Query [45]– y, en los dos casos puntuales ya comentados, con Firebase Storage y Firebase Cloud Messaging; además de los tipos de datos que describen cómo es cada objeto del dominio dentro de la aplicación.
- **Infrastructure** (`api`, `firebase`, `maps`): la configuración e inicialización de los distintos SDK y clientes externos, es decir, el código mínimo necesario para que el resto de la aplicación pueda empezar a usarlos. El paquete `api` contiene el cliente HTTP (basado en `fetch/axios`) configurado para incluir automáticamente el token JWT en cada petición y para redirigir al *login* si el servidor responde con un error de autenticación.

La Figura 4.1 muestra esta organización en forma de diagrama de paquetes, donde las flechas discontinuas indican qué paquete depende de (necesita) cuál.

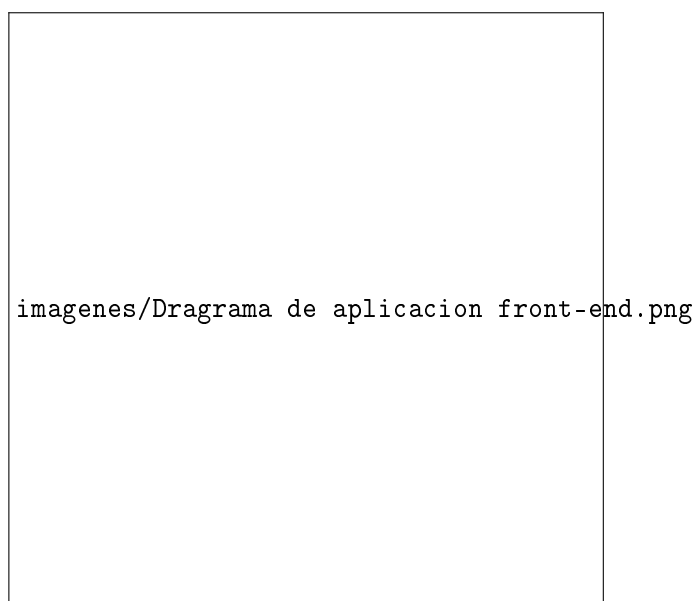


Figura 4.1: Diagrama de paquetes de la aplicación cliente

Estructura del back-end A diferencia del planteamiento inicial basado en Firebase, el back-end de este proyecto sí incluye código propio, organizado también en capas siguiendo el patrón habitual de una API Spring Boot [11]:

- **Controller:** expone los *endpoints* REST de cada recurso, recibe las peticiones HTTP, delega la lógica en la capa de servicio y traduce el resultado a la respuesta HTTP correspondiente (código de estado y cuerpo en JSON).
- **Service:** contiene la lógica de negocio propiamente dicha –por ejemplo, comprobar que un código de acceso es válido y no ha sido ya utilizado antes de vincular a un usuario con una cofradía, o comprobar que quien solicita cancelar una procesión pertenece a la Junta de Cofradías de esa ciudad–. Es en esta capa donde se aplican las comprobaciones de autorización por rol (RNF-05) que antes resolvían las reglas de seguridad de Firestore.
- **Repository:** interfaces de Spring Data JPA, una por entidad, que abstraen el acceso a PostgreSQL y de las que Spring genera automáticamente la implementación.
- **Security:** configuración de Spring Security, el filtro que intercepta cada petición para validar el token JWT, y la generación de tokens en el proceso de autenticación.
- **DTO (Data Transfer Object):** clases que definen qué datos entran y salen exactamente de cada *endpoint*, evitando exponer directamente las entidades de persistencia y desacoplando el modelo interno de la base de datos del contrato público de la API.

Todas las entidades del dominio –Ciudad, Cofradia, Paso, Evento, Procesion, Recorrido, PuntoRuta, Usuario y Notificacion– siguen esta misma estructura de cuatro capas, tal y como recoge la Figura 4.2. El esquema completo de la base de datos subyacente se detalla en el Apéndice C, y el razonamiento seguido para diseñarla se explica en el apartado Diseño de la base de datos más adelante en esta misma sección.



Figura 4.2: Diagrama de paquetes del backend propio

Diagrama de despliegue Un diagrama de despliegue representa, de forma física, dónde se ejecuta cada parte del sistema y cómo se comunican entre sí esos entornos. La Figura 4.3 recoge el despliegue completo de este proyecto: el dispositivo móvil ejecuta la aplicación React Native, que se comunica mediante HTTPS con cuatro destinos independientes: la API propia (Spring Boot, desplegada en Railway [35]), que a su vez es el único componente que accede a la base de datos PostgreSQL (también alojada en Railway) y quien, mediante el Firebase Admin SDK [46], dispara el envío de notificaciones *push*; Firebase Storage, para la lectura y subida de imágenes; Firebase Cloud Messaging, para el registro del dispositivo y la recepción de notificaciones; y Google Maps Platform, para la visualización de mapas y el cálculo de rutas.

A diferencia del planteamiento inicial, aquí sí existe un nodo de servidor propio, lo que también cambia cómo se cubre el requisito RNF-03 (disponibilidad de la información estática sin conexión a internet): al no existir ya una caché local automática como la que ofrecía Cloud Firestore, esta funcionalidad se resuelve en el cliente mediante TanStack Query [45], que persiste en el dispositivo la última respuesta recibida de cada *endpoint* y la sirve como datos disponibles mientras no hay conexión, revalidándola en segundo plano en cuanto la conexión se recupera.

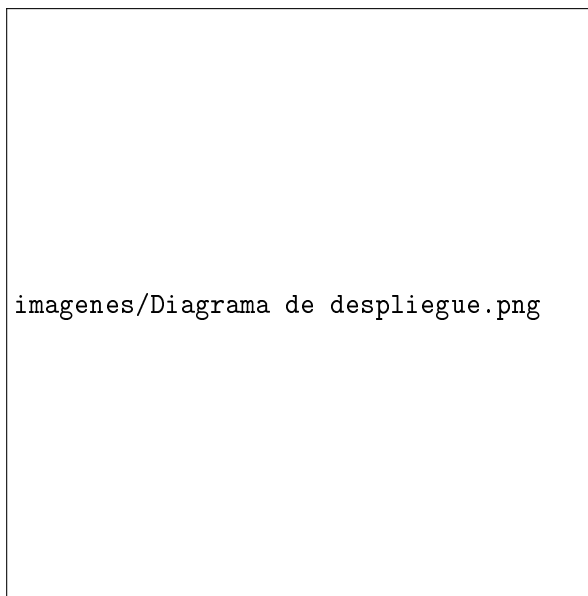


Figura 4.3: Diagrama de despliegue de la aplicación

Diseño de la base de datos

A diferencia de una base de datos documental como Cloud Firestore, PostgreSQL [40] es una base de datos **relacional**: la información se organiza en *tablas* con filas y columnas, y las relaciones entre entidades se expresan mediante **claves foráneas**⁸, que el propio motor de base de datos se encarga de validar. El modelo de dominio de la Figura 3.5 se ha traducido a este modelo relacional siguiendo estas decisiones:

- `pasos` es una tabla independiente con una clave foránea a su tabla “padre” (`cofradia_id`), en lugar de una subcolección anidada como en el planteamiento inicial con Firestore. Al tratarse de un modelo relacional, no es necesario un mecanismo especial para expresar que “siempre se consulta junto a su padre”: basta con una consulta que una (*join*) ambas tablas por esa clave foránea.
- `puntos_ruta` no referencia directamente a un recorrido: durante la construcción del backend se detectó que un mismo punto (la misma iglesia, la misma plaza) puede formar parte de varios recorridos –varias procesiones pueden compartir tramo–, así que la relación con recorridos pasó de 1:N a N:M, resuelta con la tabla intermedia `recorrido_puntos_ruta`, que además guarda el orden y la hora prevista de paso *de esa relación concreta* y no del punto en sí (Apéndice C).
- `ubicaciones` es una tabla nueva, sin equivalente directo en el modelo de dominio original: extrae a una tabla propia (latitud, longitud, dirección) el punto geográfico que necesitan tanto `eventos` como `puntos_ruta`, en lugar de repetir esas tres columnas en cada una. Al ser una tabla independiente, una misma ubicación –por ejemplo, la iglesia donde se celebra un evento y donde además hay un punto de interés del recorrido– puede reutilizarse desde varias entidades sin duplicar datos.

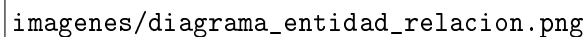
⁸Clave foránea (*foreign key*): columna de una tabla que referencia la clave primaria de otra tabla, y que la propia base de datos utiliza para garantizar que esa relación siempre apunta a un registro que realmente existe (integridad referencial).

- `posiciones_actuales` se modela como una tabla independiente, en lugar de como columnas dentro de la propia tabla `procesiones` –el motivo es el mismo que en el planteamiento inicial: estas escrituras frecuentes de posición (cada 30 segundos) no deben afectar a los datos más estables de la procesión (horario, recorrido, pasos), que rara vez cambian–, pero a diferencia del planteamiento inicial ya no es una única fila por procesión que se sobrescribe con cada actualización (UPSERT), sino un **histórico de lecturas GPS sueltas** (*pings*), cada una **anónima** –sin ninguna referencia a la persona que la envió, ver el punto sobre usuarios más abajo–. La “posición actual” de una procesión deja de ser un dato guardado como tal y pasa a ser un valor **calculado** en el momento de la consulta: el promedio de los *pings* recibidos en los últimos 60 segundos. La aplicación cliente sigue consultando este valor mediante la API propia (GET /procesiones/{id}/ubicacion) con la cadencia que exige el requisito RNF-08, tal y como se explica en el apartado Decisiones tecnológicas.
- `favoritos` sigue sin guardarse en la base de datos, sino de forma local en el propio dispositivo (por ejemplo, con `AsyncStorage`), ya que el ciudadano no tiene cuenta ni sesión en la que persistir esta información en el servidor; únicamente se guarda el identificador de la cofradía o procesión marcada como favorita.
- `codigos_acceso` y `notificaciones_entregadas` son tablas con una clave foránea a la cofradía, procesión, usuario o notificación correspondiente. A diferencia del planteamiento inicial con `Firesore`, donde estas referencias debían gestionarse manualmente para evitar duplicar información y que se desactualizara, aquí es la propia base de datos –mediante las restricciones de clave foránea– quien garantiza que una referencia nunca pueda apuntar a un registro inexistente.
- La tabla `usuarios` solo contiene los perfiles con credenciales propias –junta de cofradías y administrador–: el ciudadano sigue sin registrarse, y **el cofrade dejó de generar ninguna fila** durante la construcción del backend. Validar un código de acceso (tabla `codigos_acceso`, con estados `EMITIDO`, `VALIDADO` y `REVOCADO`, tal y como recoge el caso de uso “Acceso como cofrade”, Sección 3.4.1) ya no crea ninguna fila en `usuarios`: el servidor comprueba que el código no está revocado y devuelve directamente un token JWT identificado con la **cofradía** cuyo código se validó, no con una persona –el propio código de acceso actúa como credencial, sin que en ningún momento se le pida al cofrade establecer una contraseña–. Esta decisión evita cargar el sistema con el registro de personas cuya participación es, además, estacional. `JuntaCofradias` y `Administrador` sí tienen credenciales propias (email y una contraseña almacenada como *hash*⁹ mediante `BCrypt`, el algoritmo utilizado por defecto por `Spring Security` [43]), y se comprueba su rol en la capa de servicio de la API (*Service*, Sección 4.1) antes de autorizar cualquier operación protegida, cubriendo así el requisito RNF-05. Cada código de acceso lleva una **referencia a la cofradía** en cuyo nombre lo emite la Junta de esa ciudad, de forma que un cofrade que lo valida puede compartir su ubicación en tiempo real durante las procesiones en las que participa esa cofradía, que es la única acción que el modelo de dominio le permite realizar directamente sobre el sistema (Figura 3.5).
- Tanto `usuarios` como `eventos` son, además, la raíz de una **herencia real de tablas** (estrategia `JOINED` de JPA), no solo una traducción directa del modelo de dominio: `procesiones` extiende a `eventos` y `puntos_de_interes` extiende a `puntos_ruta` de la misma forma en que `miembros_junta_cofradia`

⁹Hash: transformación irreversible de una contraseña en una cadena de caracteres, de forma que el sistema pueda comprobar que una contraseña es correcta sin necesidad de guardarla en texto plano.

y administradores extienden a usuarios –cada subtipo en su propia tabla, cuyo id es a la vez su clave primaria y una clave foránea a la tabla raíz–, en lugar de una única tabla con muchas columnas que quedan nulas según el caso, como habría ocurrido de traducir la herencia del modelo de dominio de la forma más directa posible.

- Las relaciones N:M del modelo de dominio –pasos que desfilan en varias procesiones, cofradías que participan en varios eventos, y notificaciones que se entregan a varios usuarios– se resuelven mediante **tablas intermedias** (`pasos_procesiones`, `eventos_cofradias` y `notificaciones_entregadas`, esta última con los atributos adicionales `leida` y `fecha_lectura`), la forma habitual de modelar relaciones muchos-a-muchos en un esquema relacional.



imagenes/diagrama_entidad_relacion.png

Figura 4.4: Diagrama entidad-relación de la base de datos

El Apéndice C recoge, a nivel de campo, el esquema completo de cada tabla –tipos de dato, claves primarias y foráneas incluidas– para quien necesite el detalle completo de la base de datos.

Interfaces de usuario

Paralelamente a las decisiones de arquitectura y de base de datos, se ha diseñado en Figma [47] el conjunto de pantallas del perfil de ciudadano mediante *mockups*¹⁰, que sirven de referencia visual para la implementación posterior. Se ha optado por diseñar de forma conjunta todas las pantallas del perfil ciudadano, aunque no todas se implementen en esta primera iteración, para garantizar una línea visual coherente desde el principio y evitar rediseños parciales más adelante. De ellas, en esta primera iteración se implementan únicamente la selección de ciudad, la pantalla de inicio y los listados y fichas de detalle de cofradías, procesiones, pasos y eventos; el resto se retoman en la iteración a la que corresponden: el mapa en vivo en la Iteración 2, el modo cofrade y la ubicación compartida del perfil en la Iteración 3, y el calendario, la búsqueda y las preferencias de notificaciones del perfil en iteraciones posteriores.

Todas las pantallas comparten una estética oscura con acentos dorados, que evoca la iluminación de cirios y el ambiente nocturno característico de las procesiones, y una barra de navegación inferior fija con cinco secciones (*Inicio*, *Calendario*, *Mapa*, *Buscar* y *Perfil*), que se corresponde directamente con el paquete *navigation/* descrito en el apartado de arquitectura. Los listados de cofradías, procesiones, pasos y eventos ya no ocupan una sección propia de la barra de navegación, sino que se accede a ellos desde un menú contextual en la pantalla de *Inicio*, tal y como se explica más adelante.

La paleta de color se apoya en tonos marrón muy oscuro, casi negro, para el fondo (#1E1008, #120A06, #34150E, #472F17), dorados como color de acento para títulos, iconos activos y elementos interactivos (#C9A84C, #9A7D60), tonos crema para tarjetas y textos secundarios (#F0E0C8, #E3D595) y blanco (#FFFFFF) para el texto principal sobre fondo oscuro; un verde puntual (#37E37E) se reserva para indicadores de estado, como una procesión en curso. En cuanto a la tipografía, se combinan tres familias: **Cormorant Garamond**, una serifa elegante, para los títulos, coherente con la estética evocada; **Lora**, también serifa pero más neutra, para el texto de lectura extensa (historia de una cofradía, descripción de un paso); y **Sora**, de palo seco, para los elementos de interfaz (botones, etiquetas, barra de navegación), donde interesa más la legibilidad a tamaños pequeños que el carácter estético. Estas decisiones se apoyan, además del criterio propio, en las pautas de *Material Design 3* [48], la guía de diseño de interfaz de Google, en particular en lo relativo al contraste de color, el tamaño mínimo táctil de los elementos interactivos y la estructura de la barra de navegación inferior.

Selección de ciudad Es la primera pantalla que ve cualquier usuario, ya que toda la información de la aplicación (cofradías, procesiones, eventos) está filtrada por la ciudad seleccionada, tal y como se refleja en la colección *ciudades* descrita en el apartado anterior. El usuario puede buscar su ciudad o elegirla de una lista con el número de procesiones disponibles en cada una.

¹⁰Mockup: representación visual estática de la interfaz de una aplicación, sin funcionalidad real, utilizada para planificar el diseño antes de su implementación.



Figura 4.5: Mockup de la pantalla de selección de ciudad

Inicio Muestra la información general del día de Semana Santa seleccionado: un aviso destacado si existe alguna incidencia relevante (por ejemplo, un retraso en el horario de salida), la procesión que está en curso en ese momento si la hay, unas estadísticas generales de la ciudad (número de cofradías, procesiones y nazarenos) y el listado de procesiones y eventos programados para el día, marcando cuáles son favoritos del ciudadano. Desde el menú contextual de esta pantalla (icono *overflow* superior derecho) se accede, además, a los listados independientes de cofradías, procesiones, pasos y eventos.

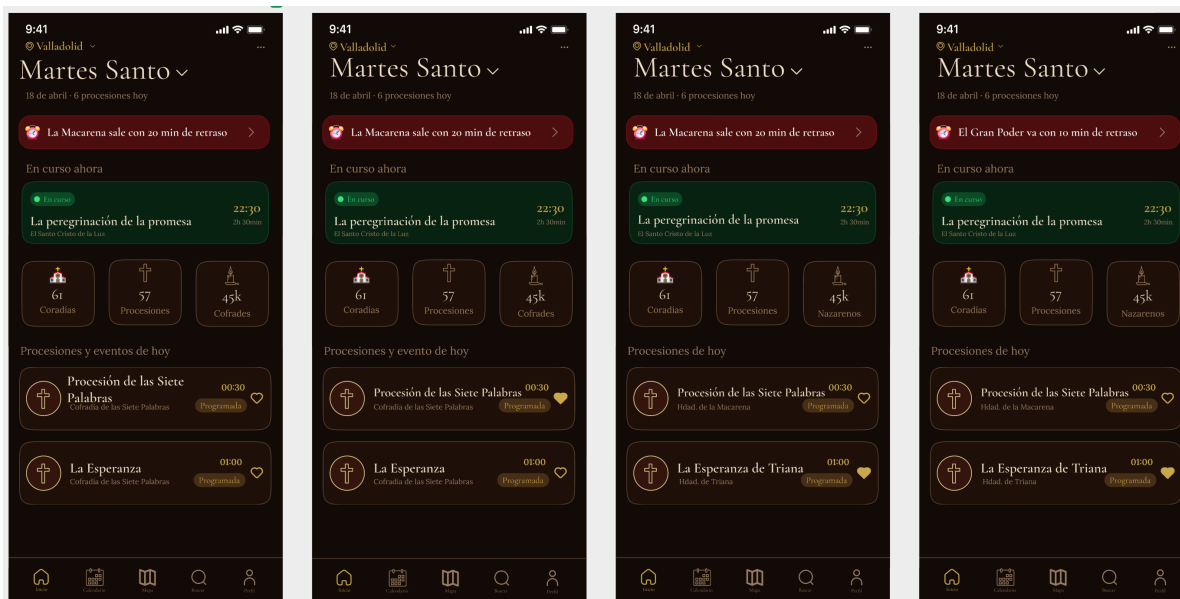


Figura 4.6: Mockup de la pantalla de inicio: aviso de retraso, procesión en curso, estadísticas y agenda del día

Calendario Complementa a la pantalla de inicio permitiendo consultar cualquier otro día de la Semana Santa, no solo el actual. Ofrece dos niveles de detalle: una vista mensual completa, con un indicador en los días que tienen procesiones programadas, y una vista semanal con la agenda del día seleccionado, listando sus procesiones con horario y duración.



Figura 4.7: Mockup de la pantalla de calendario: vista mensual y agenda del día seleccionado

Listados de cofradías, procesiones, pasos y eventos Cada uno de estos cuatro elementos del dominio cuenta con su propio listado, accesible desde el menú de la pantalla de Inicio, con buscador propio y sin más información que el nombre de cada elemento, ya que su detalle se consulta accediendo a la ficha correspondiente.

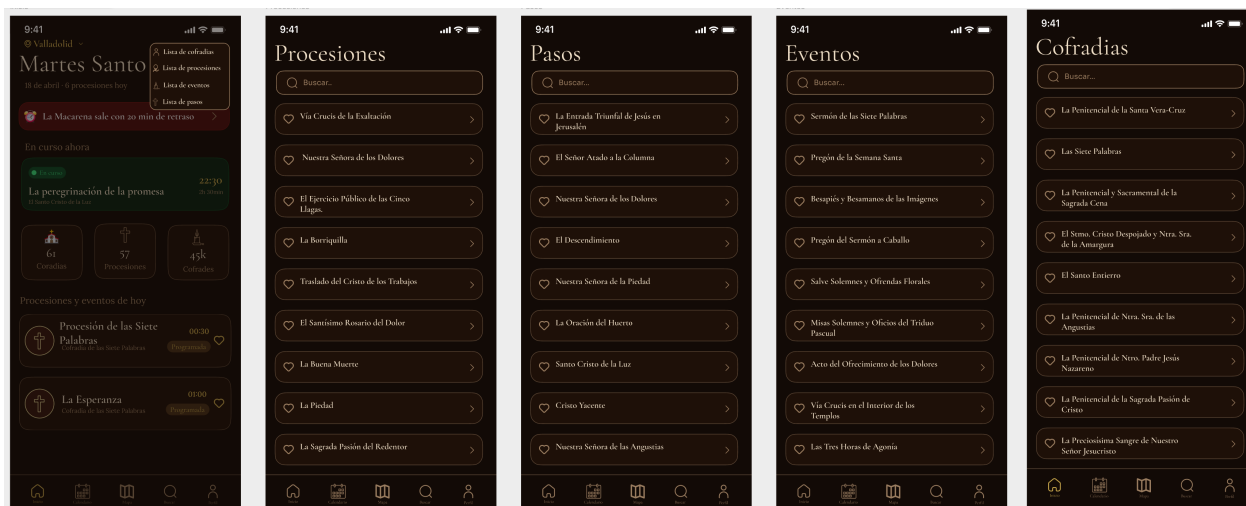


Figura 4.8: Mockup del menú de la pantalla de inicio y de los listados de procesiones, pasos, eventos y cofradías

Detalle de cofradía y de paso La ficha de una cofradía muestra su historia, su web oficial, las procesiones y el evento que organiza y los pasos que la componen. De forma análoga, la ficha de un paso muestra una imagen, su historia y origen, un análisis artístico y su web oficial, siguiendo la misma estructura visual que la ficha de cofradía para mantener la coherencia del conjunto.

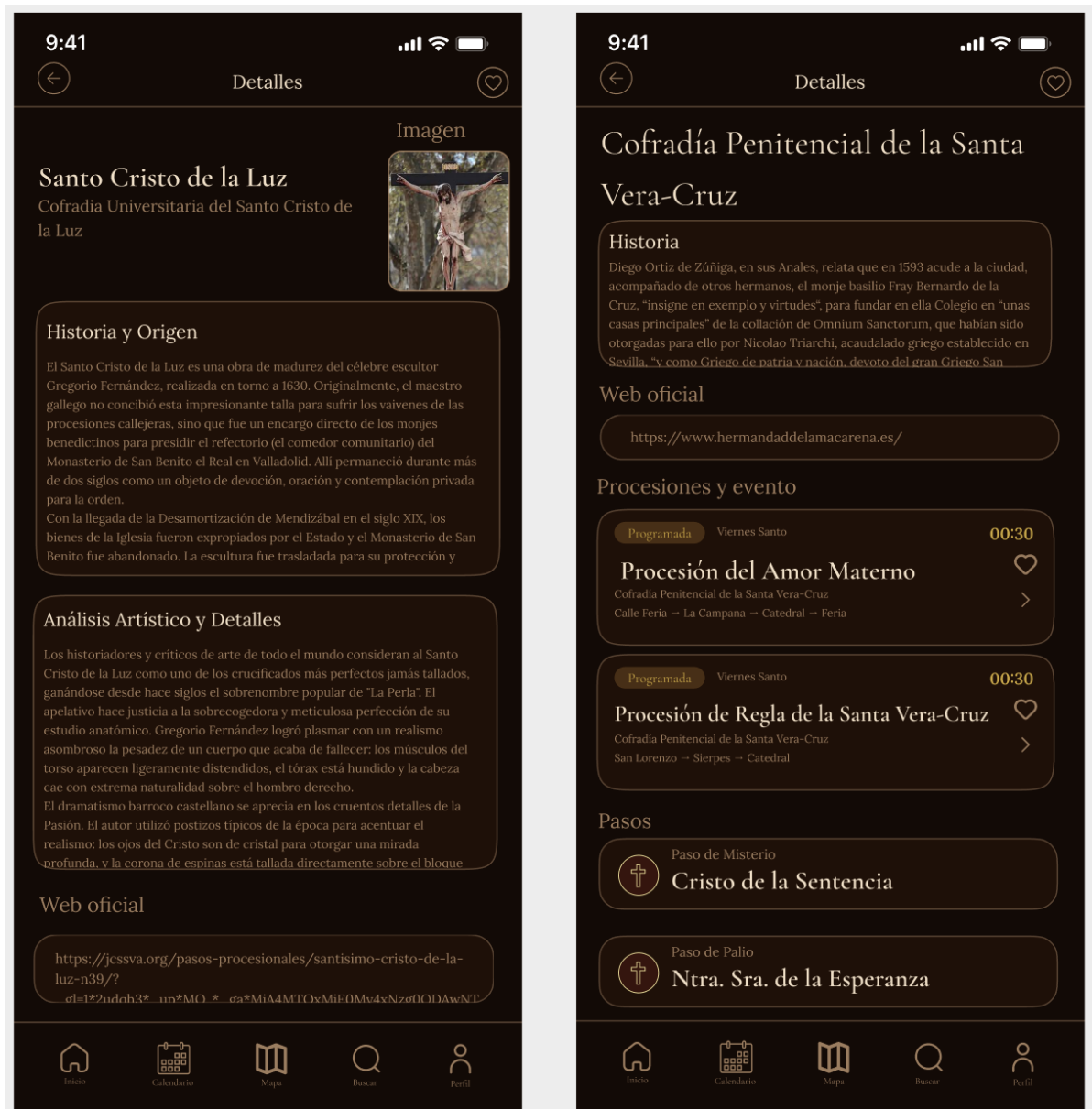


Figura 4.9: Mockup del detalle de un paso y de una cofradía

Detalle de procesión Muestra el estado de la procesión (programada, en curso o finalizada), su día y hora de salida, duración, número de nazarenos, los pasos que participan en ella y su recorrido sobre un mapa, con un botón de acceso directo al mapa en vivo si la procesión está en curso. Además, de forma similar a la ficha de una cofradía, cada procesión tiene una pantalla de información ampliada con la historia y el origen del desfile.

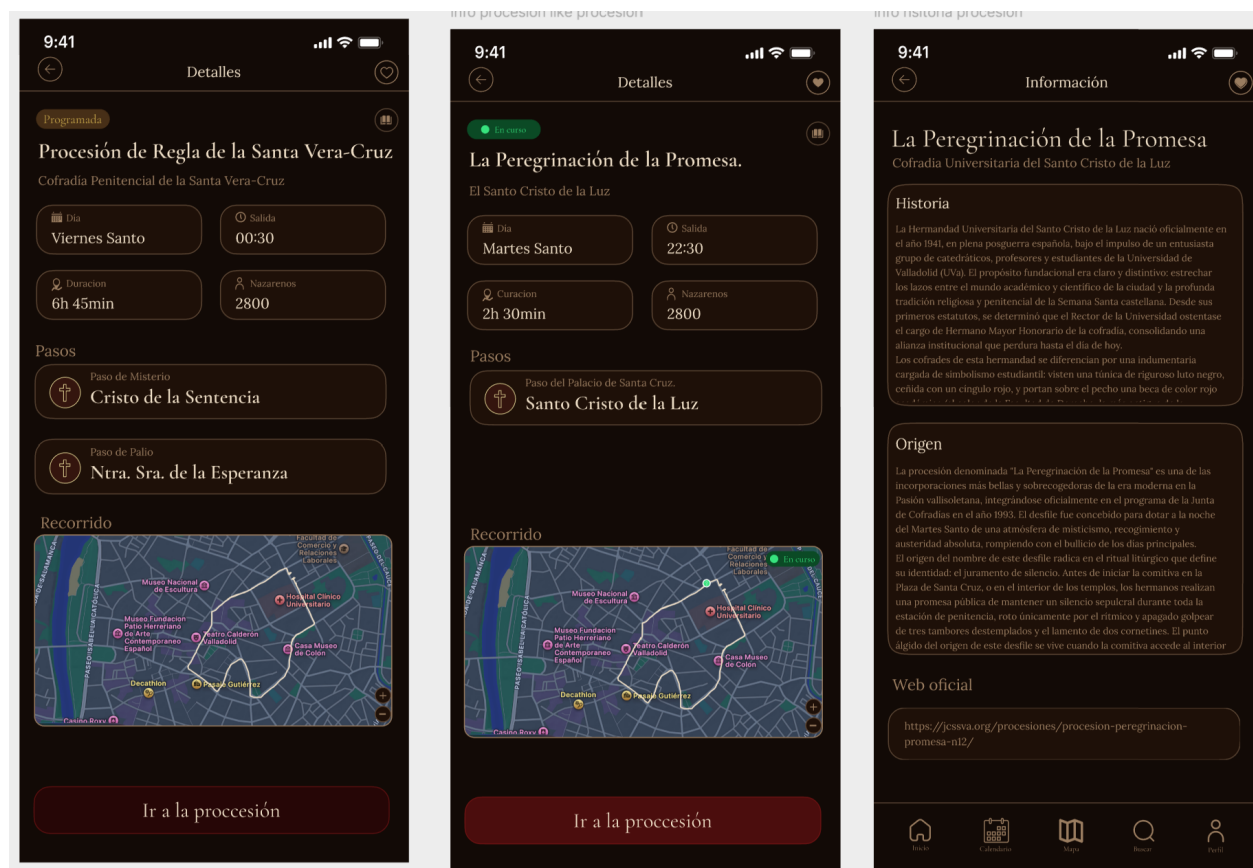


Figura 4.10: Mockup del detalle de una procesión (programada y en curso) y de su información ampliada

Buscar Permite localizar procesiones, pasos y eventos combinando texto libre con filtros por día de Semana Santa y por estado (programada, en curso o finalizada), mostrando un mensaje cuando ningún resultado cumple los filtros seleccionados. Su funcionalidad se implementa en una iteración posterior.

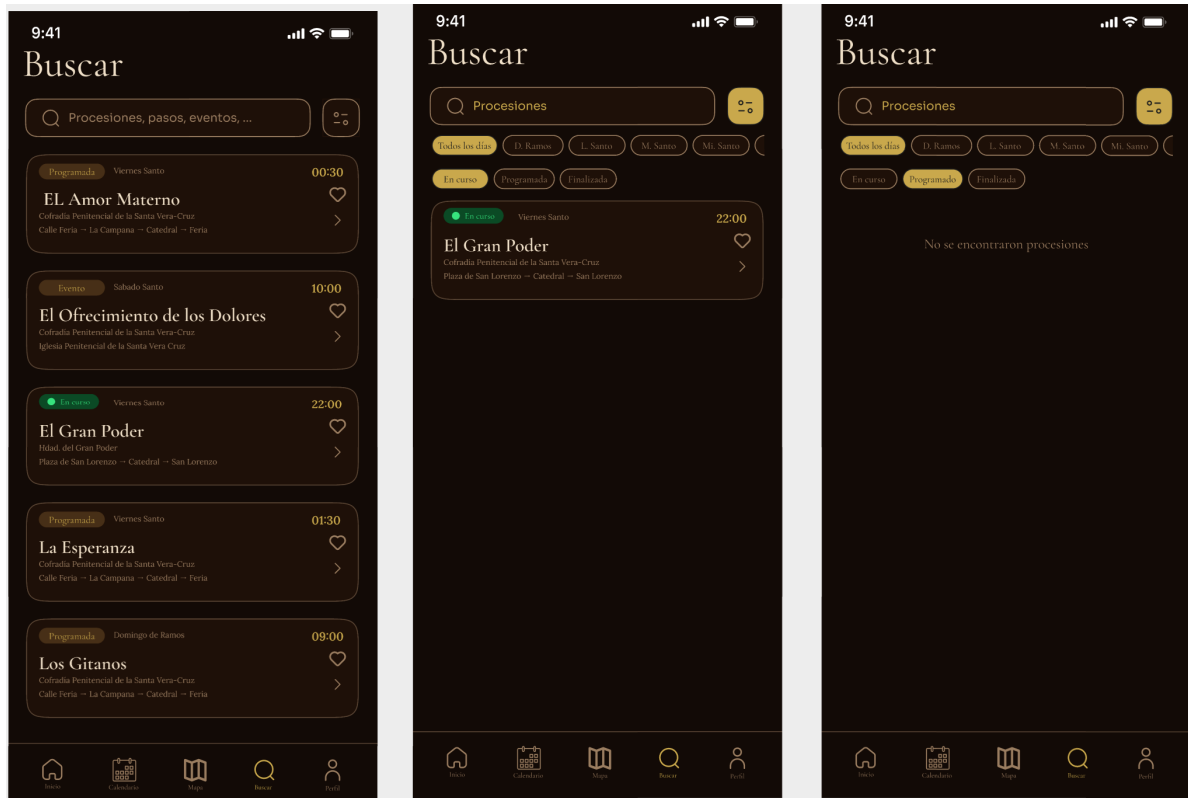


Figura 4.11: Mockup de la pantalla de búsqueda con filtros por día y estado

Mapa en Vivo Mostrará sobre un mapa de Google Maps la posición en tiempo real de las procesiones en curso y la ubicación del propio ciudadano, junto con un listado de las procesiones actualmente en movimiento desde el que centrar el mapa sobre una de ellas. Se implementa en la Iteración 2.



Figura 4.12: Mockup del mapa en vivo

Perfil Reúne la ciudad seleccionada y las cofradías/procesiones favoritas del ciudadano, el cambio entre el modo ciudadano y el modo cofrade, y las preferencias de notificaciones (avisos de procesiones en curso, alertas de tráfico y retrasos). En modo cofrade se añade además el botón para activar la ubicación compartida durante una procesión. El cambio de modo y la ubicación compartida se implementan en la Iteración 3; las preferencias de notificaciones, en una iteración posterior.



Figura 4.13: Mockup de la pantalla de perfil, en modo ciudadano y en modo cofrade

Patrones de diseño

Un patrón de diseño [15] es una solución ya conocida y reutilizable a un problema que se repite con frecuencia en el desarrollo de software; su uso facilita la legibilidad, el mantenimiento y la escalabilidad del código. Para seleccionar los patrones aplicables a este proyecto se han revisado dos TFG previos de la escuela con una arquitectura similar (aplicación móvil apoyada en Firebase), de los que se han descartado los patrones *Builder* y *Abstract Factory*: en Java se justifican porque los constructores de clases no admiten bien parámetros opcionales, pero en JavaScript/TypeScript un objeto literal con esos mismos campos opcionales ya resuelve el mismo problema de forma nativa, y en ningún punto del sistema es necesario intercambiar familias completas de implementaciones. Los siete patrones finalmente aplicados son los siguientes:

- **Singleton:** patrón que garantiza que una clase tenga una única instancia en toda la aplicación y un punto de acceso global a ella, evitando crear varias copias innecesarias. En este proyecto, tanto el cliente HTTP de la API propia (`api/client`, configurado una única vez con el interceptor que añade el token JWT a cada petición) como la inicialización de los SDK de Firebase que se mantienen (Storage y Cloud Messaging) se instancian una sola vez y esa misma instancia se reutiliza en el resto de la aplicación.
- **Repository:** patrón que separa la lógica de acceso a los datos del resto de la aplicación, ocultando los detalles de cómo y dónde se almacenan. En el cliente, la capa `services` es la única parte del código que conoce el cliente HTTP de la API propia; el resto de la aplicación solo invoca funciones como `procesionService.obtenerActivas(ciudadId)`, sin saber que por detrás se está llamando a un *endpoint* REST respaldado por PostgreSQL. En el backend, este mismo patrón aparece de forma directa en la capa `repository` (Sección 4.1): las interfaces de Spring Data JPA son, de hecho, una implementación literal del patrón Repository, no solo una analogía. Esta doble capa facilitaría cambiar de proveedor de base de datos en el futuro sin modificar ninguna pantalla ni ningún controlador.
- **Observer:** patrón en el que uno o varios elementos (“observadores”) se suscriben a los cambios de otro elemento y son notificados automáticamente cuando este cambia, en lugar de tener que preguntar repetidamente. En este proyecto se aplica a la recepción de notificaciones *push*: el cliente se registra como observador de Firebase Cloud Messaging (`onMessage`) y es notificado automáticamente cada vez que llega un aviso o una alerta, sin necesidad de preguntar al servidor si hay algo nuevo. No se aplica, en cambio, a la posición en vivo de una procesión (RNF-08), que se resuelve mediante consultas periódicas (*polling*) a la API cada 30 segundos (Sección 4.1): al ser el cliente quien pregunta activamente en lugar de ser notificado, este mecanismo es la operación inversa a Observer y no debe describirse como tal.
- **Factory Method:** patrón que centraliza en una única función la creación de un objeto, delegando en ella la decisión de qué variante concreta construir. El dominio distingue Aviso y Alerta como subtipos de `Notificacion` (Figura 3.5); una función fábrica construye el objeto correcto según los campos `tipoAlerta` y `fechaExpiracion` antes de enviarlo a la API, que se encarga de persistirlo en PostgreSQL (Apéndice C).
- **Context/Provider:** convención propia de React (no un patrón GoF¹¹ clásico) que permite exponer un dato o estado a cualquier pantalla de la aplicación sin tener que pasarlo manualmente de componente en componente. `AuthContext` y `LocationContext` exponen así el usuario autenticado (y su token JWT) y los permisos de ubicación a cualquier pantalla que los necesite.
- **Facade:** patrón que ofrece una única función sencilla como punto de entrada a una operación que, por detrás, involucra varios pasos o subsistemas distintos. El caso de uso “Compartir ubicación durante procesión” (RF-15) esconde tras la llamada `geoService.iniciarCompartirUbicacion(procesionId)` la solicitud de permiso de GPS, el envío periódico (POST) de la posición a la API y la gestión de una posible pérdida de conexión.

¹¹GoF (*Gang of Four*): apodo con el que se conoce a los cuatro autores del libro *Design Patterns* [49] (1994), que popularizó el catálogo clásico de patrones de diseño orientado a objetos (Singleton, Observer, Factory Method, entre otros).

- **Adapter:** patrón que convierte el formato de datos de un sistema externo al formato interno que espera la aplicación, permitiendo que ambos “encajen” sin modificar ninguno de los dos. Las notificaciones *push* de Firebase Cloud Messaging llegan con un formato plano (título, cuerpo, y un mapa de texto), distinto del modelo interno tipado *Alerta/Aviso*; una función adaptadora convierte ese *payload* recibido al modelo de dominio antes de utilizarlo en la aplicación.

4.1.3. Implementación

Se ha implementado el backend propio descrito en el apartado de diseño anterior: un proyecto Spring Boot [38] generado con Spring Initializr, organizado en los paquetes `model`, `repository`, `service`, `controller`, `dto`, `security` y `exception` que siguen la estructura de capas ya justificada, y con el esquema de PostgreSQL gestionado mediante migraciones versionadas de Flyway [?] (un script SQL numerado por cada cambio de esquema), en lugar de dejar que Hibernate genere o modifique las tablas automáticamente; así el esquema queda documentado paso a paso y se aplica igual en local, en Railway y en cualquier entorno nuevo, tal y como recoge el manual de despliegue (Apéndice D).

Sobre esa base se han implementado las cuatro capas para todas las entidades necesarias para el módulo de ciudadano de esta primera iteración –Ciudad, JuntaCofradías, Cofradía, Paso y Evento–, cada una con su correspondiente conjunto de *endpoints* REST de consulta y gestión (GET, POST, PUT, DELETE), documentados automáticamente en `/swagger-ui.html` mediante `springdoc-openapi` [50]. JuntaCofradías se implementa en esta misma iteración por ser, igual que Ciudad, un dato base del que depende Cofradía (Apéndice C), aunque su gestión efectiva –dar de alta, editar o dar de baja una Junta– corresponda al panel de administrador de la Iteración 5.

Durante esta implementación se han detectado varias necesidades no recogidas en el diseño inicial de la base de datos, que se han ido resolviendo sobre la marcha y quedan documentadas con su justificación en el Apéndice C: entre ellas, que la información de una cofradía y de un paso necesitaba más campos de los previstos inicialmente (web en lugar de un archivo adjunto, `analisis_artistico` para el paso), o que Cofradía debía depender de Ciudad y no de JuntaCofradías, para que la información de las cofradías de una ciudad no se viera afectada por una renovación de su Junta.

Queda pendiente para el cierre de esta iteración la implementación de las pantallas de cliente correspondientes sobre esta API, que hasta ahora consumían datos de prueba (*mock*).

4.1.4. Pruebas

En esta fase del proyecto la verificación del backend ha sido manual: compilación exitosa tras cada cambio (`mvn compile`) y comprobación funcional de los *endpoints* mediante la documentación interactiva de `springdoc-openapi` (`/swagger-ui.html`) contra una instancia de PostgreSQL local levantada con Docker. Queda pendiente una batería de pruebas automatizadas (pruebas unitarias de la capa `service` y pruebas de integración de los controladores con `Spring Boot Test`), que se abordará más adelante y se documentará en el Capítulo 6.

4.2. 2ª Iteración

4.2.1. Análisis

4.2.2. Diseño

4.2.3. Implementación

Esta segunda iteración implementa en el backend las entidades necesarias para el sistema de mapas: Recorrido, Ubicacion y PuntoRuta, siguiendo la misma estructura de cuatro capas ya establecida en la Iteración 1 y documentada a nivel de campo en el Apéndice C.

Ubicacion es una tabla nueva, sin equivalente directo en el modelo de dominio original (Figura 3.5): agrupa en un único lugar la latitud, la longitud y la dirección que necesitan tanto Evento como PuntoRuta, para poder reutilizar el mismo punto geográfico desde varias entidades sin duplicar datos –por ejemplo, la sede de una cofradía y un punto de interés del recorrido que coincidan en el mismo templo–.

PuntoRuta es, además, la raíz de una jerarquía de herencia real de tablas (estrategia JOINED de JPA): un punto “de paso” simple se guarda directamente en puntos_ruta, mientras que un punto “de interés”, con nombre, descripción, imagen y una categoría, extiende esa tabla en PuntoDeInteres. Durante esta implementación se detectó, además, que un mismo punto de ruta puede formar parte de **varios** recorridos –varias procesiones pueden compartir un mismo tramo de calle–, así que la relación entre Recorrido y PuntoRuta se resolvió como una relación N:M mediante la tabla intermedia recorrido_puntos_ruta, que además guarda el orden del punto y su hora prevista de paso *en ese recorrido concreto*, ya que un mismo punto puede ocupar posiciones y horas distintas según el recorrido del que forme parte.

4.2.4. Pruebas

Al igual que en la Iteración 1, la verificación de estas entidades se ha realizado de forma manual mediante la documentación interactiva de springdoc-openapi (/swagger-ui.html). La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

4.3. 3ª Iteración

4.3.1. Análisis

4.3.2. Diseño

4.3.3. Implementación

Esta tercera iteración implementa la geolocalización en tiempo real y el perfil de cofrade, lo que exige dos piezas que hasta ahora no hacían falta: la entidad *Procesion* propiamente dicha y, por primera vez en el proyecto, un mecanismo de autenticación –hasta este punto todos los *endpoints* eran de lectura o gestión pública, ya que el ciudadano nunca se registra (RI-01)–.

Procesion se implementa como una extensión real de *Evento* (herencia con estrategia JOINED de JPA, Apéndice C) en lugar de una simple referencia: hereda nombre, historia, fecha, estado, ubicación y cofradías participantes de *Evento*, y añade únicamente su fecha de inicio y fin y su *Recorrido* asociado (relación 1:1, opcional –una procesión puede programarse antes de tener ruta definida–).

Para el acceso como cofrade (caso de uso “Acceso como cofrade”, Sección 3.4.1) se implementa *CodigoAcceso* junto con la infraestructura de autenticación: tokens **JWT** generados y validados con la biblioteca *jjwt*, un filtro de Spring Security que intercepta cada petición para comprobar el token, y el *endpoint* POST `/auth/codigo-acceso`, que valida el código y devuelve directamente un token identificado con la cofradía correspondiente, sin registrar ninguna fila de usuario (Apéndice C, Sección C.10). La clase *Cofrade* del modelo de dominio (Figura 3.5) se implementa así como una representación construida en memoria a partir de ese token en cada petición, no como una entidad persistida.

Por último, se implementa la funcionalidad de compartir ubicación (RF-15) mediante el *endpoint* POST `/procesiones/{id}/posiciones`, que registra cada actualización de posición del cofrade como una lectura –o *ping*– anónima en *PosicionActual*, y GET `/procesiones/{id}/ubicacion`, que calcula la posición agregada de la procesión en ese momento como el promedio de los *pings* recibidos en los últimos 60 segundos, cubriendo así el requisito RNF-08.

4.3.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual mediante *springdoc-openapi* (`/swagger-ui.html`), incluyendo la obtención de un token válido con POST `/auth/codigo-acceso` y su uso en las peticiones protegidas subsiguientes. La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

4.4. 4ª Iteración

4.4.1. Análisis

4.4.2. Diseño

4.4.3. Implementación

Esta cuarta iteración implementa el panel de la Junta de Cofradías: el acceso mediante credenciales propias y el mecanismo de control de acceso que restringe, entidad por entidad, qué puede gestionar cada Junta.

Se implementa `MiembroJuntaCofradia` –persona con credenciales que pertenece a la Junta de una ciudad, pudiendo haber varios miembros por Junta– y el `endpoint` `POST /auth/login`, que compara el email y la contraseña (*hash* BCrypt) recibidos y devuelve un token JWT con el identificador del miembro y su rol, siguiendo el mismo mecanismo de autenticación introducido en la Iteración 3.

Sobre esa autenticación se implementa la autorización por rol que exige el requisito RNF-05: en la capa `Service` de cada entidad de contenido –Cofradia, Paso, Evento, Procesion y `CodigoAcceso`–, las operaciones de creación, edición y eliminación comprueban que quien las solicita es un miembro de la Junta que gestiona la ciudad a la que pertenece esa cofradía, antes de permitir la operación; en caso contrario, la API responde con un código 403 (prohibido). Esta comprobación se centraliza en un único método reutilizable en lugar de repetirse en cada entidad, para evitar duplicar la misma lógica de autorización varias veces. Las entidades `Recorrido`, `Ubicacion`, `PuntoRuta` y `PuntoDeInteres`, al no pertenecer a una única cofradía –se diseñaron como recursos reutilizables entre procesiones e incluso entre ciudades distintas, Sección 4.2–, solo exigen que quien las gestione sea una Junta de Cofradías cualquiera, sin comparar ciudad.

4.4.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual: obtención de un token con `POST /auth/login` y comprobación de que las operaciones de gestión se aceptan o se rechazan (403) según la ciudad que gestiona cada Junta de prueba. La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

4.5. 5ª Iteración

4.5.1. Análisis

4.5.2. Diseño

4.5.3. Implementación

Esta quinta y última iteración implementa el panel de administrador global del sistema. El sistema de notificaciones previsto para esta misma iteración queda pendiente de implementar.

Se implementa Administrador –rol global del sistema, sin ninguna referencia a Ciudad, ya que cualquier administrador puede gestionar cualquier ciudad– y su acceso mediante el mismo *endpoint* POST /auth/login ya utilizado por la Junta de Cofradías en la Iteración 4, que distingue el rol a partir del tipo de cuenta encontrado.

Sobre esa autenticación se completa el mecanismo de control de acceso por rol: las operaciones de creación, edición y eliminación de Ciudad pasan a exigir que quien las solicita sea un Administrador, sin comparar ninguna ciudad concreta –a diferencia de la comprobación introducida en la Iteración 4 para la Junta de Cofradías, aquí no hay ninguna instancia con la que comparar–. De igual forma, dar de alta, editar o dar de baja una JuntaCofradias, y dar de alta o dar de baja a sus MiembroJuntaCofradia, quedan restringidos también al Administrador, que es quien gestiona qué personas tienen acceso al panel de cada Junta.

4.5.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual: obtención de un token de Administrador con POST /auth/login y comprobación de que las operaciones sobre Ciudad, JuntaCofradias y MiembroJuntaCofradia se aceptan únicamente con ese rol. La batería de pruebas automatizadas queda pendiente y se documentará en el Capítulo 6.

Capítulo 5

Estado final de la aplicación

5.0.1. Historias de usuario

ID	HU01
Nombre	Login de un usuario
Prioridad	Alta
Riesgo	Bajo
Descripción	Como usuario quiero poder acceder a la aplicación a través de una pantalla de login
Validación	- Quiero poder introducir mi nick y contraseña para acceder a mi perfil de usuario - Quiero que mi nick sea único

Tabla 5.1: Historia de usuario HU01

Capítulo 6

Pruebas

Capítulo 7

Conclusiones

7.1. Trabajo futuro

Como líneas de trabajo futuro se plantean las siguientes ampliaciones del sistema.

En primer lugar, el desarrollo de una página web para la aplicación, que permita a los usuarios acceder a determinadas funcionalidades sin necesidad de instalar la aplicación móvil.

En segundo lugar, se plantea la posibilidad de generalizar el sistema desarrollado para dar soporte no solo a la Semana Santa, sino a cualquier tipo de evento o celebración tradicional (por ejemplo, ferias, romerías o fiestas patronales). Para ello, entre otros cambios, las entidades específicas del dominio actual deberían sustituirse por clases más genéricas: las cofradías o hermandades pasarían a modelarse como *organizaciones*, sus miembros o cofrades como *miembros*, y las Juntas de Cofradías como *conjuntos de organizaciones* encargados de coordinar a varias organizaciones dentro de un mismo evento. De esta forma, el resto del sistema (geolocalización en tiempo real, gestión de recorridos, notificaciones, etc.) podría reutilizarse prácticamente sin cambios, ya que estas funcionalidades no dependen de la naturaleza religiosa del evento, sino de su estructura organizativa. Esta generalización estaría alineada con el objetivo de escalabilidad planteado inicialmente para el proyecto (Sección 1.4), y permitiría ampliar el alcance de la aplicación mucho más allá del ámbito de la Semana Santa.

Por último, sería interesante adaptar la interfaz en función de la Semana Santa de cada ciudad, asociando a cada una la terminología propia que se emplea localmente, de forma que la experiencia resulte más cercana y personalizada para cada usuario.

7.2. Puntos débiles

El sistema presenta una seguridad limitada en el acceso de los cofrades, que se realiza mediante el escaneo de un código QR. No obstante, se ha priorizado la usabilidad frente a la seguridad, ya que el nivel de riesgo asociado a este método de acceso se considera asumible dado el contexto de uso de la

aplicación.

Por otro lado, la introducción de información a través del dispositivo móvil resulta costosa para el usuario. Una alternativa más adecuada sería ofrecer una página web con capacidad de edición tanto para las juntas de cofradías como para los administradores. Sin embargo, por motivos de acotación del alcance del proyecto y de la carga de trabajo disponible, esta funcionalidad no se ha llegado a implementar.

Por otro lado, el uso de un backend propio con Spring Boot y PostgreSQL, en lugar de una plataforma *Backend-as-a-Service* como Firebase, traslada a la propia autora responsabilidades que antes asumía la plataforma de forma automática: el dimensionado y escalado de la infraestructura, las copias de seguridad de la base de datos, y la aplicación de parches de seguridad al propio servidor. Railway [35], la plataforma elegida para el despliegue (Apéndice D), simplifica buena parte de esta gestión, pero no la elimina por completo como sí ocurría con Firebase.

Asimismo, la posición en vivo de una procesión (RNF-08) se resuelve mediante una API REST convencional consultada periódicamente por el cliente, en lugar de mediante los *listeners* en tiempo real que ofrecía Cloud Firestore de forma nativa. Esta solución cumple el requisito tal y como está definido (una actualización cada 30 segundos como mínimo), pero no ofrece la misma sensación de instantaneidad que un mecanismo de *push*, y su eficiencia se degrada más rápido al aumentar el número de usuarios simultáneos que la consultan, ya que cada consulta es una petición HTTP independiente en lugar de una única notificación que el servidor reenvía a quien esté suscrito. Del mismo modo, la disponibilidad sin conexión de los datos estáticos (RNF-03), cubierta antes de forma automática por la caché local de Firestore, exige ahora una solución explícita en el cliente (Sección 4.1), con el consiguiente esfuerzo de implementación adicional.

Por último, el desarrollo de un backend propio con una tecnología (Spring Boot) con la que la autora contaba con una experiencia previa limitada –un único proyecto de prácticas durante el Grado– ha supuesto una curva de aprendizaje que no era necesaria con el planteamiento inicial basado en Firebase, y que se ha gestionado mediante una prueba de concepto acotada antes de comenzar la implementación completa, tal y como recoge el riesgo correspondiente (Sección 2.3).

Por último, al asociarse los códigos de acceso a nivel de cofradía y no de procesión concreta (Sección 4.1), un cofrade recibe notificaciones de todas las procesiones de su cofradía y no únicamente de aquellas en las que participa. Se considera una limitación menor, asumible dado que el número de procesiones por cofradía es reducido.

Bibliografía

- [1] Google, “Maps JavaScript API — documentación oficial.” <https://developers.google.com/maps/documentation/javascript>, 2024. Consultado el 1 de enero de 2025.
- [2] Evaluando Software, “La geolocalización funciona.” <https://www.evaluandosoftware.com/bpm/la-geolocalizacion-funciona/>, 2026. Consultado el 9 de mayo de 2026.
- [3] PlayCofrade, “El penitente — aplicación móvil.” <https://elpenitente.app/>, 2026. Consultado el 9 de mayo de 2026.
- [4] PlayCofrade, “El penitente — aplicación móvil (google play).” <https://play.google.com/store/apps/details?id=com.playcofrade.elpenitente&hl=es>, 2026. Consultado el 9 de mayo de 2026.
- [5] Valladolid Cofrade, “Lanzamos la nueva app de Valladolid Cofrade: toda la Semana Santa 2026 en tu móvil y con avisos en tiempo real.” <https://www.valladolidcofrade.com/noticias-2026/3660-lanzamos-la-nueva-app-de-valladolid-cofrade-toda-la-semana-santa-2026-en-tu-movil-y-con-avisos-en-tiempo-real>, 2026. Consultado el 9 de mayo de 2026.
- [6] Mozilla Developer Network, “¿qué es una PWA?.” https://developer.mozilla.org/es/docs/Web/Progressive_web_apps, 2026. Consultado el 9 de mayo de 2026.
- [7] Ayuntamiento de Zaragoza, “ssantazgz — aplicación móvil.” <https://www.zaragoza.es/sede/portal/turismo/post/app-semana-santa>, 2026. Consultado el 9 de mayo de 2026.
- [8] Junta Pro Semana Santa de Zamora, “Junta pro semana santa de zamora — aplicación móvil.” <https://semanasantadezamora.com/>, 2026. Consultado el 9 de mayo de 2026.
- [9] Uber Technologies, “Uber — página oficial.” <https://www.uber.com>, 2024. Consultado el 1 de enero de 2025.
- [10] M. Llorente Pérez, “Mejoras en la usabilidad de la aplicación móvil OrientaTree para orientación deportiva aplicada a la educación,” trabajo de fin de grado, Universidad de Valladolid, 2025. <https://uvadoc.uva.es/handle/10324/79500>.
- [11] I. Sommerville, *Software Engineering*. Boston, MA: Pearson, 10 ed., 2015. <https://dn790001.ca.archive.org/0/items/bme-vik-konyvek/Software%20Engineering%20-%20Ian%20Sommerville.pdf>.

-
- [12] ProjectManager, “What is a Gantt chart?.” <https://www.projectmanager.com/guides/gantt-chart>, 2026. Consultado el 9 de mayo de 2026.
- [13] IEEE, “IEEE Std 29148-2018: Systems and software engineering — requirements engineering,” Standard 29148-2018, Institute of Electrical and Electronics Engineers, 2018. <https://ieeexplore.ieee.org/document/8559686>.
- [14] Object Management Group, “UML — unified modeling language, especificación oficial.” <https://www.omg.org/spec/UML>, 2017. Versión 2.5.1. Consultado el 1 de enero de 2025.
- [15] C. Larman, *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Upper Saddle River, NJ: Prentice Hall, 3 ed., 2004. <https://www.pearson.com/en-us/subject-catalog/p/applying-uml-and-patterns/P200000003228>.
- [16] OWASP Foundation, “Access control cheat sheet.” https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html, 2024. Consultado el 1 de enero de 2025.
- [17] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Newtown Square, PA: Project Management Institute, 7 ed., 2021. <https://www.pmi.org/pmbok-guide-standards>.
- [18] B. W. Boehm, *Software Risk Management*. Los Alamitos, CA: IEEE Computer Society Press, 1989.
- [19] OpenStreetMap Foundation, “Openstreetmap — mapa libre del mundo.” <https://www.openstreetmap.org>, 2024. Consultado el 1 de mayo de 2025.
- [20] GitHub, Inc., “Github — plataforma de desarrollo de software.” <https://github.com>, 2024. Consultado el 1 de mayo de 2025.
- [21] Glassdoor, “Sueldo: Jefe de proyecto en españa.” https://www.glassdoor.es/Sueldos/jefe-de-proyecto-sueldo-SRCH_K00,16_IP2.htm, 2025. Consultado el 9 de julio de 2026.
- [22] Glassdoor, “Sueldo: Analista funcional en españa.” https://www.glassdoor.es/Sueldos/analista-funcional-sueldo-SRCH_K00,18.htm, 2025. Consultado el 9 de julio de 2026.
- [23] Glassdoor, “Sueldo: Ux designer en españa.” https://www.glassdoor.es/Sueldos/ux-designer-sueldo-SRCH_K00,11.htm, 2025. Consultado el 9 de julio de 2026.
- [24] Glassdoor, “Sueldo: Backend developer en españa.” https://www.glassdoor.es/Sueldos/backend-developer-sueldo-SRCH_K00,17.htm, 2026. Consultado el 9 de julio de 2026.
- [25] Glassdoor, “Salario de desarrollador móvil junior en españa.” <https://www.glassdoor.es>, 2025. Consultado el 1 de mayo de 2025.
- [26] Glassdoor, “Sueldos para el puesto de qa tester en españa.” https://www.glassdoor.es/Sueldos/qa-tester-sueldo-SRCH_K00,9.htm, 2025. Consultado el 9 de julio de 2026.
- [27] Google, “Android studio — entorno de desarrollo oficial para android.” <https://developer.android.com/studio>, 2024. Consultado el 1 de mayo de 2025.

- [28] Microsoft, “Visual studio code — editor de código fuente.” <https://code.visualstudio.com>, 2024. Consultado el 1 de mayo de 2025.
- [29] Google Firebase, “Firebase cloud messaging — documentación oficial.” <https://firebase.google.com/docs/cloud-messaging>, 2024. Consultado el 1 de enero de 2025.
- [30] Railway Corp, “Railway — planes y precios.” <https://railway.com/pricing>, 2026. Consultado el 7 de agosto de 2026.
- [31] Dribba, “¿cuánto cuesta subir una app a google play?.” <https://dribba.com/blog/cuanto-cuesta-subir-una-app-al-google-play>, 2026. Consultado el 9 de julio de 2026.
- [32] Figma, Inc., “Planes y precios de figma.” <https://www.figma.com/es-es/pricing/>, 2026. Consultado el 9 de julio de 2026.
- [33] Atlassian, “Planes de precios de jira software.” <https://www.atlassian.com/es/software/jira/pricing>, 2026. Consultado el 9 de julio de 2026.
- [34] Fotocasa, “Oficinas de alquiler en centro, valladolid capital.” <https://www.fotocasa.es/es/alquiler/oficinas/valladolid-capital/centro/1>, 2026. Consultado el 9 de julio de 2026.
- [35] Railway Corp, “Railway — plataforma de despliegue en la nube.” <https://railway.com/>, 2026. Consultado el 7 de agosto de 2026.
- [36] Meta Platforms, “React native — documentación oficial.” <https://reactnative.dev/docs/getting-started>, 2024. Consultado el 15 de julio de 2026.
- [37] Expo, “Expo — documentación oficial.” <https://docs.expo.dev/>, 2024. Consultado el 7 de agosto de 2026.
- [38] VMware, “Spring boot — documentación oficial.” <https://spring.io/projects/spring-boot>, 2024. Consultado el 7 de agosto de 2026.
- [39] Google Firebase, “Cloud storage for firebase — documentación oficial.” <https://firebase.google.com/docs/storage>, 2024. Consultado el 7 de agosto de 2026.
- [40] PostgreSQL Global Development Group, “PostgreSQL — documentación oficial.” <https://www.postgresql.org/docs/>, 2024. Consultado el 7 de agosto de 2026.
- [41] VMware, “Spring data JPA — documentación oficial.” <https://spring.io/projects/spring-data-jpa>, 2024. Consultado el 7 de agosto de 2026.
- [42] Red Hat, “Hibernate ORM — documentación oficial.” <https://hibernate.org/orm/documentation/>, 2024. Consultado el 7 de agosto de 2026.
- [43] VMware, “Spring security — documentación oficial.” <https://spring.io/projects/spring-security>, 2024. Consultado el 7 de agosto de 2026.
- [44] IETF, “RFC 7519: JSON web token (JWT).” <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. Consultado el 7 de agosto de 2026.

- [45] TanStack, “TanStack query — documentación oficial.” <https://tanstack.com/query/latest>, 2024. Consultado el 7 de agosto de 2026.
- [46] Google Firebase, “Firebase admin SDK — enviar mensajes desde un servidor.” <https://firebase.google.com/docs/cloud-messaging/send-message>, 2024. Consultado el 7 de agosto de 2026.
- [47] Figma, Inc., “Figma — herramienta de diseño de interfaces colaborativa.” <https://www.figma.com>, 2024. Consultado el 1 de mayo de 2025.
- [48] Google, “Material design 3.” <https://m3.material.io/>, 2024. Consultado el 27 de julio de 2026.
- [49] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
- [50] springdoc, “springdoc-openapi — documentación oficial.” <https://springdoc.org/>, 2024. Consultado el 7 de agosto de 2026.

Anexos

Apéndice A

Acrónimos

- **API:** Application Programming Interface.
- **PWA:** Progressive Web App (ya lo explicas en el texto, pero debería estar en el apéndice)
- **GPS:** Global Positioning System
- **UML:** Unified Modeling Language
- **RF:** Requisito Funcional
- **RI:** Requisito de Información
- **RNF:** Requisito No Funcional
- **HU:** Historia de Usuario
- **TFG:** Trabajo de Fin de Grado

Apéndice B

Casos de uso adicionales

Este apéndice recoge los casos de uso adicionales del sistema que, por no considerarse esenciales para la comprensión general del proyecto, no se han incluido en el cuerpo principal de la memoria (Sección 3.3). Cada uno de ellos sigue la misma estructura que los presentados en dicha sección: resumen, actor, precondition, postcondition, secuencia base, secuencia alternativa y requisitos relacionados.

Caso de uso: Consultar información de una cofradía, procesión, evento o paso	
Resumen	El usuario consulta la información detallada de una cofradía, procesión, evento o paso de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad o el usuario ha autorizado que la aplicación pueda consultar su ubicación actual.
Postcondición	Se muestra la lista de cofradías con su información.
Secuencia Base	
[1] El usuario accede al listado de cofradías, procesiones, eventos o pasos (RF-37).	
[2] El sistema muestra la lista correspondiente de la ciudad seleccionada.	
[3] El usuario selecciona un elemento de la lista.	
[4] El sistema muestra la información detallada del elemento seleccionado.	
Secuencia Alternativa	
[2.a.1] Sin elementos disponibles: El sistema muestra un mensaje indicando que no hay elementos registrados de ese tipo en esa ciudad.	
Requisitos Relacionados	RF-03, RF-04, RF-05 y RF-17.

Tabla B.1: Caso de uso: Consultar información de una cofradía, procesión, evento o paso

Caso de uso: Acceso como cofrade	
Resumen	Un usuario se registra en la aplicación como miembro de una cofradía.
Actor	Cofrade
Precondición	El usuario no está registrado.
Postcondición	El usuario queda registrado y puede acceder a funciones de cofrade.
Secuencia Base	
[1] El usuario selecciona la opción de registro.	
[2] El sistema muestra el formulario de registro (Introducir código de acceso para dicha cofradía).	
[3] El usuario introduce el código de acceso de la cofradía.	
[4] El sistema valida el código.	
[5] El sistema muestra mensaje informando del correcto acceso.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Código inválido: El sistema muestra los errores de validación.	
Requisitos Relacionados	RF-02.

Tabla B.2: Caso de uso: Acceso como cofrade

Caso de uso: Añadir favoritos	
Resumen	Un usuario marca elementos como favoritos.
Actor	Ciudadano
Precondición	
Postcondición	El elemento queda guardado de favoritos
Secuencia Base	
[1] El usuario navega a una cofradía, evento o procesión.	
[2] El usuario pulsa el botón de favorito.	
[3] El sistema guarda el elemento como favorito.	
[4] El sistema muestra confirmación visual.	
Secuencia Alternativa	
[2.a.1] El elemento ya es favorito: El sistema lo elimina de favoritos (caso de uso Eliminar favoritos).	
Requisitos Relacionados	RF-12.

Tabla B.3: Caso de uso: Añadir favoritos

Caso de uso: Eliminar favoritos	
Resumen	Un usuario desmarca elementos como favoritos.
Actor	Ciudadano
Precondición	El elemento esté marcado como favorito
Postcondición	El elemento queda eliminado de favoritos
Secuencia Base	
[1] El usuario navega a una cofradía, evento o procesión.	
[2] El usuario pulsa el botón de favorito.	
[3] El sistema quita el elemento de favorito.	
[4] El sistema muestra confirmación visual.	
Secuencia Alternativa	
[2.a.1] El elemento no es favorito: El sistema lo añade a favoritos (caso de uso Añadir favoritos).	
Requisitos Relacionados	RF-12.

Tabla B.4: Caso de uso: Eliminar favoritos

Caso de uso: Configurar notificaciones	
Resumen	Un ciudadano se suscribe o desuscribe de las notificaciones sobre cofradías, eventos o procesiones concretas.
Actor	Ciudadano
Precondición	–
Postcondición	Las preferencias de notificaciones quedan guardadas
Secuencia Base	
[1] El usuario accede a la configuración de notificaciones.	
[2] El sistema muestra las opciones disponibles.	
[3] El usuario activa/desactiva tipos de notificaciones.	
[4] El usuario guarda los cambios.	
[5] El sistema confirma que se han guardado.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
Requisitos Relacionados	RF-22.

Tabla B.5: Caso de uso: Configurar notificaciones

Caso de uso: Crear cofradía/Hermandad	
Resumen	Un miembro de la Junta de Cofradías crea una nueva cofradía.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías
Postcondición	La cofradía queda creada en el sistema
Secuencia Base	
[1] El usuario accede al panel de gestión.	
[2] El usuario selecciona crear nueva cofradía.	
[3] El sistema muestra el formulario de creación.	
[4] El usuario introduce los datos de la cofradía.	
[5] El usuario guarda la cofradía.	
[6] El sistema valida y guarda la información.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[6.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
Requisitos Relacionados	RF-21 y RF-33.

Tabla B.6: Caso de uso: Crear cofradía/Hermandad

Caso de uso: Crear procesión	
Resumen	Un miembro de la Junta de Cofradías crea una nueva procesión.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías y que exista al menos una cofradía
Postcondición	La procesión queda creada en el sistema
Secuencia Base	
[1] El usuario accede al panel de gestión.	
[2] El usuario selecciona crear nueva procesión.	
[3] El sistema muestra el formulario de creación.	
[4] El usuario introduce los datos de la procesión.	
[5] El usuario define el recorrido en el mapa.	
[6] El usuario guarda la procesión.	
[7] El sistema valida y guarda la información.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[7.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
Requisitos Relacionados	RF-21 y RF-30.

Tabla B.7: Caso de uso: Crear procesión

Caso de uso: Actualizar estado de procesión	
Resumen	La Junta de Cofradías actualiza el estado de una procesión.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías y existe una procesión programada o en curso
Postcondición	El estado de la procesión queda actualizado y se notifica a los usuarios
Secuencia Base	
[1] El usuario accede al panel de gestión de procesiones.	
[2] El usuario selecciona la procesión a actualizar.	
[3] El sistema muestra la información actual de la procesión.	
[4] El usuario selecciona el nuevo estado.	
[5] El usuario confirma el cambio.	
[6] El sistema actualiza el estado.	
[7] El sistema envía notificaciones automáticas a los usuarios interesados.	
Secuencia Alternativa	
[2.a.1] Sin conexión: El sistema muestra error de conexión.	
[2.b.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[4.a.1] Sin conexión: El sistema muestra error de conexión.	
[4.b.1] Caso de uso crear alerta: El usuario crea una alerta.	
[4.c.1] Estado no válido: El sistema muestra los estados disponibles según el estado actual.	
[4.d.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Sin conexión: El sistema muestra error de conexión.	
[5.b.1] Cancelación de la operación: El usuario desea cancelar la operación.	
Requisitos Relacionados	RF-28 y RF-27.

Tabla B.8: Caso de uso: Actualizar estado de procesión

Caso de uso: Gestionar información de Semana Santa	
Resumen	La Junta de Cofradías introduce o modifica toda la información de la Semana Santa de su ciudad.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías
Postcondición	La información de la Semana Santa queda actualizada en el sistema
Secuencia Base	
[1] El usuario selecciona el módulo a gestionar (cofradías, procesiones, eventos, pasos).	
[2] El sistema muestra la lista de elementos existentes.	
[3] El usuario puede crear, editar o eliminar elementos.	
[4] El usuario introduce/modifica los datos requeridos.	
[5] El sistema valida la información.	
[6] El sistema guarda los cambios.	
[7] El sistema muestra confirmación.	
Secuencia Alternativa	
[3.a.1] Sin conexión: El sistema muestra error de conexión	
[3.b.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[4.a.1] Sin conexión: El sistema muestra error de conexión.	
[4.b.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[4.c.1] Eliminar elemento con dependencias: El sistema advierte y solicita confirmación.	
[6.a.1] Datos inválidos: El sistema muestra errores de validación.	
Requisitos Relacionados	RF-20, RF-21 y RF-31.

Tabla B.9: Caso de uso: Gestionar información de Semana Santa

Caso de uso: Definir recorrido de procesión	
Resumen	La Junta de Cofradías define o modifica el recorrido de una procesión en el mapa.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías y existe una procesión creada.
Postcondición	El recorrido de la procesión queda definido con todos sus puntos.
Secuencia Base	
[1] El usuario accede a la gestión de procesiones.	
[2] El usuario selecciona la procesión.	
[3] El usuario accede a la edición del recorrido.	
[4] El sistema muestra el mapa interactivo.	
[5] El usuario marca los puntos del recorrido en orden.	
[6] El usuario define horarios estimados para cada puntos.	
[7] El usuario marca puntos de interés especiales.	
[8] El usuario guarda el recorrido.	
[9] El sistema calcula distancia y tiempo estimado.	
Secuencia Alternativa	
[2.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[3.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.2] Recorrido inválido: El sistema indica que debe tener al menos punto de inicio y fin.	
[6.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[7.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[8.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
Requisitos Relacionados	RF-30 y RF-29.

Tabla B.10: Caso de uso: Definir recorrido de procesión

Caso de uso: Dar de alta Junta de Cofradías	
Resumen	El administrador da de alta la Junta de Cofradías encargada de gestionar una ciudad.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador y existe la ciudad a la que se va a asociar la Junta de Cofradías.
Postcondición	La nueva Junta de Cofradías queda creada y asociada a la ciudad seleccionada.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de ciudades existentes.	
[3] El usuario selecciona la ciudad para la que desea crear la Junta de Cofradías.	
[4] El usuario introduce los datos de la Junta de Cofradías.	
[5] El sistema valida y guarda la nueva Junta de Cofradías, asociándola a la ciudad seleccionada.	
Secuencia Alternativa	
[3.a.1] Ciudad con Junta ya asignada: El sistema informa de que la ciudad ya tiene una Junta de Cofradías asociada.	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
Requisitos Relacionados	RF-34.

Tabla B.11: Caso de uso: Dar de alta Junta de Cofradías

Caso de uso: Editar Junta de Cofradías	
Resumen	El administrador modifica los datos de una Junta de Cofradías existente.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador y existe al menos una Junta de Cofradías registrada.
Postcondición	Los datos de la Junta de Cofradías quedan actualizados en el sistema.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de Juntas de Cofradías existentes.	
[3] El usuario selecciona la Junta de Cofradías que desea editar.	
[4] El usuario modifica los datos de la Junta de Cofradías.	
[5] El sistema valida y guarda los cambios.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
Requisitos Relacionados	RF-34.

Tabla B.12: Caso de uso: Editar Junta de Cofradías

Caso de uso: Dar de baja Junta de Cofradías	
Resumen	El administrador retira una Junta de Cofradías del sistema.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador y existe al menos una Junta de Cofradías registrada.
Postcondición	La Junta de Cofradías deja de estar disponible en el sistema y la ciudad asociada queda sin Junta asignada.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de Juntas de Cofradías existentes.	
[3] El usuario selecciona la Junta de Cofradías que desea eliminar.	
[4] El sistema comprueba si la Junta de Cofradías tiene cofradías asociadas.	
[5] El usuario confirma la eliminación.	
[6] El sistema elimina la Junta de Cofradías.	
Secuencia Alternativa	
[4.a.1] Junta con cofradías asociadas: El sistema advierte de que la Junta de Cofradías tiene cofradías registradas antes de continuar.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la eliminación.	
Requisitos Relacionados	RF-34.

Tabla B.13: Caso de uso: Dar de baja Junta de Cofradías

Caso de uso: Editar ciudad	
Resumen	El administrador modifica los datos de una ciudad existente.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador y existe al menos una ciudad registrada.
Postcondición	Los datos de la ciudad quedan actualizados en el sistema.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de ciudades existentes.	
[3] El usuario selecciona la ciudad que desea editar.	
[4] El usuario modifica los datos de la ciudad.	
[5] El sistema valida y guarda los cambios.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Datos incompletos: El sistema muestra los campos requeridos.	
Requisitos Relacionados	RF-26.

Tabla B.14: Caso de uso: Editar ciudad

Caso de uso: Dar de baja ciudad	
Resumen	El administrador retira una ciudad disponible de la aplicación.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador y existe al menos una ciudad registrada.
Postcondición	La ciudad deja de estar disponible en el sistema.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El sistema muestra la lista de ciudades existentes.	
[3] El usuario selecciona la ciudad que desea eliminar.	
[4] El sistema comprueba si la ciudad tiene datos asociados (cofradías, eventos o procesiones).	
[5] El usuario confirma la eliminación.	
[6] El sistema elimina la ciudad.	
Secuencia Alternativa	
[4.a.1] Ciudad con datos asociados: El sistema advierte de que la ciudad tiene cofradías, eventos o procesiones registrados antes de continuar.	
[5.a.1] Cancelación de la operación: El usuario desea cancelar la eliminación.	
Requisitos Relacionados	RF-26.

Tabla B.15: Caso de uso: Dar de baja ciudad

Caso de uso: Consultar calendario de Semana Santa	
Resumen	El ciudadano consulta el calendario con los días de la Semana Santa.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	Se muestra el calendario con los días señalados.
Secuencia Base	
[1] El usuario accede al calendario desde el menú principal.	
[2] El sistema muestra el calendario, indicando los días con eventos o procesiones programadas.	
Secuencia Alternativa	
[2.a.1] Sin conexión: El sistema muestra la última información almacenada en local.	
Requisitos Relacionados	RF-35.

Tabla B.16: Caso de uso: Consultar calendario de Semana Santa

Caso de uso: Consultar procesiones de un día	
Resumen	El ciudadano consulta el listado de procesiones programadas en un día concreto.
Actor	Ciudadano
Precondición	El usuario está consultando el calendario de Semana Santa.
Postcondición	Se muestra el listado de procesiones de ese día.
Secuencia Base	
[1] El usuario selecciona un día en el calendario.	
[2] El sistema muestra el listado de procesiones programadas ese día.	
Secuencia Alternativa	
[2.a.1] Sin procesiones: El sistema indica que no hay procesiones programadas ese día.	
Requisitos Relacionados	RF-36.

Tabla B.17: Caso de uso: Consultar procesiones de un día

Caso de uso: Acceder a listados desde el menú principal	
Resumen	El ciudadano accede a los listados completos de procesiones, pasos, eventos o cofradías.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	Se muestra el listado completo solicitado.
Secuencia Base	
[1] El usuario accede al menú principal.	
[2] El usuario selecciona el tipo de listado (procesiones, pasos, eventos o cofradías).	
[3] El sistema muestra el listado completo correspondiente.	
Secuencia Alternativa	
[3.a.1] Sin elementos disponibles: El sistema muestra un mensaje indicando que no hay elementos de ese tipo.	
Requisitos Relacionados	RF-37.

Tabla B.18: Caso de uso: Acceder a listados desde el menú principal

Caso de uso: Gestionar perfil de la Junta de Cofradías	
Resumen	La Junta de Cofradías consulta o modifica la información de su propio perfil.
Actor	Junta de Cofradías
Precondición	El usuario tiene rol de Junta de Cofradías.
Postcondición	Los datos del perfil quedan actualizados en el sistema.
Secuencia Base	
[1] El usuario accede a la sección de perfil.	
[2] El sistema muestra los datos actuales del perfil.	
[3] El usuario modifica los datos.	
[4] El sistema valida y guarda los cambios.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[4.a.1] Datos inválidos: El sistema muestra los errores de validación.	
Requisitos Relacionados	RF-39.

Tabla B.19: Caso de uso: Gestionar perfil de la Junta de Cofradías

Caso de uso: Gestionar miembros de las Juntas de Cofradías	
Resumen	El administrador da de alta, edita o da de baja a los miembros de una Junta de Cofradías.
Actor	Administrador
Precondición	El usuario tiene rol de Administrador y existe al menos una Junta de Cofradías registrada.
Postcondición	Los miembros de la Junta de Cofradías quedan actualizados en el sistema.
Secuencia Base	
[1] El usuario accede al panel de administración.	
[2] El usuario selecciona la Junta de Cofradías.	
[3] El sistema muestra la lista de miembros de esa Junta.	
[4] El usuario da de alta, edita o da de baja a un miembro.	
[5] El sistema valida y guarda los cambios.	
Secuencia Alternativa	
[4.a.1] Cancelación de la operación: El usuario desea cancelar la operación.	
[5.a.1] Datos inválidos: El sistema muestra los errores de validación.	
Requisitos Relacionados	RF-40.

Tabla B.20: Caso de uso: Gestionar miembros de las Juntas de Cofradías

Apéndice C

Esquema de la base de datos

Este apéndice recoge, a nivel de detalle de campo, el esquema completo de las tablas de PostgreSQL [40] introducidas en el apartado Diseño de la base de datos (Sección 4.1), incluyendo los tipos de dato de cada campo y las claves primarias y foráneas. Los tipos de dato mostrados corresponden a tipos nativos de PostgreSQL (BIGINT, VARCHAR, TEXT, BOOLEAN, TIMESTAMP, DOUBLE PRECISION) o a tipos enumerados propios (ENUM) definidos para este esquema.

C.1. Ciudad (ciudades)

Requisito: RI-10, RI-09

Agrupar las procesiones y actos de cada localidad.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
comunidad_autonoma	VARCHAR	
descripcion	TEXT	

Tabla C.1: Campos de ciudades

C.2. Junta de Cofradías (juntas_cofradias)

Requisito: RI-10

Junta que gestiona una ciudad. Relación 1:1 con ciudades.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
email	VARCHAR	
telefono	VARCHAR	
ciudad_id	BIGINT	FK → ciudades(id), UNIQUE

Tabla C.2: Campos de juntas_cofrarias

C.3. Cofradía / Hermandad (cofradias)

Requisito: RI-02

Asociación religiosa que organiza eventos y procesiones. Pertenece a la ciudad –nunca a la Junta de Cofradías que la gestiona–: Ciudad es quien agrupa estructuralmente a sus cofradías (relación 1:N), mientras que JuntaCofrarias es solo la cuenta de gestión de esa ciudad, sustituible sin afectar a los datos de las cofradías que agrupa.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
web	VARCHAR	URL del sitio propio de la cofradía
fecha_creacion	TIMESTAMP	
ciudad_id	BIGINT	FK → ciudades(id)

Tabla C.3: Campos de cofrarias

C.4. Paso (pasos)

Pasos que posee la cofradía. Relación N:M con procesiones a través de pasos_procesiones.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
analisis_artistico	TEXT	
imagen	VARCHAR	URL a Firebase Storage
cofradia_id	BIGINT	FK → cofrarias(id)

Tabla C.4: Campos de pasos

C.4.1. Paso-Procesión (pasos_procesiones) — tabla intermedia N:M

Campo	Tipo	Nota
paso_id	BIGINT	PK compuesta, FK → pasos(id)
procesion_id	BIGINT	PK compuesta, FK → procesiones(id)

Tabla C.5: Campos de pasos_procesiones

C.5. Código de Acceso (codigos_acceso)

Códigos que emite la Junta de Cofradías en nombre de una cofradía de su ciudad (Cofradia no es un actor con cuenta propia, no puede emitir nada por sí misma). Quien lo valida obtiene un token identificado con esa cofradía y el rol de cofrade, sin que ello genere ningún registro de usuario –ver la explicación completa en la Sección C.10–. El código no se invalida al usarse por primera vez: sigue sirviendo para volver a autenticarse mientras no esté revocado.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
codigo	VARCHAR	UNIQUE, generado por el servidor
estado	ENUM	EMITIDO, VALIDADO, REVOCADO
cofradia_id	BIGINT	FK → cofradias(id)

Tabla C.6: Campos de codigos_acceso

C.6. Ubicación (ubicaciones)

Punto geográfico reutilizable (latitud, longitud y dirección textual), referenciado por eventos y por puntos_ruta. Se modela como tabla independiente, en lugar de como columnas repetidas en cada entidad que necesita una posición, para poder compartir la misma ubicación entre varias entidades –por ejemplo, la sede de una cofradía y un punto de interés del recorrido que coincidan en el mismo templo– sin duplicar datos.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
latitud	DOUBLE PRECISION	
longitud	DOUBLE PRECISION	
direccion	VARCHAR	

Tabla C.7: Campos de ubicaciones

C.7. Evento (eventos)

Requisito: RI-03

Acto que puede tener más de una cofradía participando (por ejemplo, un Vía Crucis conjunto o una procesión con pasos de varias hermandades): la relación con cofradías es N:M, no 1:N, resuelta mediante la tabla intermedia eventos_cofradías. Evento es además la tabla raíz de la que procesiones hereda sus columnas comunes (Sección C.8).

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
tradicion	TEXT	
fecha	TIMESTAMP	
estado	ENUM	PROGRAMADO, EN_CURSO, FINALIZADO, CANCELADO
ubicacion_id	BIGINT	FK → ubicaciones(id)

Tabla C.8: Campos de eventos

C.7.1. Evento-Cofradía (eventos_cofradías) — tabla intermedia N:M

Campo	Tipo	Nota
evento_id	BIGINT	PK compuesta, FK → eventos(id)
cofradia_id	BIGINT	PK compuesta, FK → cofradías(id)

Tabla C.9: Campos de eventos_cofradías

C.8. Procesión (procesiones)

Requisito: RI-04

A diferencia del planteamiento inicial, en el que procesiones referenciaba a eventos mediante una clave foránea 1:N, la traducción final a PostgreSQL usa **herencia real de tablas** (estrategia JOINED de JPA): *Procesion* es un *Evento* –hereda nombre, historia, tradicion, fecha, estado, ubicacion_id y sus cofradías participantes (eventos_cofradías)– y añade únicamente sus columnas propias en esta tabla. El id de procesiones es a la vez su clave primaria y una clave foránea a eventos(id): al guardar una procesión, el motor inserta primero la fila común en eventos y reutiliza el identificador generado para la fila de procesiones, en lugar de generar uno nuevo. Esta decisión evita mantener columnas duplicadas (cofradia_id, estado...) sincronizadas manualmente entre las dos tablas.

Campo	Tipo	Nota
id	BIGINT	PK y FK → eventos(id)
fecha_inicio	TIMESTAMP	
fecha_fin	TIMESTAMP	
recorrido_id	BIGINT	FK → recorridos(id), UNIQUE (1:1), opcional –puede programarse una procesión antes de tener ruta definida–

Tabla C.10: Campos de procesiones

C.8.1. Posiciones actuales (posiciones_actuales) — histórico de pings anónimos

A diferencia del planteamiento inicial, esta tabla no guarda una única fila por procesión que se sobrescribe con cada actualización (UPSERT), sino un **histórico de lecturas GPS sueltas** (*pings*): cada vez que un cofrade comparte su ubicación se inserta una fila nueva, sin sobrescribir ninguna anterior. Cada fila es además **anónima**: no lleva ninguna referencia a la persona que la envió, solo a la procesión (Sección C.10 explica por qué). La “posición actual” de una procesión ya no es un dato guardado como tal, sino un valor **calculado** en el momento de la consulta: el promedio de latitud y longitud de los *pings* recibidos en los últimos 60 segundos, siendo el número de *pings* recientes una aproximación al número de cofrades compartiendo ubicación en ese instante. La aplicación cliente consulta este valor calculado mediante la API propia (GET /procesiones/{id}/ubicacion) con la cadencia que exige el requisito RNF-08, tal y como se explica en el apartado Decisiones tecnológicas.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
latitud	DOUBLE PRECISION	
longitud	DOUBLE PRECISION	
timestamp	TIMESTAMP	instante de la lectura
procesion_id	BIGINT	FK → procesiones(id)

Tabla C.11: Campos de posiciones_actuales

C.9. Recorrido (recorridos)

Requisito: RI-05, RI-06

Itinerario compuesto por puntos de ruta ordenados. Sin campo nombre: un recorrido se identifica por la procesión a la que pertenece, no por un nombre propio.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
distancia_total	DOUBLE PRECISION	metros
tiempo_estimado	INTEGER	minutos

Tabla C.12: Campos de recorridos

C.9.1. Punto de Ruta (puntos_ruta)

A diferencia del planteamiento inicial, un mismo punto de ruta puede formar parte de **varios** recorridos –varias procesiones pueden compartir un mismo tramo de calle o pasar por el mismo templo–, así que puntos_ruta ya no referencia directamente a un recorrido: la relación es N:M y se resuelve en la tabla intermedia recorrido_puntos_ruta (Sección C.9.3). PuntoRuta es además la tabla raíz de una segunda jerarquía de herencia real de tablas: un punto “de paso” simple se guarda directamente aquí, mientras que un punto “de interés” con detalle añade sus columnas propias en puntos_de_interes (Sección C.9.2).

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
ubicacion_id	BIGINT	FK → ubicaciones(id)

Tabla C.13: Campos de puntos_ruta

C.9.2. Punto de Interés (puntos_de_interes)

Subtipo de PuntoRuta con detalle: nombre, descripción, imagen y una categoría (RI-06). El tipo enumerado se llama TipoPuntoInteres –es una categoría de punto de interés, no de cualquier punto de ruta– e incluye tanto categorías de interés turístico o religioso como puntos operativos del propio recorrido de una procesión (inicio y fin de la salida procesional, o el lugar donde se celebra el evento asociado).

Campo	Tipo	Nota
id	BIGINT	PK y FK → puntos_ruta(id)
tipo	ENUM	MONUMENTO, IGLESIA, ENCuentro, ORACION, ENTRADAPROCESION, SALIDAPROCESION, UBICACIONEVENTO
nombre	VARCHAR	
descripcion	TEXT	
imagen	VARCHAR	URL a Firebase Storage

Tabla C.14: Campos de puntos_de_interes

C.9.3. Recorrido-Punto de Ruta (recorrido_puntos_ruta) — tabla intermedia N:M

El orden de un punto dentro de un recorrido y la hora prevista de paso ya no son propiedades del punto en sí (un mismo punto puede estar en la posición 3 de un recorrido y en la posición 7 de otro, a horas distintas), sino de **esta relación**. Lleva un identificador propio en lugar de una clave primaria compuesta –a diferencia de pasos_procesiones o eventos_cofrarias– porque, además de las dos claves foráneas, tiene columnas propias; dos restricciones UNIQUE garantizan que un punto no se repita dentro del mismo recorrido y que dos puntos no compartan posición.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
recorrido_id	BIGINT	FK → recorridos(id)
punto_ruta_id	BIGINT	FK → puntos_ruta(id)
orden	INTEGER	UNIQUE junto con recorrido_id
hora_prevista	TIMESTAMP	hora prevista de paso

Tabla C.15: Campos de recorrido_puntos_ruta

C.10. Usuario (usuarios)

Requisito: RI-01

A diferencia del planteamiento inicial, en el que usuarios era una única tabla con una columna rol que distinguía entre cofrade, junta de cofradías y administrador, la traducción final a PostgreSQL usa de nuevo **herencia real de tablas**: usuarios es la tabla raíz, con las columnas que de verdad comparten sus subtipos, y cada subtipo añade sus propias columnas en su propia tabla. Sin columna rol: con herencia real el tipo de usuario lo determina en qué tabla de subtipo tiene fila, no un valor guardado.

El cambio más relevante respecto al planteamiento inicial es que **el cofrade ha dejado de ser un Usuario**: ya no se guarda ninguna fila –ni en usuarios ni en ninguna tabla propia– para las personas que comparten ubicación durante una procesión. Validar un código de acceso (Sección C, codigos_acceso) no registra a nadie: el servidor comprueba que el código no está revocado y devuelve directamente un token JWT identificado con la **cofradía** cuyo código se validó, no con una persona. Esta decisión responde a que la aplicación no necesita conocer la identidad de cada cofrade para cumplir su función –agregar y mostrar la posición de una procesión–, y evita cargar el sistema con el registro de personas cuya participación es, además, estacional (solo durante los días de procesión). Cofrade sigue existiendo como clase en el modelo de dominio (Figura 3.5) y en el código del backend, pero como una representación construida en memoria a partir del token en cada petición, nunca persistida.

Por tanto, usuarios solo contiene los perfiles con credenciales propias –junta de cofradías y administrador–, que sí se autentican con email y una contraseña almacenada como *hash*¹ mediante BCrypt, el algoritmo utilizado por defecto por Spring Security [43]. El ciudadano tampoco se registra, igual que en el planteamiento inicial.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
email	VARCHAR	UNIQUE
password_hash	VARCHAR	<i>hash</i> BCrypt
fecha_ingreso	TIMESTAMP	

Tabla C.16: Campos de usuarios

¹Hash: transformación irreversible de una contraseña en una cadena de caracteres, de forma que el sistema pueda comprobar que una contraseña es correcta sin necesidad de guardarla en texto plano.

C.10.1. Miembro de Junta de Cofradías (miembros_junta_cofradia)

Persona con credenciales que pertenece a la Junta de Cofradías de una ciudad –una Junta puede tener varios miembros, cada uno con su propio inicio de sesión–. La referencia es a la JuntaCofradias, no directamente a la Ciudad: así se reutiliza la relación 1:1 ya existente entre JuntaCofradias y Ciudad en lugar de duplicarla, y de ahí –miembro.juntaCofradias.ciudad– es de donde la capa de servicio de la API obtiene qué ciudad gestiona cada miembro a la hora de comprobar la autorización por rol (RNF-05).

Campo	Tipo	Nota
id	BIGINT	PK y FK → usuarios(id)
junta_cofradias_id	BIGINT	FK → juntas_cofradias(id)

Tabla C.17: Campos de miembros_junta_cofradia

C.10.2. Administrador (administradores)

Rol global del sistema, sin ninguna columna propia ni referencia a Ciudad: cualquier administrador puede gestionar cualquier ciudad, no tiene sentido atarlo de antemano a una que todavía no exista.

Campo	Nota
id	PK y FK → usuarios(id); sin columnas propias

Tabla C.18: Campos de administradores

C.11. Notificación (notificaciones)

Requisito: RI-08

Dos tipos: Aviso (informativo, con expiración) y Alerta (con tipo y prioridad). Pendiente de implementar en el backend.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
tipo	ENUM	AVISO, ALERTA
titulo	VARCHAR	
fecha_creacion	TIMESTAMP	
fecha_expiracion	TIMESTAMP	solo Aviso, nulo en caso contrario
tipo_alerta	ENUM	solo Alerta: INCIDENCIA, CAMBIO_HORARIO, CANCELACION, CORTE_CALLE, METEOROLOGIA, SEGURIDAD
prioridad	ENUM	solo Alerta: BAJA, MEDIA, ALTA, URGENTE

Tabla C.19: Campos de notificaciones

C.11.1. Notificación Entregada (notificaciones_entregadas) — tabla intermedia N:M

Registra si el usuario ha leído cada notificación y cuándo. Pendiente de implementar en el backend; su columna `usuario_id` deberá revisarse cuando se implemente, dado que `usuarios` ya no incluye al cofrade (Sección C.10).

Campo	Tipo	Nota
<code>usuario_id</code>	BIGINT	PK compuesta, FK → <code>usuarios(id)</code>
<code>notificacion_id</code>	BIGINT	PK compuesta, FK → <code>notificaciones(id)</code>
<code>leida</code>	BOOLEAN	
<code>fecha_lectura</code>	TIMESTAMP	

Tabla C.20: Campos de `notificaciones_entregadas`**C.12. Relaciones entre tablas**

La Tabla C.21 recoge las relaciones entre tablas de este esquema, todas ellas garantizadas por claves foráneas con restricción de integridad referencial.

De	A	Relación	Cardinalidad
<code>ciudades</code>	<code>cofradias</code>	agrupa	1:N
<code>ciudades</code>	<code>juntas_cofradias</code>	gestionada por	1:1
<code>juntas_cofradias</code>	<code>miembros_junta_cofradia</code>	tiene	1:N
<code>cofradias</code>	<code>eventos</code>	participa en	N:M
<code>cofradias</code>	<code>pasos</code>	posee	1:N
<code>cofradias</code>	<code>codigos_acceso</code>	emite	1:N
<code>eventos</code>	<code>procesiones</code>	es extendido por (herencia)	1:1
<code>eventos</code>	<code>ubicaciones</code>	se celebra en	N:1
<code>procesiones</code>	<code>recorridos</code>	tiene	1:1
<code>procesiones</code>	<code>posiciones_actuales</code>	recibe	1:N
<code>recorridos</code>	<code>puntos_ruta</code>	compuesto por	N:M
<code>puntos_ruta</code>	<code>puntos_de_interes</code>	es extendido por (herencia)	1:1
<code>puntos_ruta</code>	<code>ubicaciones</code>	está en	N:1
<code>pasos</code>	<code>procesiones</code>	desfila en	N:M
<code>miembros_junta_cofradia</code>	<code>usuarios</code>	es extendido por (herencia)	1:1
<code>administradores</code>	<code>usuarios</code>	es extendido por (herencia)	1:1
<code>notificaciones</code>	<code>usuarios</code>	se entrega a	N:M

Tabla C.21: Relaciones entre tablas de PostgreSQL

Apéndice D

Manual de despliegue

Este apéndice describe los pasos necesarios para desplegar el backend propio (API Spring Boot y base de datos PostgreSQL) en Railway [35], así como la configuración necesaria en el cliente React Native y en el proyecto de Firebase para que la aplicación completa funcione en un entorno nuevo.

D.1. Requisitos previos

- Cuenta en Railway [35] vinculada al repositorio de GitHub [20] del backend.
- Cuenta de Firebase con un proyecto creado, con los servicios **Storage** y **Cloud Messaging** habilitados (ya no es necesario habilitar Firestore ni Authentication).
- Clave de API de Google Maps Platform [1] con las APIs de *Maps SDK* y *Directions* activadas.
- Java 17 o superior y Node.js instalados en el entorno de desarrollo local.

D.2. Despliegue del backend en Railway

1. Crear un nuevo proyecto en Railway y añadir dos servicios: uno de tipo **PostgreSQL** (Railway lo aprovisiona automáticamente) y otro de tipo **despliegue desde GitHub**, apuntando al repositorio del backend.
2. Railway detecta automáticamente que se trata de un proyecto Spring Boot (Maven/Gradle) y genera la imagen de despliegue sin necesidad de un `Dockerfile` propio, aunque puede añadirse uno para tener un control más explícito del proceso de construcción.
3. Configurar las siguientes variables de entorno en el servicio del backend:
 - `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD`: Railway las expone automáticamente al enlazar el servicio de PostgreSQL con el del backend.

- `JWT_SECRET`: clave secreta utilizada para firmar los tokens JWT (Sección 4.1). Debe ser una cadena aleatoria de suficiente longitud y no debe compartirse ni subirse al repositorio.
 - `JWT_EXPIRATION`: tiempo de validez del token, en milisegundos.
 - `FIREBASE_CREDENTIALS`: contenido (o ruta) del fichero de credenciales de servicio de Firebase, necesario para que el backend pueda usar el Firebase Admin SDK [46] y así disparar notificaciones *push*.
4. Al desplegar, Spring Boot ejecuta automáticamente las migraciones del esquema de base de datos (Apéndice C) sobre la instancia de PostgreSQL antes de arrancar la API.
 5. Railway asigna un dominio público HTTPS al backend (por ejemplo, `https://{proyecto}.up.railway.app`), que será la URL base que consumirá la aplicación cliente.
 6. Cada nuevo *commit* en la rama principal del repositorio dispara automáticamente un nuevo despliegue (integración continua), sin intervención manual.

D.3. Documentación de la API

Una vez desplegado el backend, la documentación interactiva de la API, generada automáticamente por `springdoc-openapi` [50] a partir del propio código, queda disponible en `/swagger-ui.html` sobre el dominio del backend, y permite consultar y probar cada *endpoint* sin necesidad de herramientas adicionales.

D.4. Configuración del cliente React Native

1. En el fichero de configuración de entorno del cliente (por ejemplo, `.env`), establecer la URL base de la API desplegada en Railway (`API_BASE_URL`).
2. Añadir la clave de API de Google Maps Platform en la configuración nativa del proyecto (`AndroidManifest.xml` en Android, `Info.plist` en iOS), según la documentación oficial de `react-native-maps`.
3. Añadir el fichero de configuración de Firebase (`google-services.json` en Android, `GoogleService-Info.plist` en iOS) descargado desde la consola de Firebase, necesario únicamente para inicializar los SDK de Storage y Cloud Messaging.

D.5. Generación del ejecutable de la aplicación móvil

La generación del paquete de instalación (`.apk/.aab` para Android) se realiza mediante las herramientas estándar de Expo/React Native (`eas build`), siguiendo la documentación oficial de React Native [36], sin diferencias respecto a un proyecto que no consumiera un backend propio.

Apéndice E

Manual de uso

Apéndice F

Contenido del TFG

